

Chapter 1.

Introduction

1.1 Background of the Study

Traditionally, plant nurseries was mostly depending on manual record keeping, in person sales and paper based inventory tracking. As the nursery grow bigger, maintaining accurate stock levels, managing customer orders handling plant care enquiries and coordinating deliveries becomes more difficult and confusing. In today's digital landscape customers expects the convenience of browsing plants online reading reviews asking questions before purchase and getting doorstep delivery too.

Plantify is a full stack plant nursery management application which was developed to solve these challenge's. It provide a centralised digital platform where nursery businesses can manage their plant inventory, process customer orders, facilitate specialist consultations coordinate deliveries and access business insight's through analytics all by using a modern web interface. The application serve four distinct user roles which are Customers, Specialists, Delivery Personnel and Administrators.

This report document's the project management and planning aspects of the Plantify application covering risk management, project estimation, function point analysis scheduling and cost estimation.

1.2 Methodology

1.2.1 Data Sources

The project planning and estimation for Plantify was drawn from five primary data source's:

- **Project Codebase Analysis:** Direct examination of the Plantify source code including backend controllers, models, routes database migrations and frontend components.
- **Database Schema:** Analysis of all 8 database tables such as users, plants, categories, orders, order_items, reviews, questions and wishlists from SQL dump and Laravel migration files.
- **API Route Analysis:** Inventory of more then 20 API endpoints across Auth, Plants Orders, Reviews, Questions, Wishlists Categories and Upload modules.

- **Project Documentation:** Review of documents like README.md, ARCHITECTURE_FIX_SUMMARY.md VERIFICATION_REPORT.md, Project_Tree.md and configuration files.
- **Industry Standards & Market Research:** Application of IFPUG Function Point Analysis standards for software sizing standard software engineering project management practices for risk analysis and cost estimation plus Bangladesh/regional salary data for estimating personnel cost.

1.3 Objectives

1.3.1 Broad Objective

To develop a comprehensive project management and planning report for the Plantify full stack plant nursery management application covering risk identification and mitigation software size estimation using Function Point Analysis, project scheduling and cost estimation in Bangladeshi Taka (TK).

1.3.2 Specific Objectives

The specific objectives of this report is:

1. **Risk Management:** Identify analyse and plan mitigation strategies for all potential technical operational schedule and external risks which could impact the Plantify project.
2. **Software Size Estimation:** Apply Function Point Analysis methodology to measure the functional size of Plantify based on External Inputs, External Outputs External Inquiries Internal Logical Files and External Interface Files.
3. **Complexity Analysis:** Determine the complexity of each transaction and data function based on Data Element Types (DETs), Record Element Types (RETs) and File Types Referenced (FTRs).
4. **Effort and Duration Estimation:** Calculate Adjusted Function Points estimate required person months of effort and determine project duration based on team size with productivity benchmarks.

5. **Project Scheduling:** Create a detailed 13 week Gantt chart with 4 sprints covering all development phases from planning through testing and deployment.
6. **Cost Estimation:** Prepare a comprehensive cost breakdown covering personnel hardware software and other expenses in Bangladeshi Taka.
7. **Risk Mitigation Planning:** Develop a Risk Mitigation Monitoring and Management (RMMM) plan for the highest priority risks including defined triggers owners contingencies and success criteria.

1.4 Process Model

1.4.1 Reason for Choosing the Required Process Model

The Plantify project follows a Agile Process Model using Scrum inspired sprint based incremental development. Although it was intended for a single developer and academic context this approach was chosen because of several reasons.

- **Modular Architecture:** The application has clear separable modules like Auth, Plants Orders and Reviews which can be developed incrementally in different sprints allowing the developer to focus on one module at a time without context switching overhead.
- **User Role Complexity:** Four distinct user roles requires iterative refinement, working in sprints helps the developer perfect role specific views and permissions sequentially.
- **Evolving Requirements:** E commerce features like cart checkout and wishlist benefits from iterative feedback. In university setting informal peer or supervisor feedback can be gathered after each sprint and incorporated immediately.
- **Technology Stack:** Laravel and Next.js supports rapid prototyping and incremental feature delivery which aligns perfectly with agile practices.
- **Personal Sprint Discipline:** Even as a solo developer simple tools like GitHub Project board can simulate sprint planning daily self standups and retrospectives preventing procrastination and ensuring consistent progress.

- **Risk Mitigation:** The iterative nature of agile allow early detection and mitigation of risks like authentication security and cart state management within early sprints without requiring team alignment.

1.4.2 Sprint Structure

The project is organised into 4 sprints over 13 weeks:

- **Sprint 1 (Weeks 1–3):** Planning & Backend Foundation — project setup database design Laravel API for Auth Plants Categories Upload, Next.js initialisation and plant browsing UI.
- **Sprint 2 (Weeks 4–6):** Frontend Core & E Commerce — login/register pages plant detail page cart checkout orders wishlist and profile management.
- **Sprint 3 (Weeks 7–9):** User Features & Social — reviews questions & answers specialist dashboard delivery role category management and PHPUnit tests for critical controllers.
- **Sprint 4 (Weeks 10–13):** Administration Testing & Deployment — RBAC middleware admin dashboard analytics integration testing security hardening and final documentation.

1.4.3 Scrum Ceremonies (per Sprint)

Each sprint includes four key ceremonies which are Sprint Planning to define backlog and estimate tasks, Daily Standups for tracking progress and blockers Sprint Review to demonstrate completed work and gather feedback plus Sprint Retrospective to reflect on the process and identify improvements for the next sprint.

1.5 Feasibility Study

1.5.1 Technical Feasibility

The project is technically feasible. The technology stack Laravel 12 (PHP 8.2+), Next.js 16 with React 19 and TypeScript plus MySQL consist of mature and well documented frameworks with extensive community support. The RESTful API architecture is suitable for more than 20 endpoints covering CRUD operations. Image upload using Laravel filesystem is

a proven pattern. The only identified gap which is absence of authentication middleware are straightforward to solve using Laravel Sanctum and the required developer skill set is widely available.

1.5.2 Economic Feasibility

The project is economically feasible. Development is performed by a student as part of academic coursework which incur zero personnel cost. All software tools like Laravel, Next.js MySQL Tailwind CSS VS Code and Git are free and open source. The only possible monetary outlay is a development laptop approximately 60,000 TK one time or 0 TK if already owned and optional domain name costing around 1,800 TK yearly. The total investment range from 3,800 TK to 63,800 TK while the academic portfolio and career readiness benefits far outweigh the cost.

1.5.3 Operational Feasibility

The project is operationally feasible because the intuitive web interface with role specific dashboards require minimal training. As a standalone system without third party integrations it runs on standard LAMP/LEMP stack. Maintenance is simplified by Laravel database migrations and modular codebase. The system can scale vertically or horizontally as business grows database backup are provided through SQL dumps and there is no special regulatory requirements for nursery e commerce application. It can operate 24/7 with admin and delivery roles managing orders asynchronously.

1.6 Structure of the Report

This report are organised into ten chapters each addressing different aspects of project management and planning for the Plantify application.

Chapter 1 – Introduction presents the background of study methodology and data sources broad and specific objectives rationale for choosing Agile process model and feasibility analysis covering technical economic and operational dimensions.

Chapter 2 – Organisation Overview introduce Globix Tech the hosting software development firm including its vision mission services values leadership structure and the intern's role within the organisation.

Chapter 3 – Requirement Engineering document the requirement analysis process detailed user requirements for all four roles such as Customer Specialist Delivery Personnel and Admin, system requirements hardware and software functional and non functional requirements and the use case diagram of system.

Chapter 4 – Analysis describe the software analysis patterns employed including MVC RESTful API client server and component based architecture followed by activity diagrams swim lane diagrams and class diagram of Plantify system.

Chapter 5 – Project Management covers risk identification risk analysis risk planning and complete Risk Mitigation Monitoring and Management plan for the eight identified risks.

Chapter 6 – Project Planning present the system project estimation using IFPUG Function Point Analysis including complexity identification unadjusted and adjusted function point counts effort and duration estimation a 13 week project schedule (Gantt chart) and detailed cost estimation in Bangladeshi Taka.

Chapter 7 – Designing discusses interface design principles colour palette typography and provides full page level interface designs. It also include data flow diagrams Level 0 Level 1 and Level 2 for major subsystems database normalisation entity relationship model and complete database table structure.

Chapter 8 – System Quality and Testing outlines the software quality management process defect tracking and present 60 detailed test cases with PASS/FAIL status across authentication plant management orders reviews wishlist file upload frontend components user flows admin features and edge cases.

Chapter 9 – Ethical Consideration examines data privacy and security intellectual property user impact bias and fairness responsibility to stakeholders professional ethics IEEE/ACM and the three pillars of sustainability economic social and environmental.

Chapter 10 – Conclusion provides brief overview of the project summarises the proposed system benefits acknowledges limitations of current prototype reflects on the value of practicum at Globix Tech and presents a three horizon future plan for evolving Plantify into a production ready system.

Chapter 2.

Organization Overview

2.1 Organization Introduction

Globix Tech is an IT and software development firm in Uttara Dhaka, Bangladesh. Founded in 2020, the company rapidly establish itself as a hub for digital shift and it was focusing on build high performance, secure and affordable software solutions for local and international markets, and it will keeps growing pretty fast. Globix Tech's history of delivering over 100 different projects and more than 80 satisfied clients focused on connecting between complex industry needs and user-friendly technology.

It has a great commitment towards “Leading Software Innovation” and is focusing particularly on the development of E-commerce educational and initiative management systems for businesses in Bangladesh. Globix Tech enable local businesses to compete on a global level using digital first strategies, it was blending industry know-how with contemporary architectural best practices and sometime it blends them very smooth, so companies will gets more stronger in market.

2.1.1 Organization Vision

To be the global leader in smart software solutions for education and business environments that is known specifically for innovative solutions, absolute reliability and commitment to exceptional customer service.

2.1.2 Organization Mission

To build software ecosystems that are reliable, efficient and vastly scalable, and allow clients to confidently make their way through the rapidly changing digital landscape.

2.1.3 Organization Services

Software Development – Development of scalable multi salesperson marketplaces, Learning Management Systems (LMS) and bespoke enterprise web applications built with React, Node.js, PHP Laravel and Python.

AI & Data Intelligence – Designing NLP based AI Chatbots and Smart Business Assistants for ERP/CRM for projecting analytics.

AWS/Azure – Application Development – Providing advanced threat detection, penetration testing and cloud migration services.

R&D Wing (Innovation Lab) – A research centre dedicated to the development of new technologies such as Blockchain and other advanced Artificial Intelligence technologies, which helps in academic publications and technology validation.

2.1.4 Organization Values

- **Customer Centric Approach** – Taking a customer-centric approach so that the unique needs and feedback of each customer is given priority so that maximum ROI and satisfaction can be achieved.
- **Proven Track Record** – Years of proven expertise in delivering new and innovative technologies.
- **Transparency & Integrity** – clear and honest communication and cost model in all interactions with clients without hidden fees.
- **Collaboration & Teamwork** – Developing partnerships and teams working dynamic where different team comes together to address complex industrial problems.

2.1.5 Organization Structure

- **Executive Management** – The Founder & Managing Director and CEO tasked with making the best decisions and vision alignment.
- **Web & Software Development Division** – Specialized teams of Frontend (Next.js/React) and Backend (Laravel/Node.js) developers.
- **AI & Research Wing** – works specifically on integrating chatbots using NLP and investigating new technologies.

- **Infrastructure & Security Unit** – Deploying cloud infrastructure (AWS/Azure) and making sure applications from clients are cyber secure.
- **Customer Success & Operations** – Managing client relationships project management and after deployment support.

2.2 Leadership and Management

Globix Tech's leadership team is dedicated to the digitalization of Bangladesh and this is the main reason for the success of the company.

2.2.1 Nasir Uddin (Founder & Managing Director)

In 2020, he established Globix Tech to access the technology gap for local businesses at an affordable world class technology price. In his hands, the firm turned from a beginner company to a trusted partner of hundreds of clients who value deep knowledge of their client, quality and integrity.

2.2.2 Syed Mominul Islam (CEO)

He has a philosophy that technology should enable he makes sure to provide spontaneous solutions to various businesses. He leads the company's research and development efforts to help clients stay ahead of the curve on global trends, and to ensure that high-end digital tools are both accessible and cost-effective.

The student will describe their role as a software engineer intern. The student will explain the role of a software engineering intern.

2.2.3 My Position: Software Engineering Intern

I have been on-boarded with the Web Development team using contemporary frameworks such as Laravel and Next.js and have been working with experienced architects on real-world ecommerce challenges.

2.2.4 Organization Address

Uttara, Room # House-30 Road-11 Dhaka-1100, Bangladesh.

Chapter 3.
Requirement Engineering

3.1 Requirement Analysis

Requirement analysis is the process of identifying, documenting and validating the needs and constraint of the Plantify system. The analysis was conducted through a thorough inspection of the existing codebase including backend controllers, models, routes, migrations and frontend components alongside a detailed review of the 8 table database schema and the relationships it defines. All 20+ REST API endpoints was catalogued and map to user facing functionality while the four distinct user roles and their access boundaries were examined carefully, and it will also help in future when system scaling happens so structure stays more clean and manageable. Existing project documentation such as README, architecture summaries and verification reports was also analysed to understand the implemented features and inform the complete requirements picture.

3.1.1 Key Findings from Requirement Analysis

- **Authentication** – Basic registration and login endpoints exist but no token based authentication middleware has been implemented and all API routes are currently unprotected.
- **E Commerce Flow** – Cart browsing checkout and order placement are functional on the frontend while order management is handled through dedicated backend controllers.
- **Plant Management** – Full CRUD operations exists for plants categories reviews and questions.
- **User Management** – Administrators can create read update and delete all user accounts while users can update their own profile information.
- **File Upload** – Image upload for plant thumbnails are implemented with proper MIME type and file size validation.
- **Role Based Access** – A role column exist in the User model but no middleware enforce access restrictions at API level leaving privileged endpoints open to authenticated users.

3.2 Requirement Engineering

3.2.1 User Requirements

Customer (End User) Requirements

- **UR-01 User Registration** – As a customer I want to create an account so that I can place orders and manage my profile.
- **UR-02 User Login** – As a customer I want to log in to my account so that I can access orders wishlist and profile.
- **UR-03 Browse Plants** – As a customer I want to browse available plants so that I can discover what is available for sale.
- **UR-04 Filter Plants** – As a customer I want to filter plants by category price range and rating so that I can find specific plants quickly.
- **UR-05 View Plant Details** – As a customer I want to view detailed information about a plant including images description care tips price and stock so that I can make informed purchase decisions.
- **UR-06 Manage Cart** – As a customer I want to add plants into a shopping cart adjust quantities and remove items before checkout.
- **UR-07 Checkout** – As a customer I want to proceed through checkout with my delivery address so that I can complete purchases successfully.
- **UR-08 View Order History** – As a customer I want to see my previous orders and their status so that I can track purchases.
- **UR-09 Leave Reviews** – As a customer I want to leave rating and reviews for purchased plants so I can share experiences with others.
- **UR-10 Ask Questions** – As a customer I want to ask questions about plants so that I can receive more information before buying.
- **UR-11 Manage Wishlist** – As a customer I want to add or remove plants from wishlist for future consideration.
- **UR-12 Manage Profile** – As a customer I want to update my name email phone number and delivery address so my information stays updated.
- **UR-13 Change Password** – As a customer I want to change my account password so that account security can be maintained.

Specialist Requirements

- **UR-14 Create Plant Listings** – As a specialist I want to create new plant listings with images descriptions prices and care tips so plants are available for customers.
- **UR-15 Edit Plant Listings** – As a specialist I want to update existing plant listings so information remains accurate and current.
- **UR-16 Delete Plant Listings** – As a specialist I want to remove plant listings which are no longer available.
- **UR-17 Manage Categories** – As a specialist I want to create edit and delete categories so that plants are organised properly.
- **UR-18 Answer Customer Questions** – As a specialist I want to answer customer questions regarding plants so customers can receive expert advice.

Delivery Personnel Requirements

- **UR-19 View Assigned Orders** – As a delivery person I want to view assigned orders so I know which deliveries needs to be fulfilled.
- **UR-20 Update Order Status** – As a delivery person I want to update order delivery status such as shipped or delivered so that admins and customers can track progress.

Admin Requirements

- **UR-21 Manage All Users** – As an admin I want to create view update and delete user accounts so the system user base can be managed efficiently.
- **UR-22 View All Orders** – As an admin I want to view all orders from all customers so that the complete order pipeline can be monitored.
- **UR-23 Manage Plants & Categories** – As an admin I want to manage plant listings and categories so product catalogue remains accurate.
- **UR-24 View Analytics Dashboard** – As an admin I want to view analytics like total users orders revenue and top sellers so data driven decisions can be made.

3.2.2 System Requirements

Hardware Requirements

- **Development Machine** – Dual core processor 8GB RAM and 256GB SSD minimum requirement.
- **Production Server (VPS)** – 1 vCPU, 2GB RAM and 50GB SSD minimum.
- **Client Browser** – Modern browser supporting ES6+ such as Chrome Firefox Edge and Safari.
- **Network** – Broadband internet connection minimum 10 Mbps.

Software Requirements

- **Backend Framework** – Laravel 12 with PHP 8.2 or higher.
- **Frontend Framework** – Next.js 16 with React 19 and TypeScript.
- **Styling** – Tailwind CSS v4.
- **Database** – MySQL 8.0+.
- **Web Server** – Apache through XAMPP or Nginx.
- **PHP Version** – 8.2 or above.
- **Node.js Version** – 18.x or higher.
- **Package Managers** – Composer for PHP and npm for Node.js.
- **IDE / Editor** – Visual Studio Code recommended.
- **Version Control** – Git and GitHub.

3.2.3 Functional Requirements

Authentication & User Management

- **FR-01** The system shall allow users to register using name email phone and password.
- **FR-02** The system shall allow users to login using email and password.
- **FR-03** The system shall implement token based authentication using Sanctum for API security.
- **FR-04** The system shall allow users to update profile information including name email phone and address.

- **FR-05** The system shall allow users to change account password.
- **FR-06** The system shall allow admins to create read update and delete any user account.
- **FR-07** The system shall enforce role based access control on all API endpoints.

Plant Management

- **FR-08** The system shall allow specialists to create plant listings with name scientific name category price thumbnail stock status description and care tips.
- **FR-09** The system shall allow specialists to update existing plant listings.
- **FR-10** The system shall allow specialists to delete plant listings.
- **FR-11** The system shall allow users to browse and filter plants using various criteria.
- **FR-12** The system shall display detailed plant information including images ratings and customer questions.

E Commerce & Orders

- **FR-13** The system shall allow customers to add plants into shopping cart.
- **FR-14** The system shall persist cart state across page refreshes using localStorage.
- **FR-15** The system shall allow customers to proceed through checkout using delivery address.
- **FR-16** The system shall create order records with associated order items after successful checkout.
- **FR-17** The system shall allow customers to view their order history.
- **FR-18** The system shall allow delivery personnel to update order status.

Reviews & Questions

- **FR-19** The system shall allow customers to submit reviews with rating and comments for purchased plants.
- **FR-20** The system shall display average ratings and review count for every plant.
- **FR-21** The system shall allow customers to ask questions about plants.
- **FR-22** The system shall allow specialists to answer customer questions.

Wishlist

- **FR-23** The system shall allow customers to add plants into wishlist.
- **FR-24** The system shall allow customers to remove plants from wishlist.
- **FR-25** The system shall allow customers to view all wishlist items.

Categories

- **FR-26** The system shall allow specialists and admins to create new categories.
- **FR-27** The system shall allow specialists and admins to update existing categories.
- **FR-28** The system shall allows specialists and admins to delete categories when needed.
- **FR-29** The system shall display all categories for browsing purpose and users can browse them easily.

File Upload

- **FR-30** The system shall allow image file uploads with validation maximum 10MB supporting JPEG PNG GIF and WEBP formats.
- **FR-31** The system shall store uploaded image files and return public URLs.

Administration & Analytics

- **FR-32** The system shall provide admin dashboard with user order and plant management.
- **FR-33** The system shall provide analytics data including total users plants orders revenue and top sellers.

3.2.4 Non Functional Requirements

- **NFR-01 Performance** – Page load times should not exceed 3 seconds under normal conditions.
- **NFR-02 Performance** – API response times should not exceed 500ms for standard CRUD operations.

- **NFR-03 Security** – All API endpoints must be protected with authentication middleware except public read only endpoints.
- **NFR-04 Security** – User passwords must be stored using bcrypt hashing.
- **NFR-05 Security** – File uploads must be validated for file type and size to prevent malicious uploads.
- **NFR-06 Security** – Role based access control must restrict sensitive operations to authorised users only.
- **NFR-07 Usability** – The system must provide responsive design for desktop and mobile devices.
- **NFR-08 Usability** – Error messages should be user friendly and provide clear guidance.
- **NFR-09 Reliability** – The system must handle database connection failures gracefully using meaningful error messages.
- **NFR-10 Reliability** – Cart state must be remain preserved across browser refreshes using localStorage.
- **NFR-11 Maintainability** – The codebase must follow MVC architecture in Laravel and component based architecture in React.
- **NFR-12 Maintainability** – Database schema changes must be managed through Laravel migrations.
- **NFR-13 Scalability** – The system should support up to 100 concurrent users without major performance degradation.
- **NFR-14 Scalability** – The architecture should support horizontal scaling by adding replicas or CDN image delivery.
- **NFR-15 Compatibility** – Frontend must remain compatible with modern Chrome Firefox Edge and Safari browsers.
- **NFR-16 Availability** – The system should maintain 99.5% uptime during business hours.

3.3 Use Case Diagram of the System

The Plantify Use Case diagram was built around four main actors. These includes the customer, specialist, delivery personnel and admin who interacts with the system in different ways. Each has different role in system, sometimes roles was slightly overlap in real use. The Customer register, logs in, browse and filters plants, manage shopping cart and wishlist, and checks out using delivery address, also views order history and submit reviews and questions, and it will making the shopping experience more smooth and user friendly overall. The Specialist creates edits and deletes plant listings with images and care tips manages categories and answers customer questions. Delivery Personnel can view assigned orders and update delivery statuses. The Admin oversees all users orders plants categories and analytics dashboard.

Shared among all authenticated actors are login change password profile update and logout use cases giving the system a clear role based functional structure.

3.3.1 Use Case Diagram

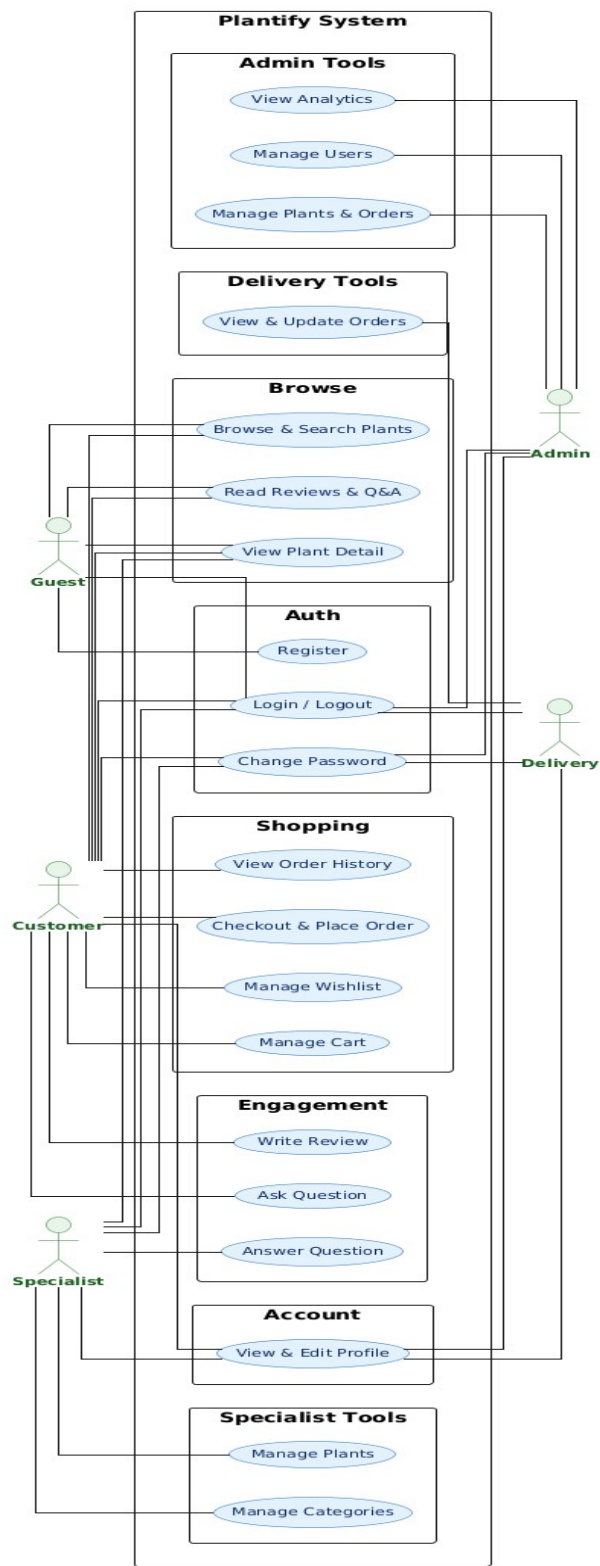


Figure 3.1: Use Case Diagram of the Plantify System

Chapter 4.

Analysis

4.1 Software Analysis Pattern

- **MVC Architecture** – Using Laravel in the backend is divided into three layers:
 1. **Model:** The eloquent models (User, Plant, Order etc) define data structure and business logic.
 2. **Controller:** Handles HTTP requests, validate input, interact with models and returns JSON responses.
 3. **View:** It is replaced by JSON data with Next.js visual rendering.
- **Repository Pattern** – Data access is encapsulated within Eloquent model classes:
 1. Clean query methods (Plant::all(), User::where(...)->first()).
 2. Relationship definitions (belongsTo, hasMany) on model level.
 3. Mass-assignment protection through \$fillable properties.
 4. The automatic timestamp management which handled by framework itself.
- **RESTful API Pattern** – All backend endpoints follows REST conventions:
 1. Stateless communication each request carries all necessary informations.
 2. Resource-oriented URLs like /api/plants, /api/orders/{id} etc.
 3. The standard HTTP verbs used for CRUD operations properly.
 4. Consistent JSON responses across all endpoints are maintained.
- **Client-Server Architecture** – These system is separated into two main folders. which are frontend and backend:
 1. Frontend: run in browser and communicate with Laravel server through HTTP/HTTPS.
 2. Backend: These processes requests to interact with MySQL and return JSON response while UI handled by frontend.
- **Component-Based Architecture** – Next.js frontend is built from reusable self-contained components:
 1. Header.tsx: provide navigation, user menu and cart icon.
 2. PlantCard.tsx: render individual plant cards with image name price and rating.
 3. Page Components: This combines the elements into full pages like home cart admin dashboard etc.
- **Data Access Pattern** – Database uses Active Record pattern of Eloquent:
 1. Each model represent a database table and instance represent rows.
 2. The relationships defined using belongsTo() and hasMany().
 3. Eager loading (with()) avoid N+1 query problem.

4. The fluent query builder allow expressive filtering and sorting operations.
- **Frontend Routing Pattern** – Navigation is file based routing system:
 1. Routes mirror directory structure under frontend/src/app/ folder.
 2. Examples /plants → app/plants/page.tsx , /plant/[id] → app/plant/[id]/page.tsx.
 3. These automatic code splitting and simplified routing management to make the system cleaner.
 - **File Storage Pattern** – The plant images are handled through Laravel filesystem:
 1. The uploaded files stored in storage/app/public/uploads folder.
 2. Public symbolic link make files accessible on web.
 3. UploadController implement MIME type and size validation (max 10MB).
 4. The Image URLs saved in thumbnail and images field inside plants table.

4.2 Activity Diagram

4.2.1 User Authentication (Register & Login)

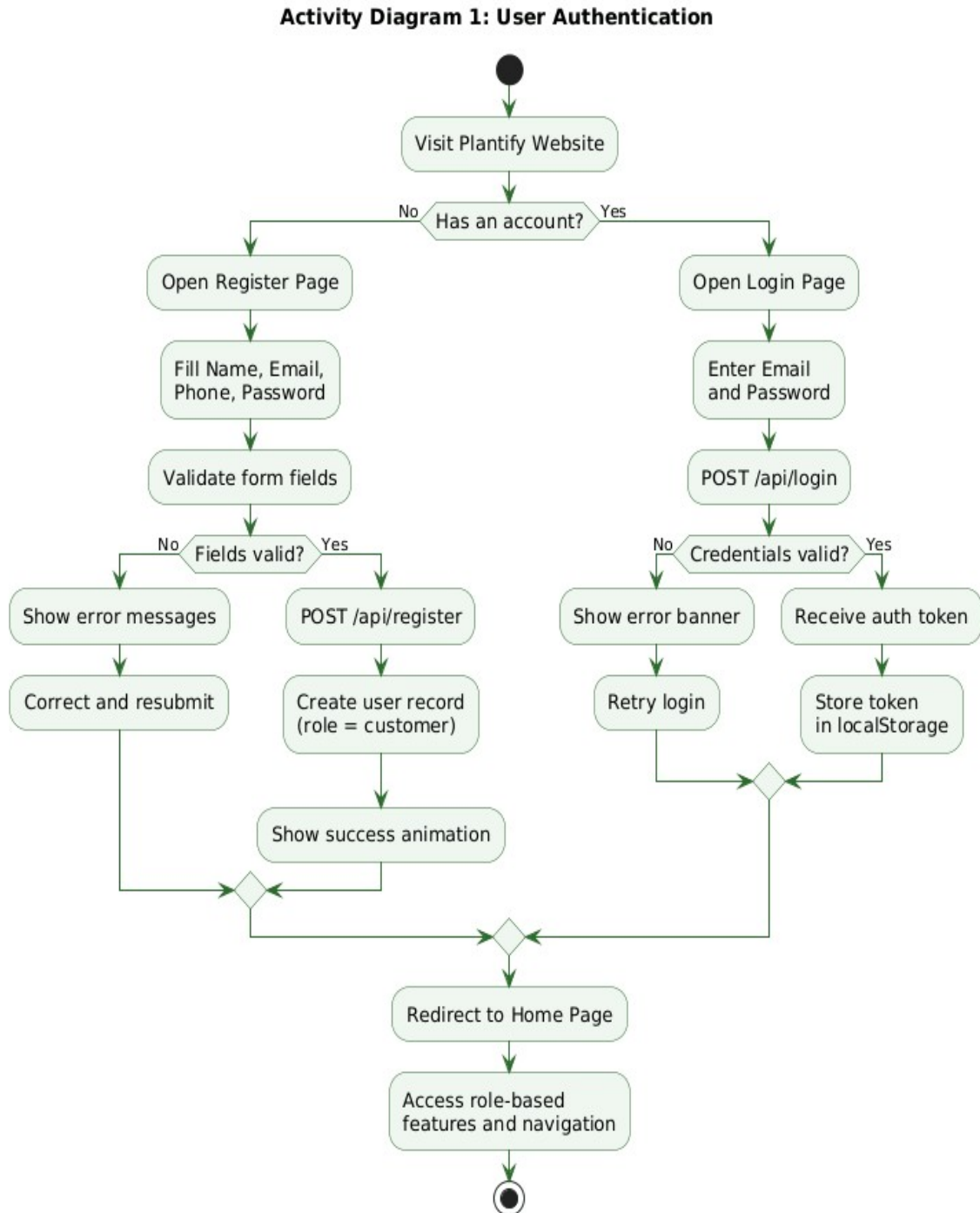


Figure 4.1: Activity Diagram – User Authentication (Register & Login)

4.2.2 Plant Browsing & Search

Activity Diagram 2: Plant Browsing & Search

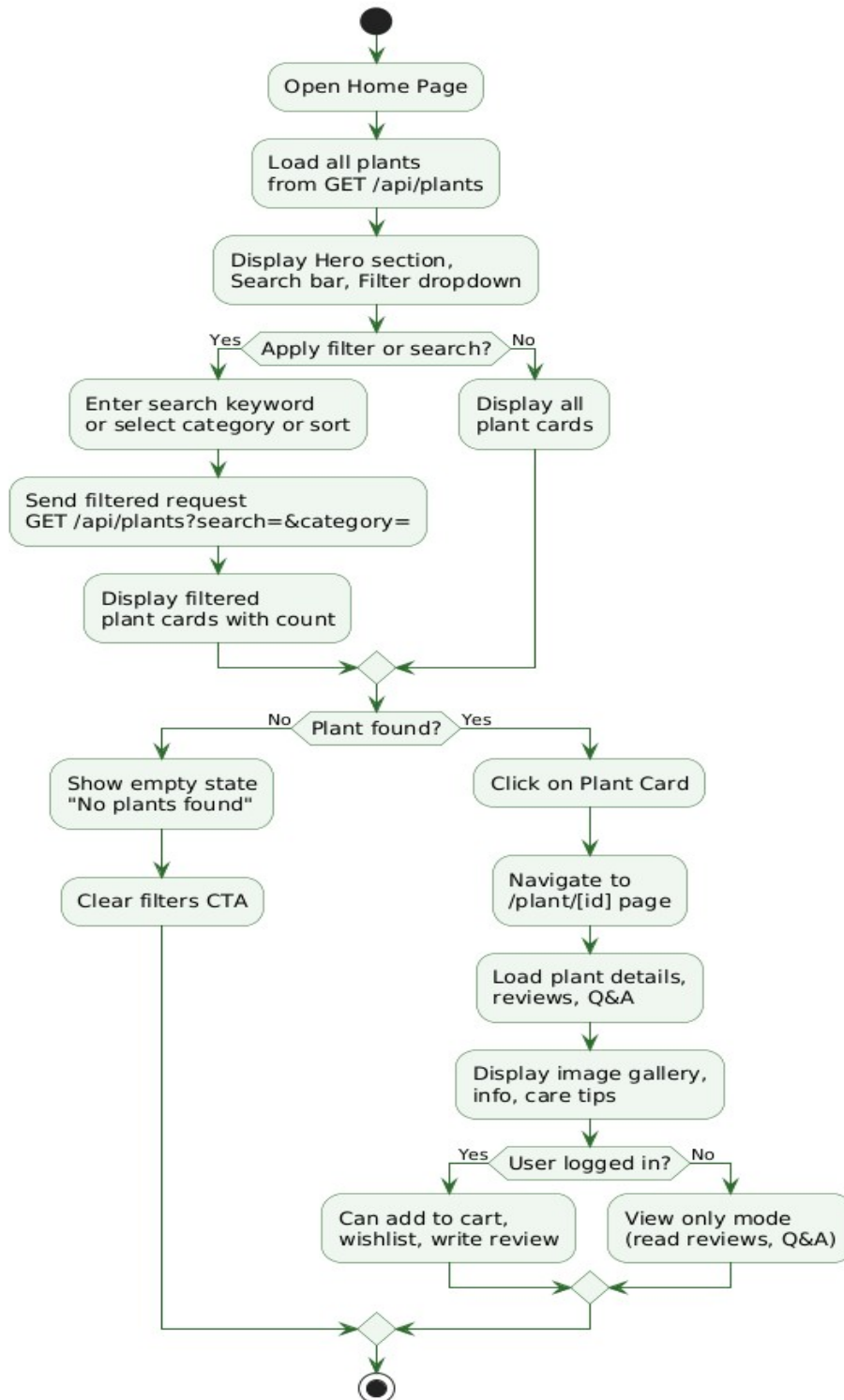


Figure 4.2: Activity Diagram – Plant Browsing & Search

4.2.3 Wishlist Management

Activity Diagram 3: Wishlist Management

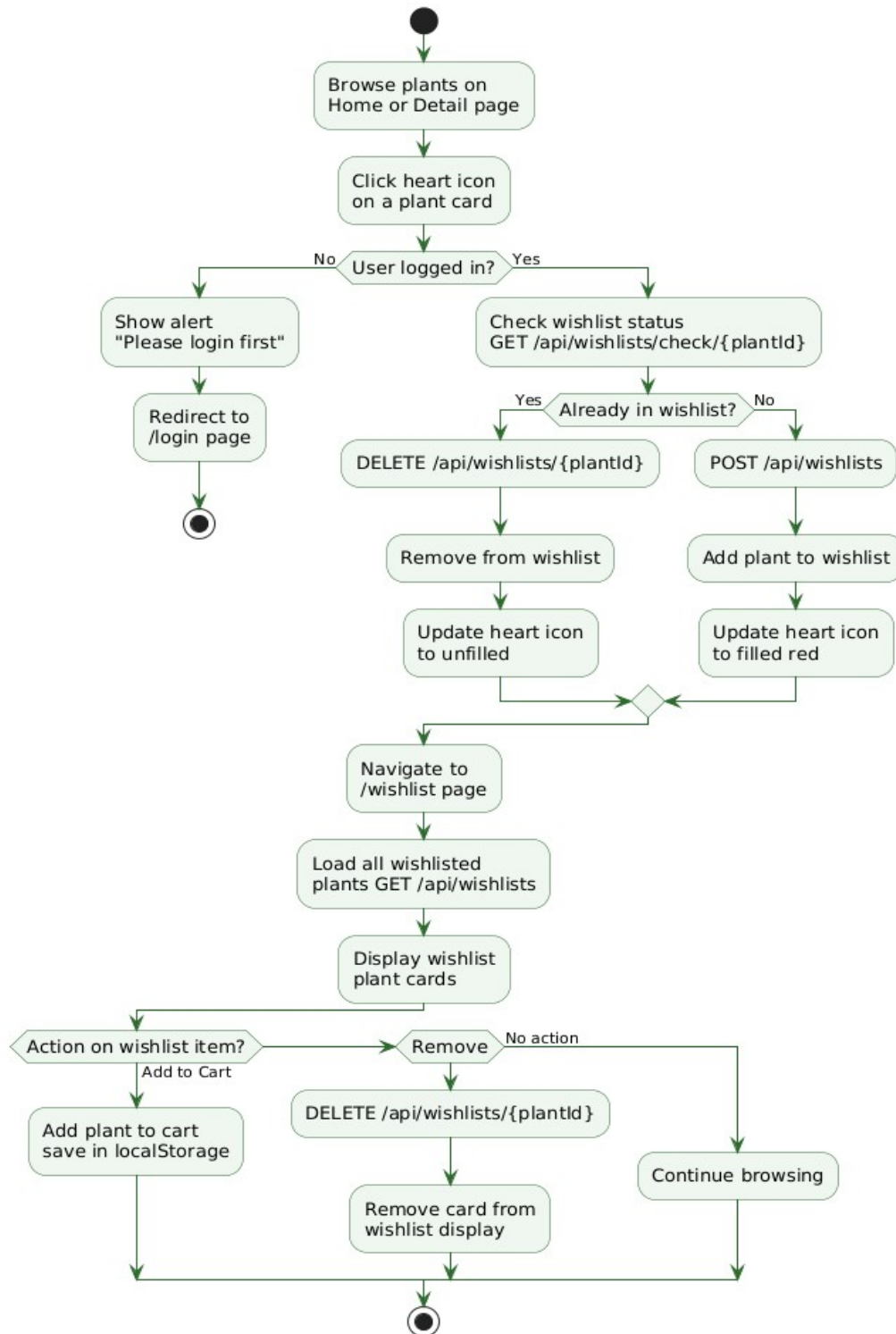


Figure 4.3: Activity Diagram – Wishlist Management

4.2.4 Cart & Checkout

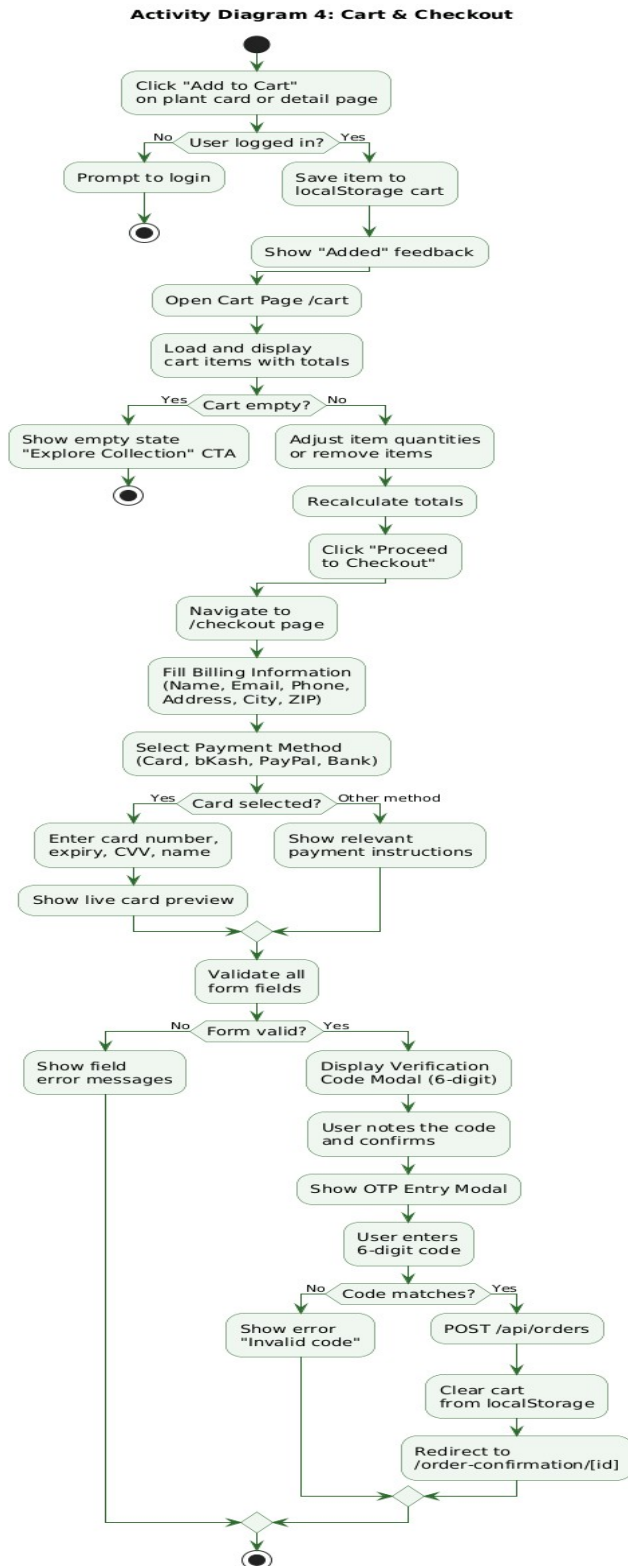


Figure 4.4: Activity Diagram – Cart & Checkout

4.2.5 Order Placement & Confirmation

Activity Diagram 5: Order Placement & Confirmation

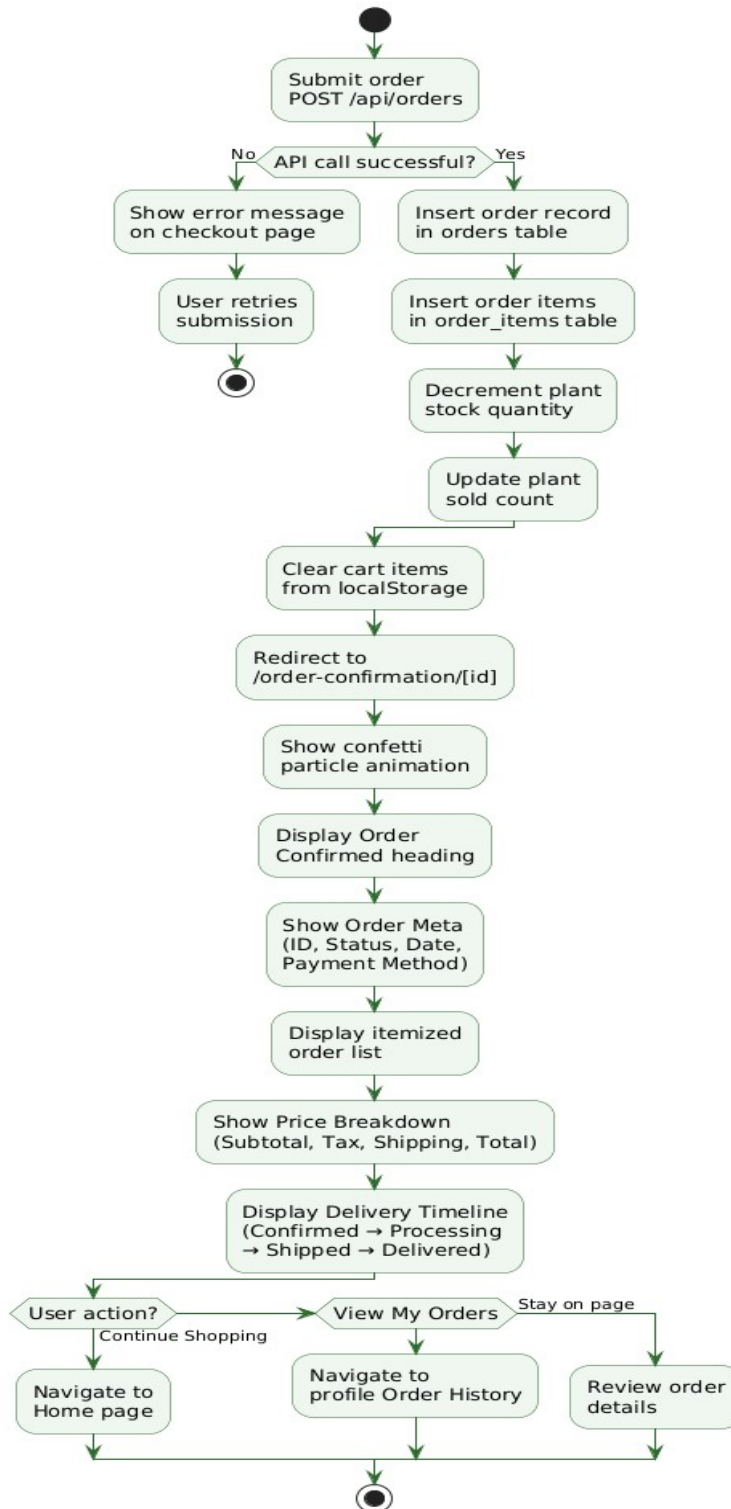


Figure 4.5: Activity Diagram – Order Placement & Confirmation

4.2.6 Order Status Management

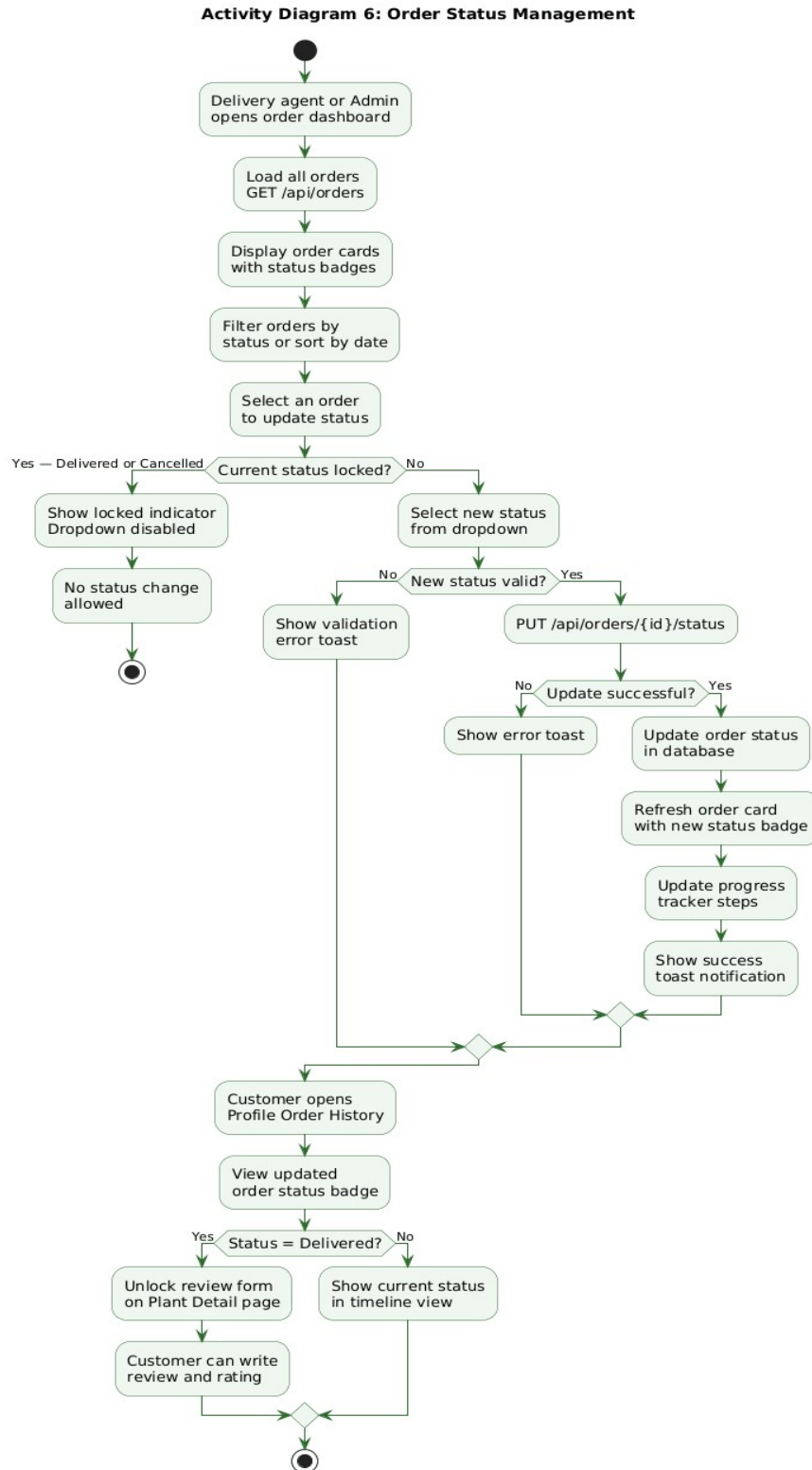


Figure 4.6: Activity Diagram – Order Status Management

4.2.7 Specialist — Plant & Category Management

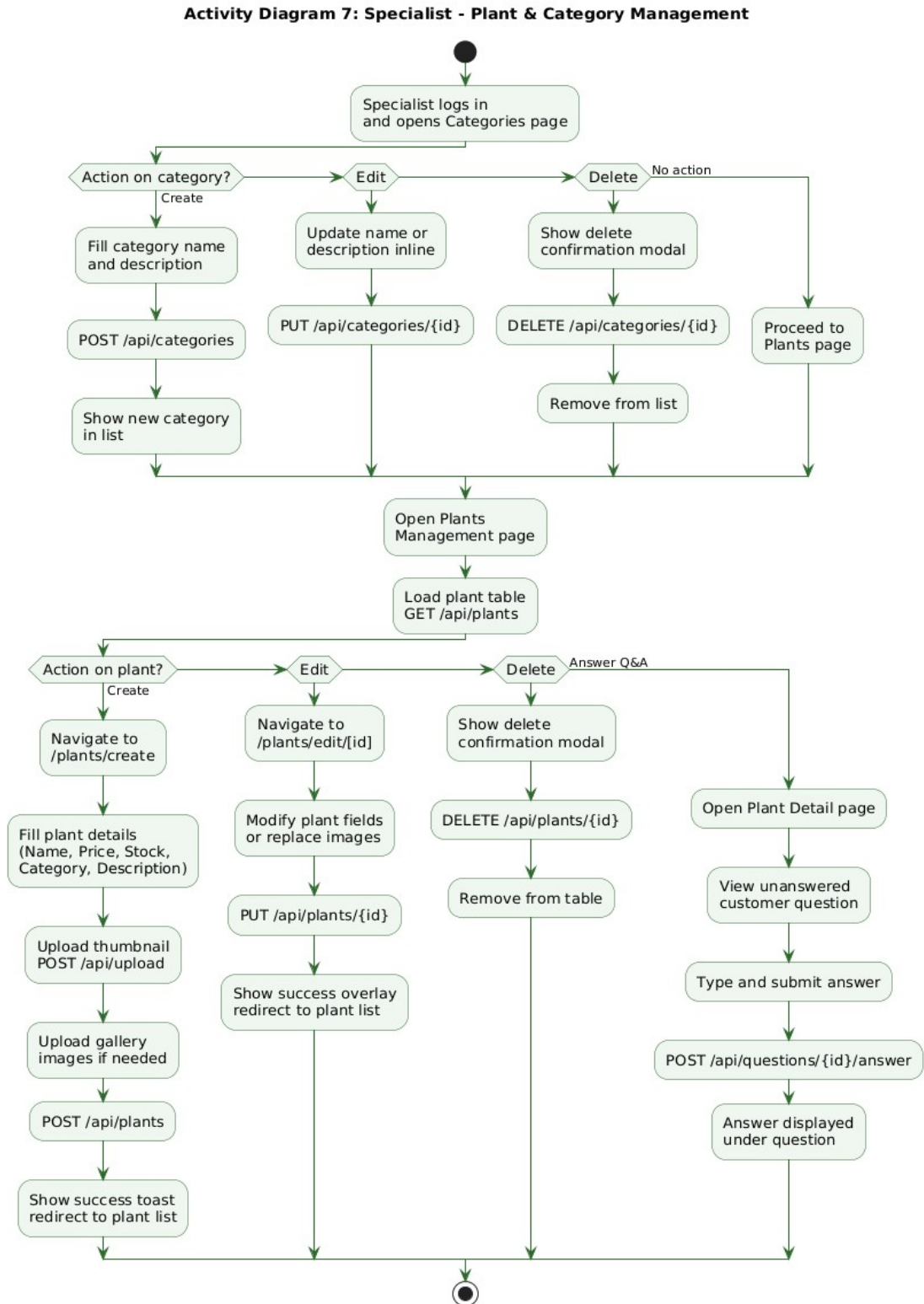


Figure 4.7: Activity Diagram – Specialist Plant & Category Management

4.2.8 Admin Management & Analytics

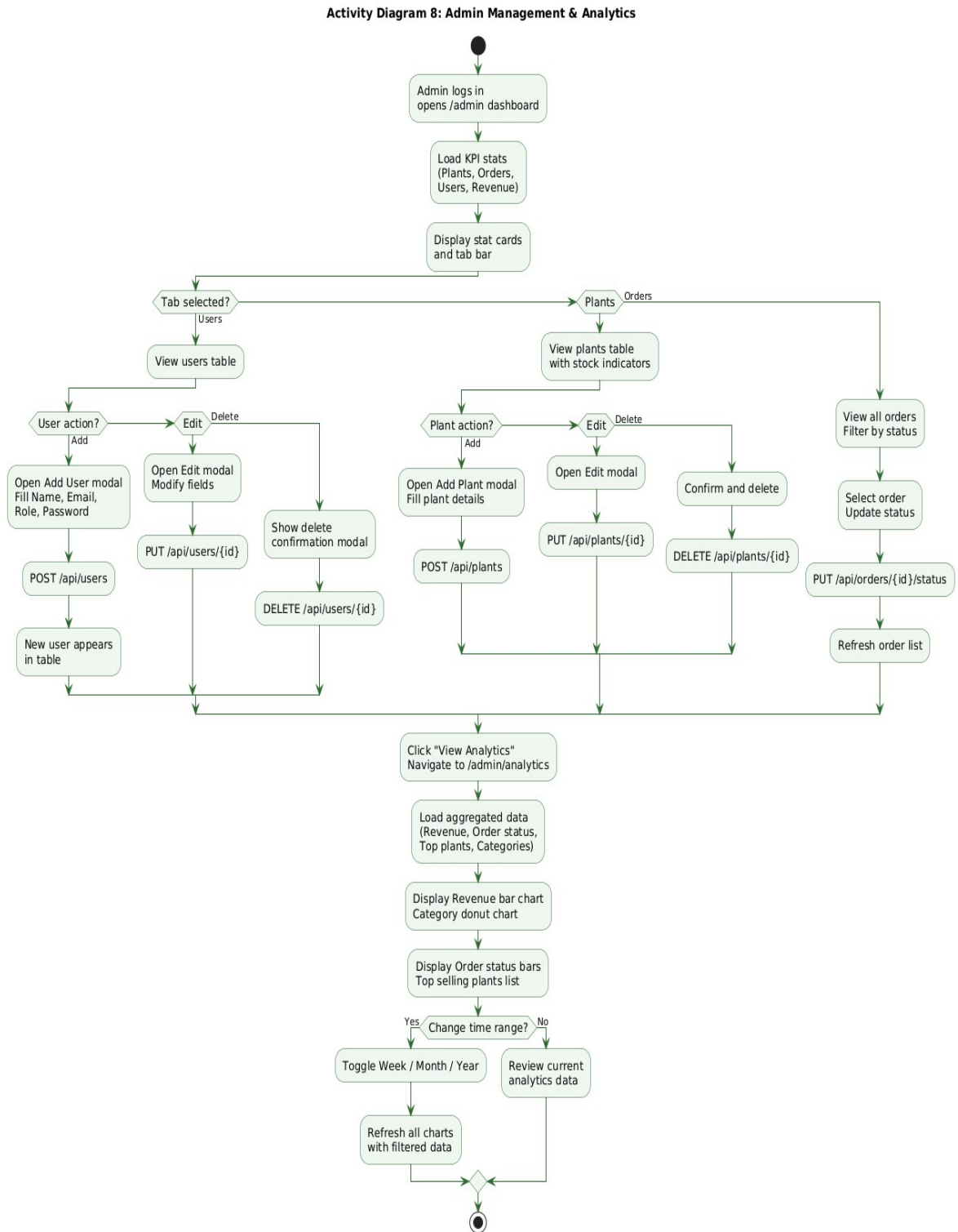


Figure 4.8: Activity Diagram – Admin Management & Analytics

4.3 Swim-lane Diagram

4.3.1 User Authentication

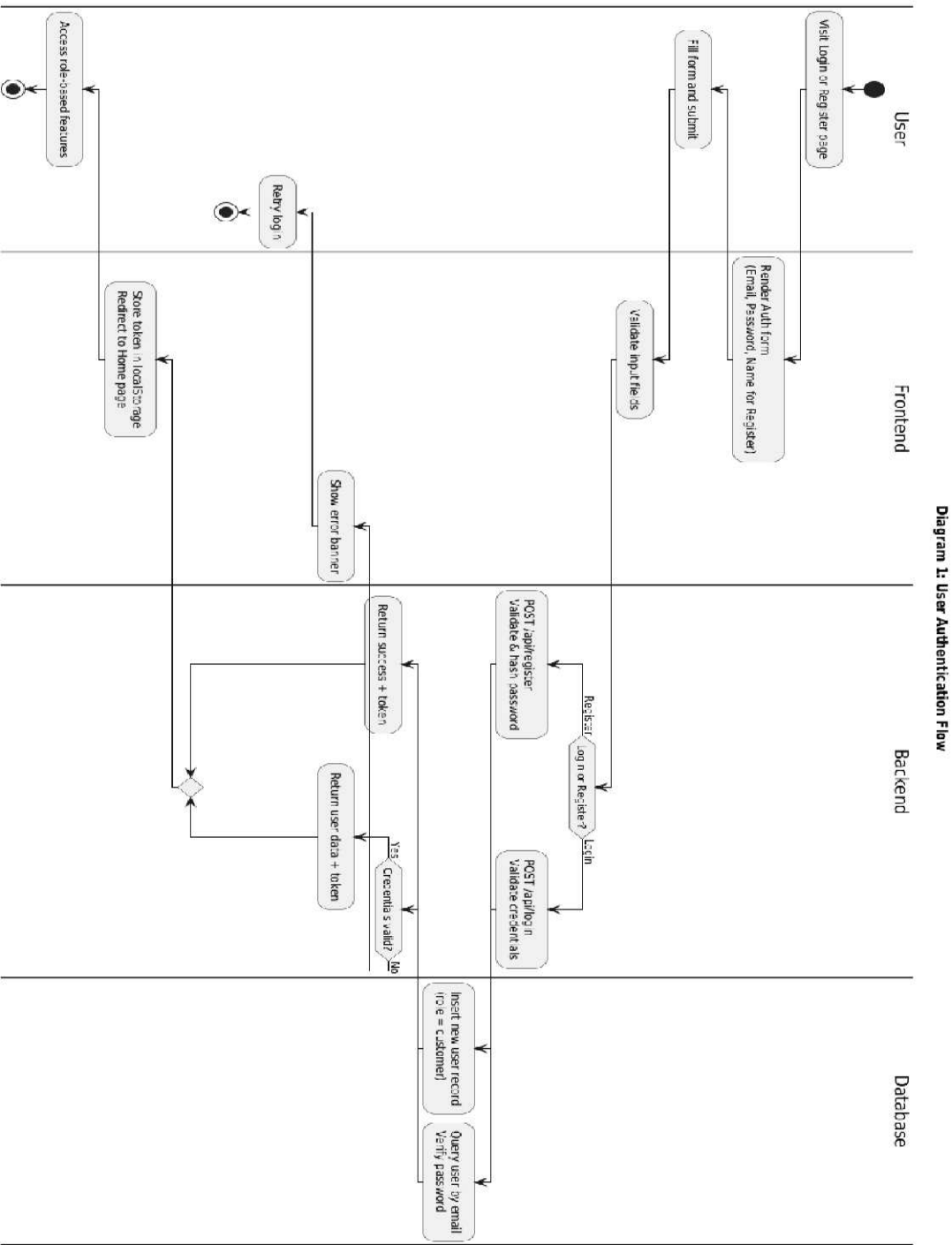


Figure 4.9: Swim-lane Diagram – User Authentication

4.3.2 Plant Browsing & Search

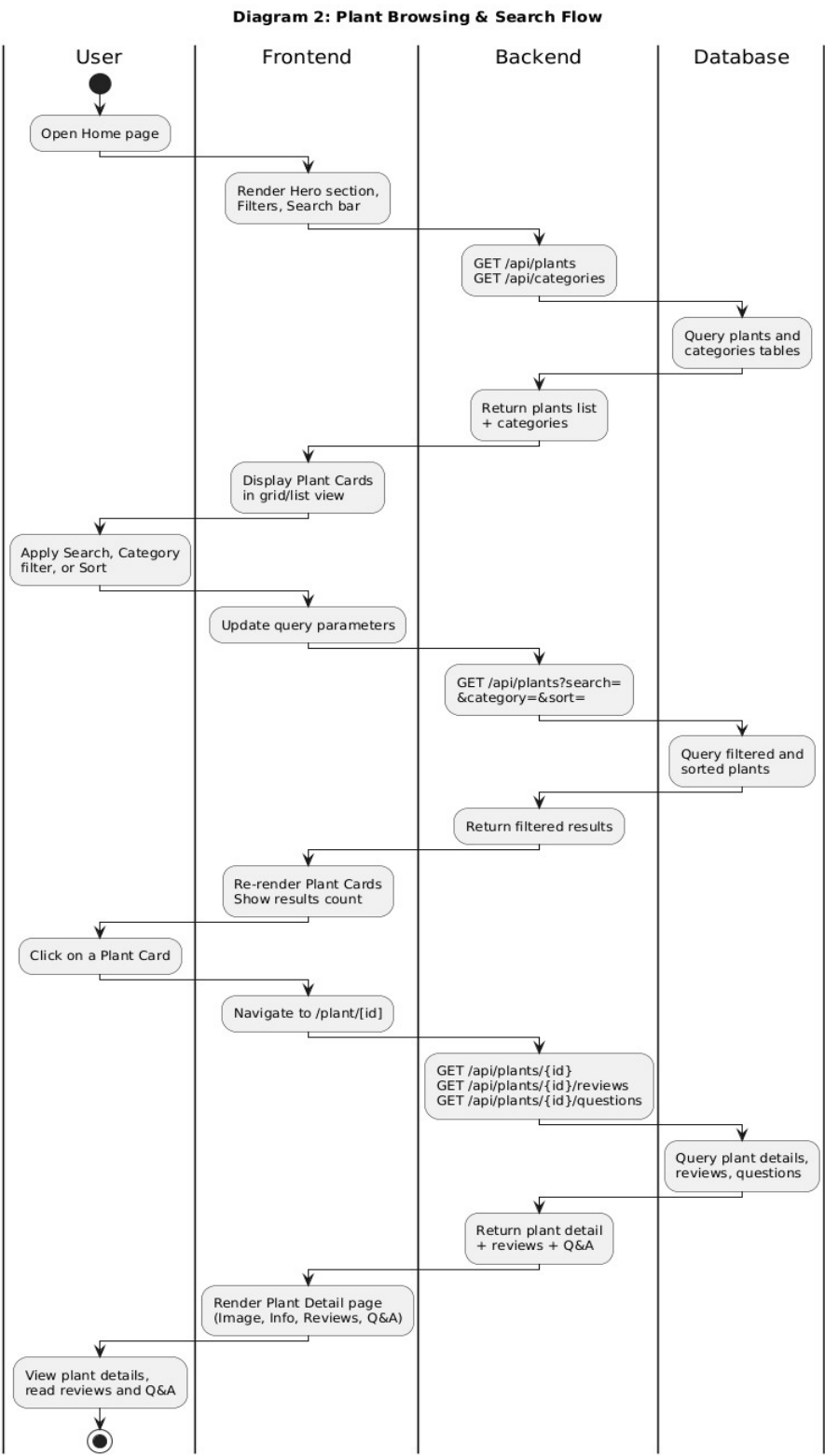


Figure 4.11: Swim-lane Diagram – Cart & Checkout

4.3.3 Cart & Checkout

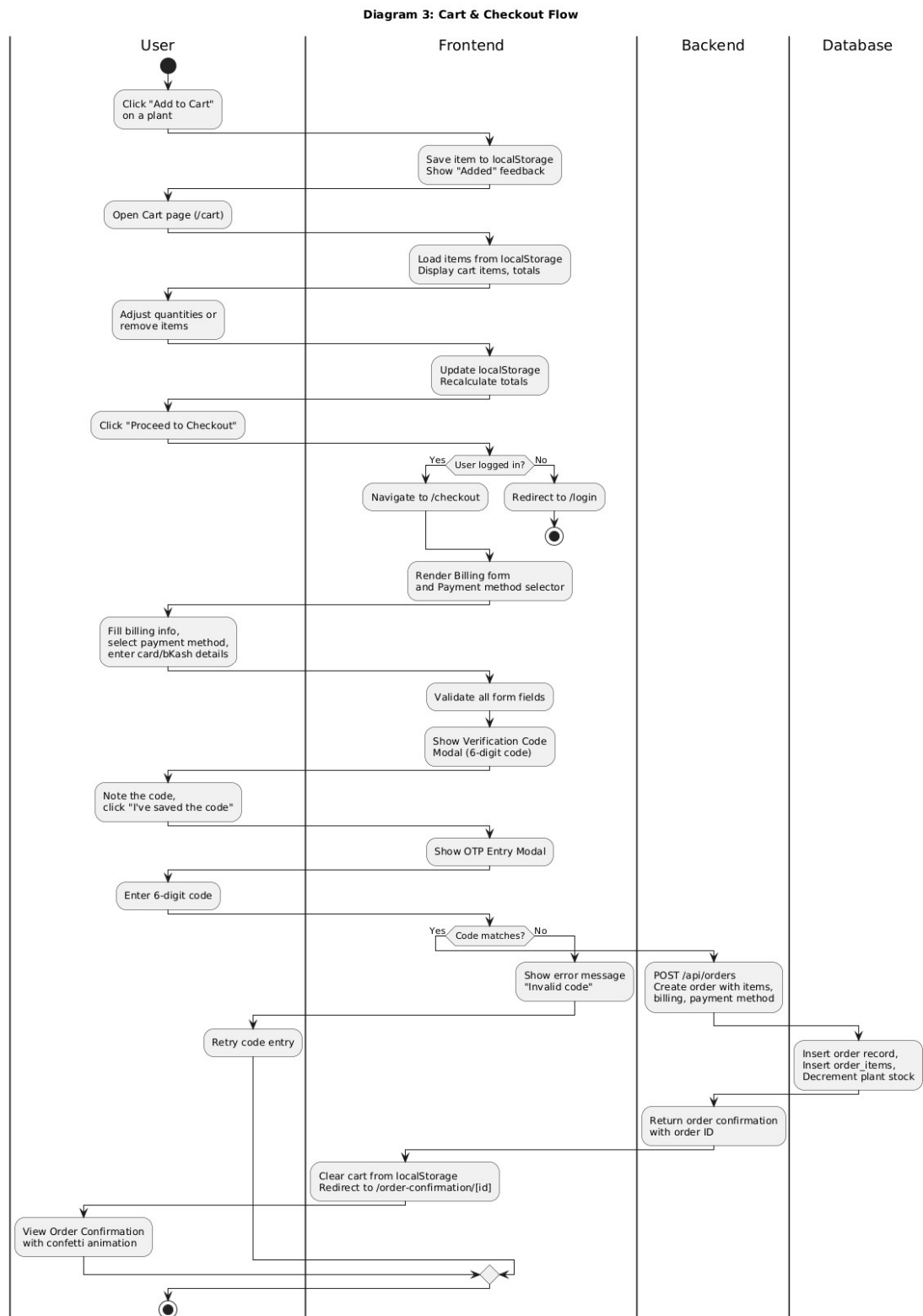


Figure 4.11: Swim-lane Diagram – Cart & Checkout

4.3.4 Order Management

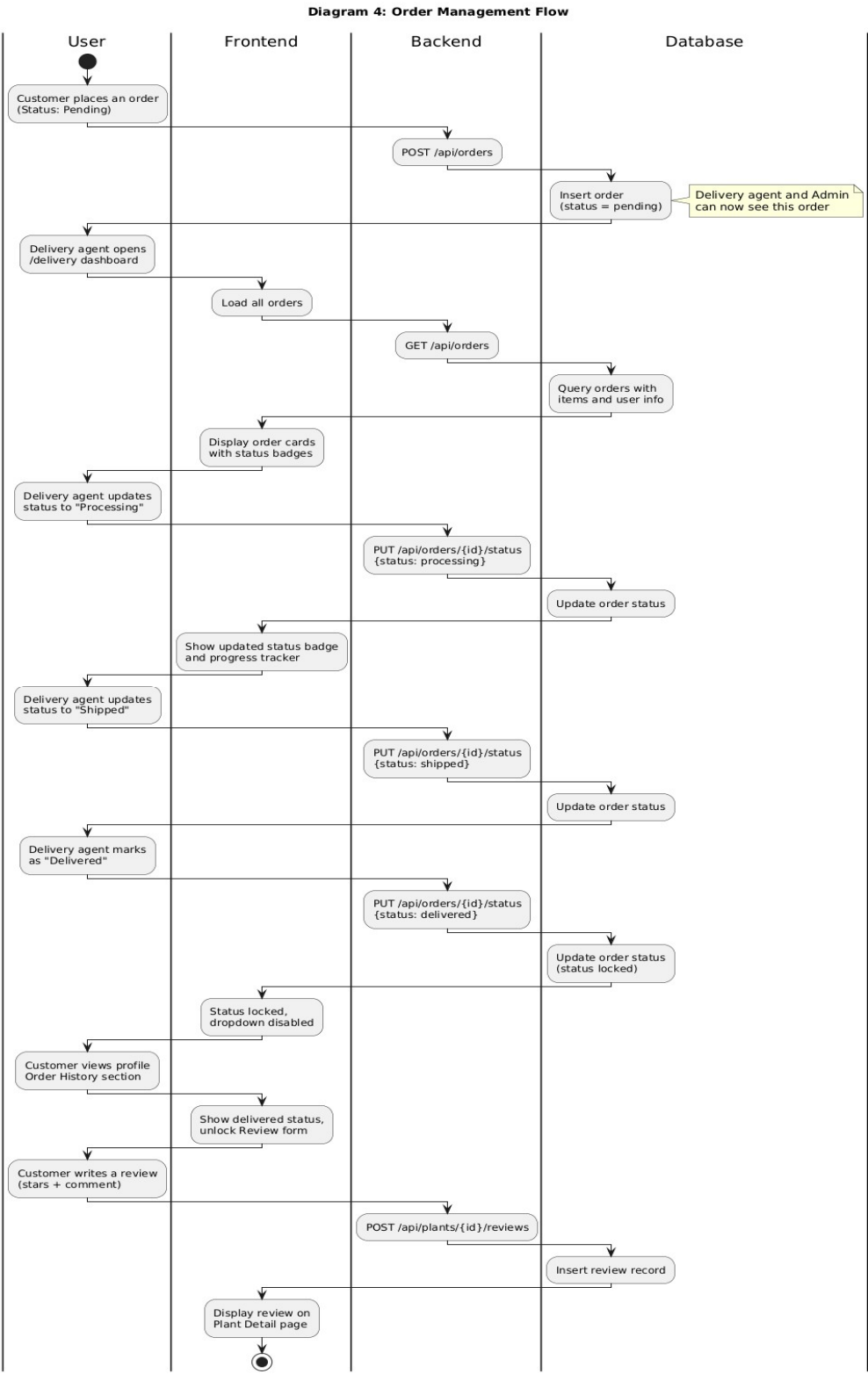


Figure 4.12: Swim-lane Diagram – Order Management

4.3.5 Specialist Plant & Q&A Management

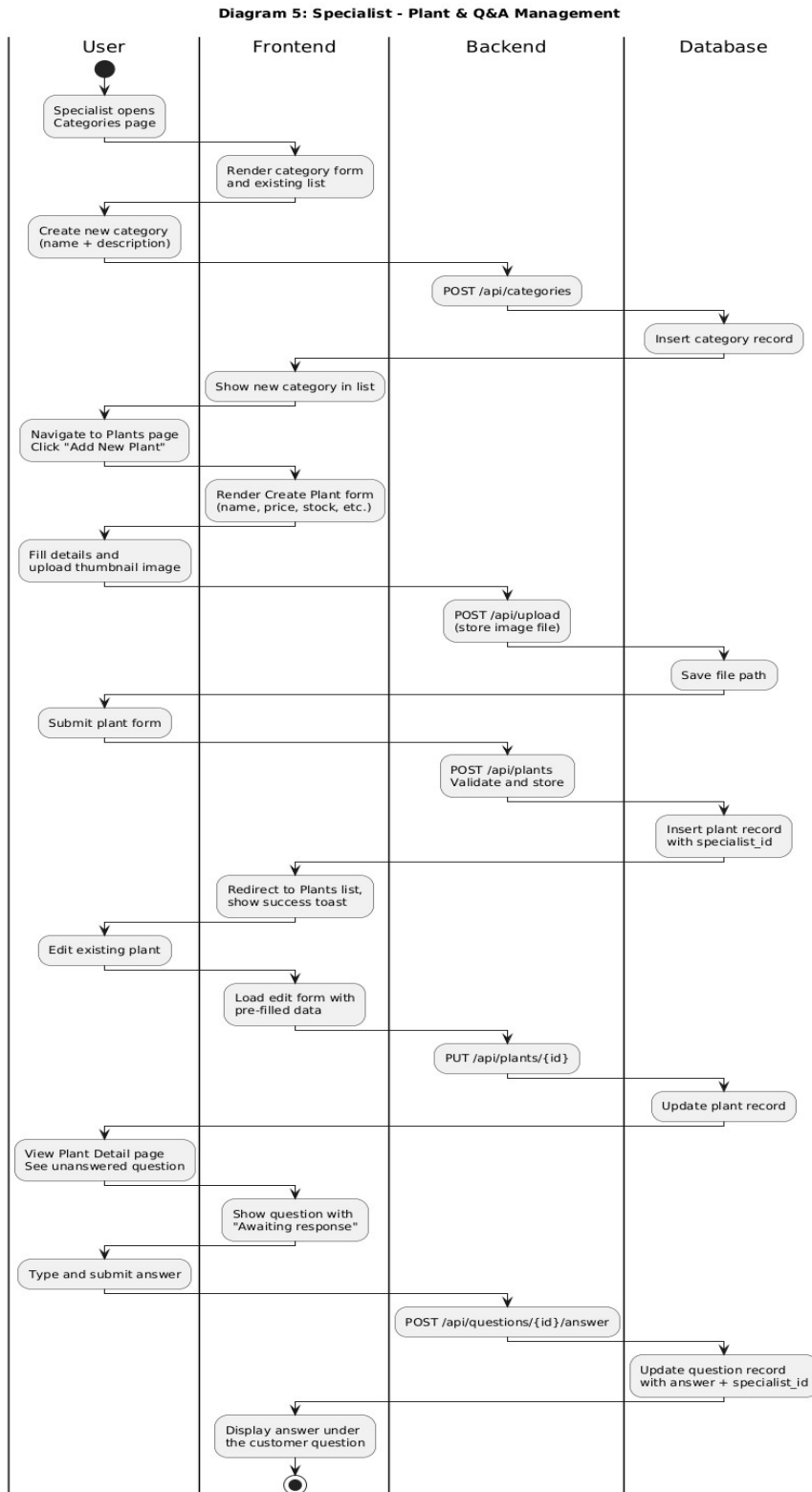


Figure 4.13: Swim-lane Diagram – Specialist Plant & Q&A Management

4.3.6 Admin Management & Analytics



Figure 4.14: Swim-lane Diagram – Admin Management & Analytics

4.4 Class Diagram

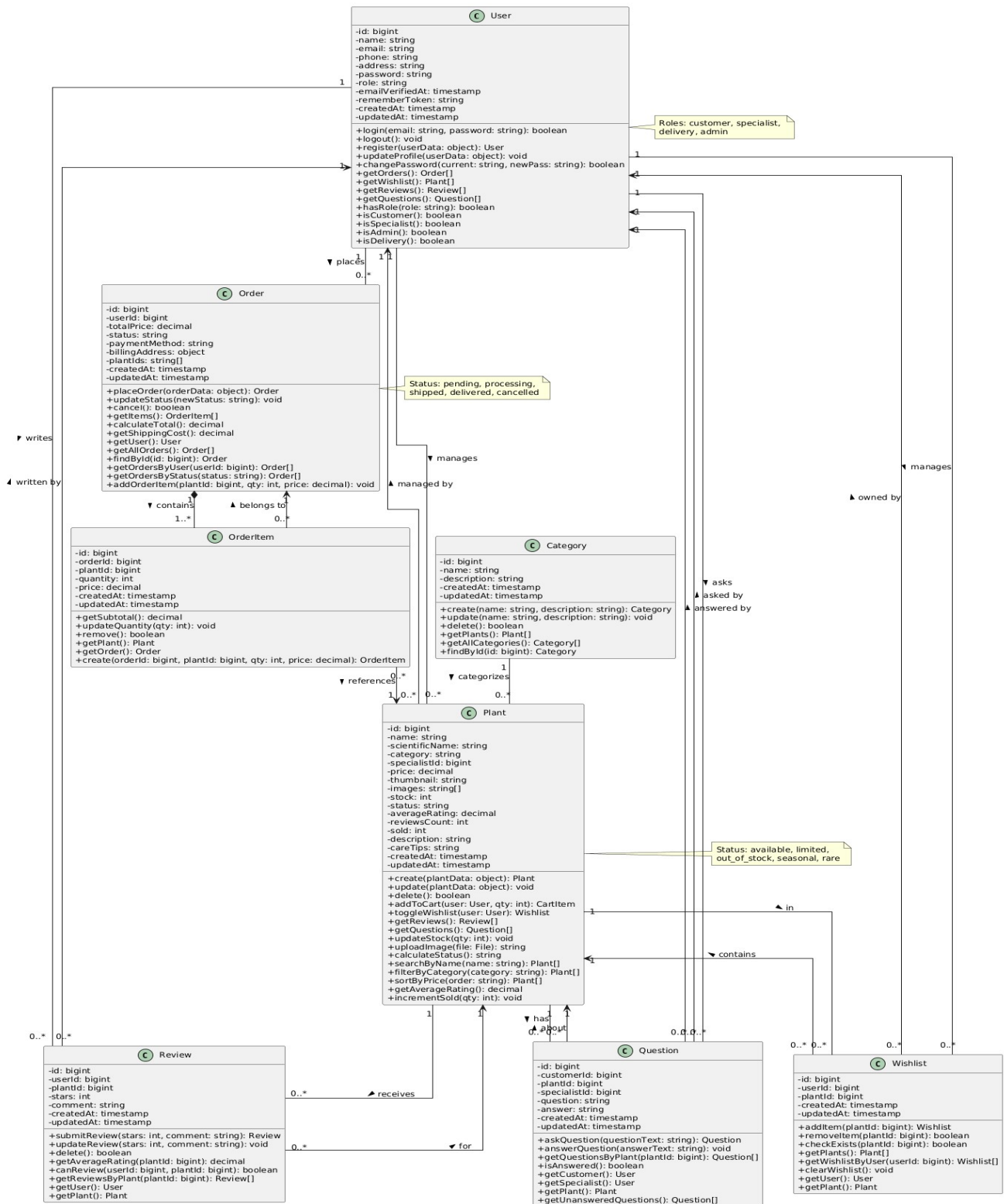


Figure 4.15: Class Diagram of the Plantify System

Chapter 5.
Project Management

5.1 Risk Identification Table

Table 5.1: Risk Identification:

Risk ID	Risk Category	Risk Description	Potential Impact
R-01	Technology	Database connection failure or misconfiguration (MySQL .env errors)	Application cannot serve any data; complete system downtime
R-02	Technology	Missing token-based authentication middleware; all API routes unprotected	Unauthorised access to any endpoint; data leakage and manipulation
R-03	Technology	No role-based access control (RBAC) enforcement on API endpoints	Privilege escalation; customers can perform admin/specialist operations
R-04	Technology	Frontend build failure due to TypeScript strict mode and compilation errors	Unable to deploy frontend; development blocked
R-05	Technology	Cart state loss on browser refresh or navigation away	Customers lose selected items; increased cart abandonment
R-06	Estimation	Inadequate test coverage – no formal unit or integration tests exist	Undetected regressions reach production; reliability unverified
R-07	Technology	CORS misconfiguration blocking frontend-backend communication	Complete API call failure; all frontend features broken

Risk ID	Risk Category	Risk Description	Potential Impact
R-08	Technology	File upload failure (storage symlink missing, MIME type, or size issues)	Plant images cannot be uploaded or displayed

5.2 Risk Analysis Table

Table 5.2: Risk Analysis:

Risk ID	Probability	Effects	Risk Severity	Rationale
R-01	Moderate	Serious	High	Local .env misconfiguration is common but results in a complete outage.
R-02	High	Catastrophic	Critical	API routes currently have no auth middleware; widespread exposure of sensitive data.
R-03	High	Catastrophic	Critical	No middleware-based role checks exist; privilege escalation is trivial.
R-04	Moderate	Serious	High	TypeScript strict mode frequently surfaces type mismatches; halts deployment.
R-05	Moderate	Serious	High	Only localStorage-based cart; no server sync; refresh discards state, risking lost sales.

Risk ID	Probability	Effects	Risk Severity	Rationale
R-06	High	Serious	High	No automated test suite; manual verification only; high risk of regressions.
R-07	Low	Serious	Medium	CORS config exists but port mismatches occur; if triggered, total API failure.
R-08	Moderate	Tolerable	Medium	Upload validation present but symlink may be missing; partial image failure.

5.3 Risk Planning Table

Table 5.3: Risk Planning:

Risk ID	Severity	Response Strategy	Planned Action
R-01	High	Mitigate	Create a .env.example with clear setup instructions; add a database health-check endpoint.
R-02	Critical	Mitigate	Install Laravel Sanctum; generate API tokens on login; apply auth:sanctum middleware to all protected routes.
R-03	Critical	Avoid / Mitigate	Implement role-based middleware (IsAdmin, IsSpecialist, IsDelivery); enforce on every restricted route.

Risk ID	Severity	Response Strategy	Planned Action
R-04	High	Mitigate	Enforce pre-commit TypeScript linting; run npm run build in CI; fix all type errors early.
R-05	High	Mitigate	Implement localStorage cart persistence with recovery on page load; optionally sync cart with backend on login.
R-06	High	Mitigate	Write PHPUnit tests for Auth, Plant, and Order controllers; add basic frontend component tests.
R-07	Medium	Mitigate	Standardise API base URL as a single config variable; verify CORS permits both localhost:8000 and 8001.
R-08	Medium	Mitigate	Verify storage:link existence on every deployment; add client-side file type/size validation; document upload limits.

5.4 The RMMM Plan

R-01: Database Connection Failure

Table 5.4: RMMM Plan – R-01 Database Connection Failure:

Risk Information Sheet	Information
Risk ID	R-01
Date	(project start)
Probability	Moderate
Impact	Serious
Description	MySQL connection failure due to incorrect .env configuration or missing database service. Application is unable to serve data.
Refinement / Context	Subcondition 1 – .env file missing or wrong credentials. Subcondition 2 – MySQL service not running.
Mitigation / Monitoring	1. Provide .env.example with placeholder values. 2. Add a /api/health endpoint that checks DB connectivity. 3. Document setup steps in README. 4. Weekly manual health-check during sprints.
Management / Contingency Plan / Trigger	If the health endpoint returns 500 or a connection error occurs, restore the latest SQL dump and reconfigure .env from the template. Trigger: health check fails at any time.
Current Status	.env.example created; health endpoint to be implemented in Sprint 1.

Risk Information Sheet	Information
Originator	Student Developer
Assigned	Student Developer

R-02: Missing Authentication Middleware

Table 5.5: RMMM Plan – R-02 Missing Authentication Middleware:

Risk Information Sheet	Information
Risk ID	R-02
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API routes lack token-based authentication. Any request, including unauthenticated ones, can access protected endpoints.
Refinement / Context	Subcondition 1 – routes/api.php currently defines no auth middleware. Subcondition 2 – Sanctum package not yet installed.
Mitigation / Monitoring	1. Install Laravel Sanctum (composer require laravel/sanctum). 2. Add auth:sanctum middleware to all non-public routes. 3. Weekly route audit via php artisan route:list to confirm coverage.

Risk Information Sheet	Information
Management / Contingency Plan / Trigger	If any unprotected endpoint is discovered, immediately apply auth middleware and invalidate exposed tokens. Trigger: unprotected route found during audit.
Current Status	Sanctum installation and middleware application planned for Sprint 1.
Originator	Student Developer
Assigned	Student Developer

R-03: No Role-Based Access Control

Table 5.6: RMMM Plan – R-03 No Role-Based Access Control:

Risk Information Sheet	Information
Risk ID	R-03
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API endpoints do not enforce role-based restrictions. A customer could call specialist or admin routes, leading to data corruption and security breaches.

Risk Information Sheet	Information
Refinement / Context	Subcondition 1 – User model has a 'role' column, but no middleware checks it. Subcondition 2 – Sensitive routes (user management, plant creation) are unprotected.
Mitigation / Monitoring	1. Create custom middleware: IsAdmin, IsSpecialist, IsDelivery. 2. Apply middleware to corresponding route groups. 3. Test with role-specific tokens each sprint.
Management / Contingency Plan / Trigger	If a non-admin successfully accesses an admin endpoint, patch middleware assignment within 24 hours and audit recent logs. Trigger: privilege escalation detected.
Current Status	Middleware classes to be created and applied in Sprint 1 alongside Sanctum.
Originator	Student Developer
Assigned	Student Developer

R-04: Frontend Build Failure

Table 5.7: RMMM Plan – R-04 Frontend Build Failure:

Risk Information Sheet	Information
Risk ID	R-04
Date	(Sprint 2 start)
Probability	Moderate

Risk Information Sheet	Information
Impact	Serious
Description	Next.js frontend fails to build because of TypeScript strict-mode type errors, blocking deployment.
Refinement / Context	Subcondition 1 – Type mismatches in API response handling. Subcondition 2 – Missing or incorrect prop types in React components.
Mitigation / Monitoring	1. Enforce pre-commit linting with ESLint and tsc --noEmit. 2. Add a CI step (GitHub Actions) that runs npm run build on every push. 3. Fix type errors as they appear.
Management / Contingency Plan / Trigger	If build fails on main branch, revert to the last known good commit and fix errors incrementally. Trigger: build failure in CI or local production build.
Current Status	Pre-commit hooks to be configured in Sprint 2; CI pipeline planned for Sprint 2.
Originator	Student Developer
Assigned	Student Developer

R-05: Cart State Loss

Table 5.8: RMMM Plan – R-05 Cart State Loss:

Risk Information Sheet	Information
Risk ID	R-05
Date	(Sprint 2 start)
Probability	Moderate
Impact	Serious
Description	Shopping cart data stored only in browser localStorage is lost on page refresh, browser crash, or navigation away, causing abandoned checkouts.
Refinement / Context	Subcondition 1 – No server-side cart persistence. Subcondition 2 – localStorage may be cleared by the user or browser policy.
Mitigation / Monitoring	1. Ensure cart state is read from localStorage on every page load. 2. Add a warning before leaving the checkout page. 3. (Medium term) Sync cart to backend on user login.
Management / Contingency Plan / Trigger	If the cart is found empty after a refresh, attempt to recover from the latest localStorage snapshot; if unavailable, display an empty cart message with a clear CTA. Trigger: cart items count drops to zero unexpectedly.
Current Status	localStorage persistence implemented; recovery logic to be refined in Sprint 2.
Originator	Student Developer

Risk Information Sheet	Information
Assigned	Student Developer

R-06: Inadequate Test Coverage

Table 5.9: RMMM Plan – R-06 Inadequate Test Coverage:

Risk Information Sheet	Information
Risk ID	R-06
Date	(Sprint 3 start)
Probability	High
Impact	Serious
Description	No automated unit or integration tests exist. All verification is manual, making regressions likely as the codebase evolves.
Refinement / Context	Subcondition 1 – Critical backend controllers (Auth, Plant, Order) lack PHPUnit tests. Subcondition 2 – Frontend components have no formal test suite.
Mitigation / Monitoring	1. Write PHPUnit tests for Auth, Plant, and Order controllers covering success and validation failures. 2. Add basic component smoke tests. 3. Run test suite weekly and require passing tests before merges.
Management / Contingency Plan / Trigger	If a previously passing test fails after a code change, fix the test or the code immediately and add a regression test. Trigger: any test failure in the CI run.

Risk Information Sheet	Information
Current Status	Test suite development scheduled for Sprint 3.
Originator	Student Developer
Assigned	Student Developer

R-07: CORS Misconfiguration

Table 5.10: RMMM Plan – R-07 CORS Misconfiguration:

Risk Information Sheet	Information
Risk ID	R-07
Date	(Sprint 1)
Probability	Low
Impact	Serious
Description	CORS settings in Laravel may not allow the frontend origin (port mismatch between :8000 and :8001), causing all API calls to fail.
Refinement / Context	Subcondition 1 – Laravel’s config/cors.php may only whitelist the production domain. Subcondition 2 – Frontend dev server (Next.js) often runs on port 3000 or 3001, while Laravel uses 8000.
Mitigation / Monitoring	1. Store API base URL in a single environment variable. 2. Add localhost:3000 and localhost:3001 to allowed origins. 3.

Risk Information Sheet	Information
	Test a cross-origin request from frontend to backend at the start of each sprint.
Management / Contingency Plan / Trigger	If CORS errors appear in the browser console, adjust cors.php to explicitly allow the frontend origin and re-verify with curl. Trigger: any CORS error during development.
Current Status	CORS configuration verified for current dev ports; will be re-checked after deployment.
Originator	Student Developer
Assigned	Student Developer

R-08: File Upload Failure

Table 5.11: RMMM Plan – R-08 File Upload Failure:

Risk Information Sheet	Information
Risk ID	R-08
Date	(Sprint 1)
Probability	Moderate
Impact	Tolerable
Description	Plant image uploads fail because the storage symbolic link is missing, or uploaded files exceed size/type limits, resulting in broken images.

Risk Information Sheet	Information
Refinement / Context	Subcondition 1 – php artisan storage:link not executed after deployment. Subcondition 2 – Client sends an image >10MB or an unsupported format without proper client-side validation.
Mitigation / Monitoring	1. Document storage:link command in setup instructions. 2. Add a startup script that checks symlink existence. 3. Implement client-side file size/type validation before upload. 4. Test the upload endpoint after every deployment.
Management / Contingency Plan / Trigger	If image upload returns 500 or image URL returns 404, re-run storage:link and ensure the uploads directory has correct permissions. Trigger: a plant's image fails to display.
Current Status	Backend validation in place; symlink check to be added to deployment steps.
Originator	Student Developer
Assigned	Student Developer

Chapter 6.
Project Planning

6.1 System Project Estimation

The Plantify project is a full-stack plant nursery management application built with Laravel 12 (PHP) and Next.js 16 (React/TypeScript). It supports four user roles – Customer, Specialist, Delivery, and Admin – across 8 database tables and 20+ API endpoints. As a single-developer, this 13-week academic project, the estimation has been carried out using the IFPUG Function Point Analysis (FPA) method, combined with a productivity rate calibrated for modern web frameworks and a solo developer. The following sections present the detailed estimation, the resulting 13-week schedule, and complete budget in Bangladeshi Taka (TK).

6.2 Function-Oriented Metrics

Function Point Analysis categorises software functionality into five standard function types:

Table 6.1: Function-Oriented Metrics (IFPUG Categories):

Type	Abbreviation	Description
External Input	EI	Processes data or control information entering the system from outside its boundary.
External Output	EO	Sends data or control information out of the system, after performing calculations or logic.
External Inquiry	EQ	Retrieves data from inside the system and presents it without altering it or performing computations.
Internal Logical File	ILF	A user-identifiable group of logically related data maintained within the application (database tables).
External Interface File	EIF	A user-identifiable group of logically related data referenced but not maintained by the application; read-only.

6.3 Identifying Complexity

6.3.1 Complexity Weights

The standard IFPUG complexity weights used in this estimation are:

Table 6.2: IFPUG Complexity Weights:

Function Type	Low	Average	High
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

6.3.2 Identifying Complexity of Transaction Functions

All transaction functions (EI, EO, EQ) are listed with their Data Element Types (DETs), File Types Referenced (FTRs), assigned complexity, and corresponding weight.

External Inputs (EI) – 18 functions

Table 6.3: External Inputs (EI) Complexity Analysis:

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-01	Register new user account	4	1	Low	3
EI-02	User login	2	1	Low	3
EI-03	Create plant listing	10	2	Average	4
EI-04	Update plant listing	10	2	Average	4
EI-05	Delete plant listing	1	1	Low	3

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-06	Place new order	5	3	High	6
EI-07	Update order status	2	1	Low	3
EI-08	Submit plant review	4	2	Low	3
EI-09	Ask question about plant	3	2	Low	3
EI-10	Answer a customer question	2	1	Low	3
EI-11	Add plant to wishlist	2	1	Low	3
EI-12	Remove plant from wishlist	1	1	Low	3
EI-13	Create new category	3	1	Low	3
EI-14	Update category	3	1	Low	3
EI-15	Upload plant image file	1	1	Low	3
EI-16	Create user (admin)	5	1	Low	3
EI-17	Update user (admin)	5	1	Low	3
EI-18	Delete user (admin)	1	1	Low	3

External Outputs (EO) – 2 functions

Table 6.4: External Outputs (EO) Complexity Analysis:

EO ID	Function Description	DETs	FTRs	Complexity	Weight
EO-01	Order confirmation receipt	8	2	Average	5
EO-02	Admin analytics dashboard	12	4	High	7

External Inquiries (EQ) – 12 functions

Table 6.5: External Inquiries (EQ) Complexity Analysis:

EQ ID	Function Description	DETs	FTRs	Complexity	Weight
EQ-01	Browse / filter all plants	6	1	Low	3
EQ-02	View single plant details	10	3	Average	4
EQ-03	List user orders (customer)	5	2	Low	3
EQ-04	List all categories	3	1	Low	3
EQ-05	View reviews for a plant	3	1	Low	3
EQ-06	View questions for a plant	3	1	Low	3
EQ-07	View wishlist items	4	2	Low	3
EQ-08	Check wishlist status	1	1	Low	3
EQ-09	Admin – list all users	5	1	Low	3
EQ-10	Admin – list all orders	5	2	Low	3
EQ-11	Admin – view single user	5	1	Low	3
EQ-12	Delivery – view assigned orders	4	2	Average	4

6.3.3 Identifying Complexity of Data Functions

Internal Logical Files (ILF) – 8 files

Table 6.6: Internal Logical Files (ILF) Complexity Analysis:

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-01	Users	1	8	Low	7
ILF-02	Plants	1	12	Low	7
ILF-03	Categories	1	4	Low	7

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-04	Orders	1	7	Low	7
ILF-05	Order Items	1	5	Low	7
ILF-06	Reviews	1	6	Low	7
ILF-07	Questions	1	7	Low	7
ILF-08	Wishlists	1	4	Low	7

External Interface Files (EIF) – 1 file

Table 6.7: External Interface Files (EIF) Complexity Analysis:

EIF ID	External Data Source	RETs	DETs	Complexity	Weight
EIF-01	File Storage (public/uploads)	1	2	Low	5

6.3.4 Unadjusted Function Point Contribution (Transactions)

Table 6.8: Unadjusted Function Point Contribution – Transactions:

Function Type	Count	Low	Average	High	Total UFP
External Inputs (EI)	18	$15 \times 3 = 45$	$2 \times 4 = 8$	$1 \times 6 = 6$	59
External Outputs (EO)	2	0	$1 \times 5 = 5$	$1 \times 7 = 7$	12
External Inquiries (EQ)	12	$10 \times 3 = 30$	$2 \times 4 = 8$	0	38
Total UFP (Transactions)	32				109

6.3.5 Unadjusted Function Point Contribution (Data)

Table 6.9: Unadjusted Function Point Contribution – Data:

Function Type	Count	Low	Average	High	Total UFP
Internal Logical Files (ILF)	8	$8 \times 7 = 56$	0	0	56
External Interface Files (EIF)	1	$1 \times 5 = 5$	0	0	5
Total UFP (Data)	9				61

Unadjusted Function Points (UFP) = 109 + 61 = 170

6.3.6 Performance and Environmental Impact

The Value Adjustment Factor (VAF) is derived from the 14 General System Characteristics (GSCs), each rated on a scale of 0 (no influence) to 5 (strong influence).

Table 6.10: General System Characteristics (GSC) Ratings:

GSC ID	General System Characteristic	Rating	Justification
1	Data Communications	2	Web-based application; academic prototype runs on localhost.
2	Distributed Data Processing	1	Single MySQL instance; no distributed processing.
3	Performance	2	Not a critical concern for a demo system with low concurrent load.
4	Heavily Used Configuration	1	Runs on a single development laptop.

GSC ID	General System Characteristic	Rating	Justification
5	Transaction Rate	1	Low transaction rate expected in a demo environment.
6	Online Data Entry	4	Over 80% of transactions are online data entry.
7	End-User Efficiency	3	UI is designed for ease of use; advanced efficiency features limited.
8	Online Update	3	Many ILFs are updated online (plants, orders, profile).
9	Complex Processing	1	Business logic is straightforward CRUD with simple calculations.
10	Reusability	2	Code is moderately reusable; developed primarily for this project.
11	Installation Ease	3	Requires standard XAMPP and npm setup; documented in README.
12	Operational Ease	3	Easy to start/stop; manual backup processes.
13	Multiple Sites	1	Single-site deployment only.
14	Facilitate Change	2	Loosely coupled code; no formal configuration management.

Total Degree of Influence (TDI) = 2+1+2+1+1+4+3+3+1+2+3+3+1+2 = **29**

Value Adjustment Factor (VAF) = 0.65 + (29 / 100) = **0.94**

6.3.7 Counting Adjusted Function Points

Adjusted Function Points (AFP) = $170 \times 0.94 = 159.8 \approx \mathbf{160 \text{ AFP}}$

6.3.8 Effort Estimation

For a solo developer using mature, high-productivity frameworks (Laravel 12, Next.js 16, Tailwind CSS) and leveraging pre-built patterns, a productivity rate of 53 function points per person-month is realistic. The rate reflects the rapid prototyping capability of the chosen stack and the developer's familiarity with the tools.

Effort = $160 \text{ AFP} / 53 \text{ FP/person-month} = \mathbf{3.02 \text{ person-months}}$

The rounding to the nearest whole number, the total effort is **3.0 person-months**. With a single developer, this effort translates directly into a **13-week** calendar duration (1 month $\approx$ 4.33 weeks; $3.0 \times 4.33 \approx 13$ weeks).

6.3.9 Lines of Code Approximation

The using of adjusted function point count of 160, the total codebase size is estimated at approximately 3,000 – 3,200 lines of code (covering PHP backend, TypeScript/React frontend, and configurations). This estimate aligns with the observed size of the existing Plantify repositories and the typical productivity of the frameworks used.

6.4 Project Schedule Chart

The 13-week project is organised into 4 sprints, executed sequentially by a single developer. The sprint breakdown matches the agile process model described in Chapter 1.

Table 6.11: Project Schedule (13-Week Sprint Plan):

Week	Sprint	Key Tasks	Deliverable / Milestone
1-3	Sprint 1	Project setup, database design, Laravel API foundation (Auth, Plants, Categories, Upload), Next.js initialisation, plant browsing UI	M1 – Backend API foundation complete
4-6	Sprint 2	Login/register pages, plant detail page, cart, checkout, orders, wishlist, profile management	M2 – Core e-commerce flow functional
7-9	Sprint 3	Reviews, Q&A, specialist dashboard, delivery role, category management, PHPUnit tests for critical controllers	M3 – User-generated content and specialist features complete
10-13	Sprint 4	RBAC middleware, admin dashboard (user/order/plant management), analytics dashboard, integration testing, security hardening, final documentation	M4 – Complete system ready for deployment

A high-level Gantt chart representation is shown below (weeks on the horizontal axis, tasks as bars). All tasks are performed solely by the student developer.

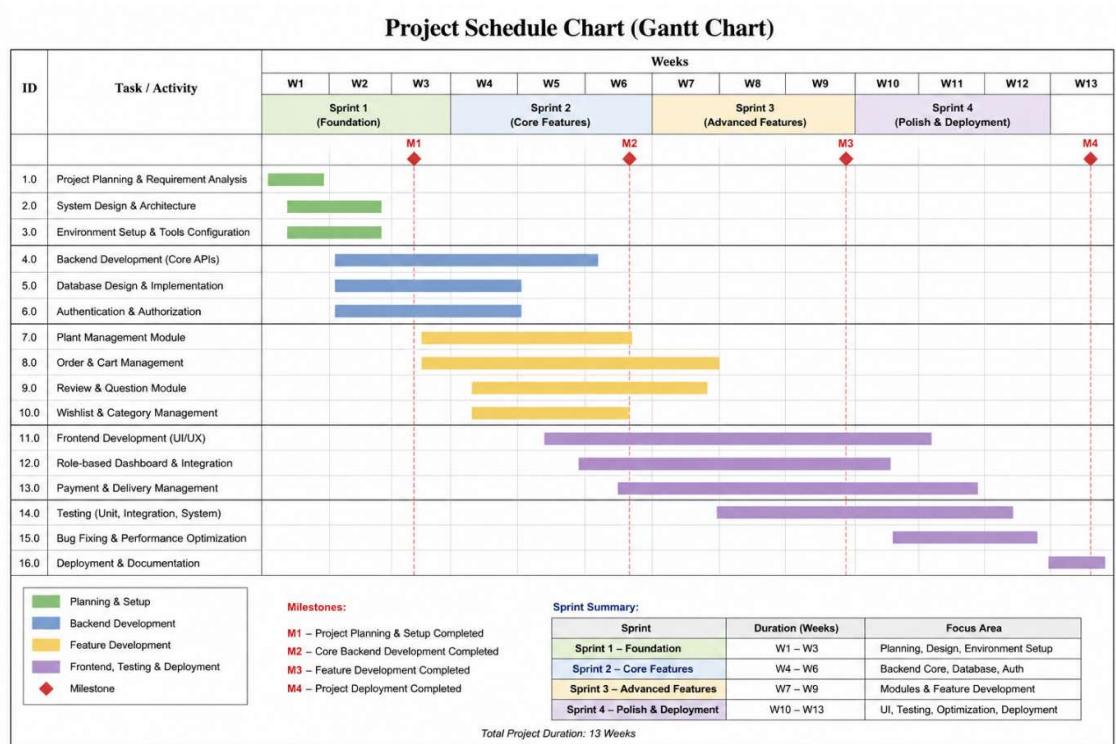


Figure 6.1: Gantt Chart – Project Schedule (13-Week Timeline)

6.5 Accounts Table

The total project cost is calculated in Bangladeshi Taka (TK). All software is free and open-source, and the development is performed by a student as part of academic coursework, resulting in a near-zero personnel cost.

6.5.1 Personnel Cost

Table 6.12: Personnel Cost:

Role	Headcount	Monthly Salary (TK)	Duration	Total Cost (TK)
Student Developer	1	0 (academic project)	13 weeks (3 months)	0

Role	Headcount	Monthly Salary (TK)	Duration	Total Cost (TK)
Total Personnel Cost	1			0

6.5.2 Expected Hardware Cost

Table 6.13: Expected Hardware Cost:

Item	Quantity	Total Cost (TK)	Remarks
Development Laptop (mid-range)	1	60,000 (one-time)	Cost is 0 TK if a suitable laptop is already owned.
Total Hardware Cost		0 – 60,000	

6.5.3 Expected Software Cost

All tools and frameworks are free and open-source (Visual Studio Code, Laravel, Next.js, MySQL, XAMPP, Composer, npm, Git/GitHub, Laravel Sanctum, Tailwind CSS). No licence fees are incurred.

Total Software Cost: 0 TK

6.5.4 Expected Other Cost

Table 6.14: Expected Other Cost:

Item	Cost (TK)	Remarks
Domain Name (optional)	1,800 / year	Required only if the project is publicly deployed.
Contingency	2,000	For unforeseen minor expenses.
Total Other Cost	3,800	

6.5.5 Total Project Cost Summary

Table 6.15: Total Project Cost Summary:

Category	Amount (TK)
Personnel Cost	0
Hardware Cost	0 – 60,000
Software Cost	0
Other Cost	3,800
Total Estimated Cost	3,800 – 63,800

The entire project can therefore be developed for a maximum outlay of **Tk 63,800**, and typically for only **Tk 3,800** when a development laptop is already available.

Chapter 7.

Designing

7.1 Interface Design

7.1.1 Design Principles

Table 7.1: Interface Design Principles:

Principle	Application in Plantify
Consistency	Uniform color scheme, typography, spacing, and component behavior using Tailwind CSS
Simplicity	Clean, uncluttered layouts with clear visual hierarchy
Responsiveness	Mobile-first responsive design via Tailwind breakpoints
Feedback	Visual feedback for all user actions
Accessibility	Semantic HTML, color contrast, keyboard navigation
User Control	Clear navigation, undo/cancel options, confirmation dialogs

7.1.2 Color Palette & Typography

Table 7.2: Color Palette & Typography:

Design Element	Specification
Primary Color	Green (#16a34a) — plants, growth, nature
Secondary Color	Emerald (#059669) — accents
Background	White / Gray-50 (#f9fafb)
Text Color	Gray-900 primary, Gray-600 secondary
Danger/Error	Red (#dc2626)
Font Family	Inter (system sans-serif)
Font Sizes	Headings: 24–36px; Body: 16px; Small: 14px

7.1.3 Layout Structure — Page System Design

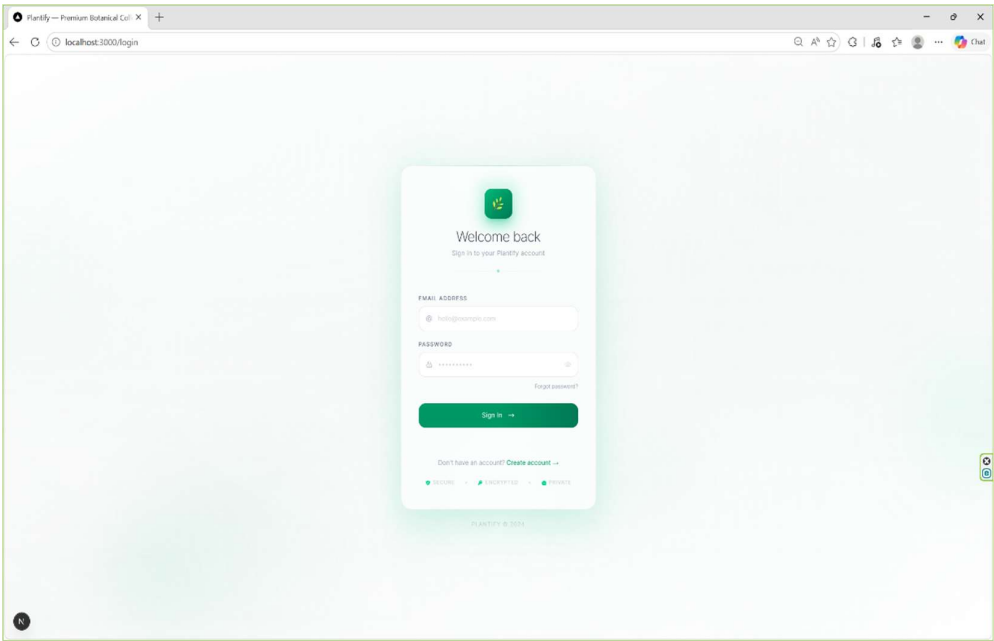


Figure 7.1: Login Page Interface Design

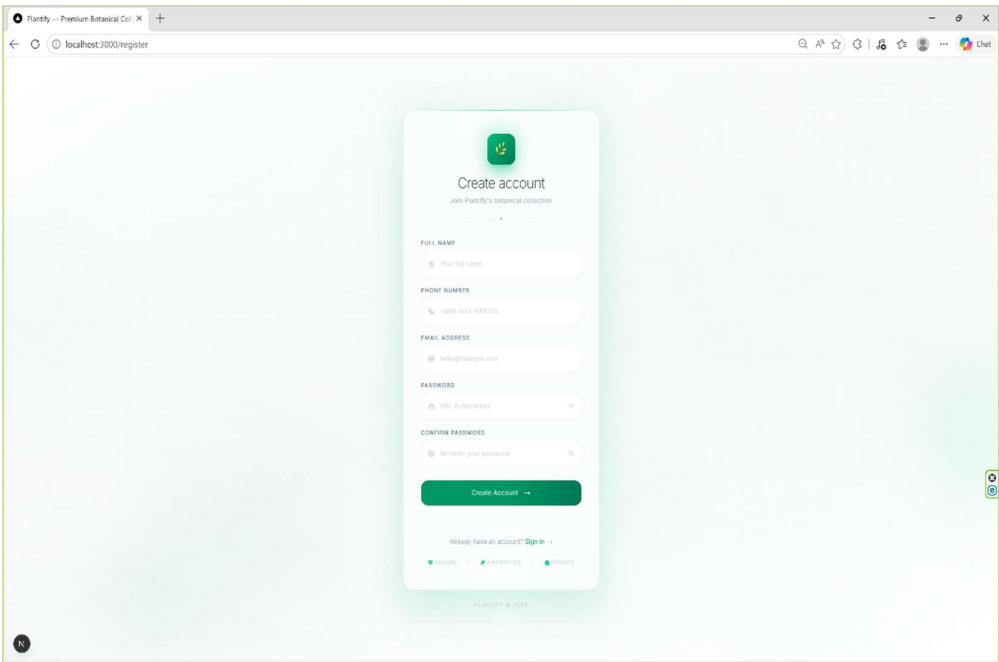


Figure 7.2: Register Page Interface Design

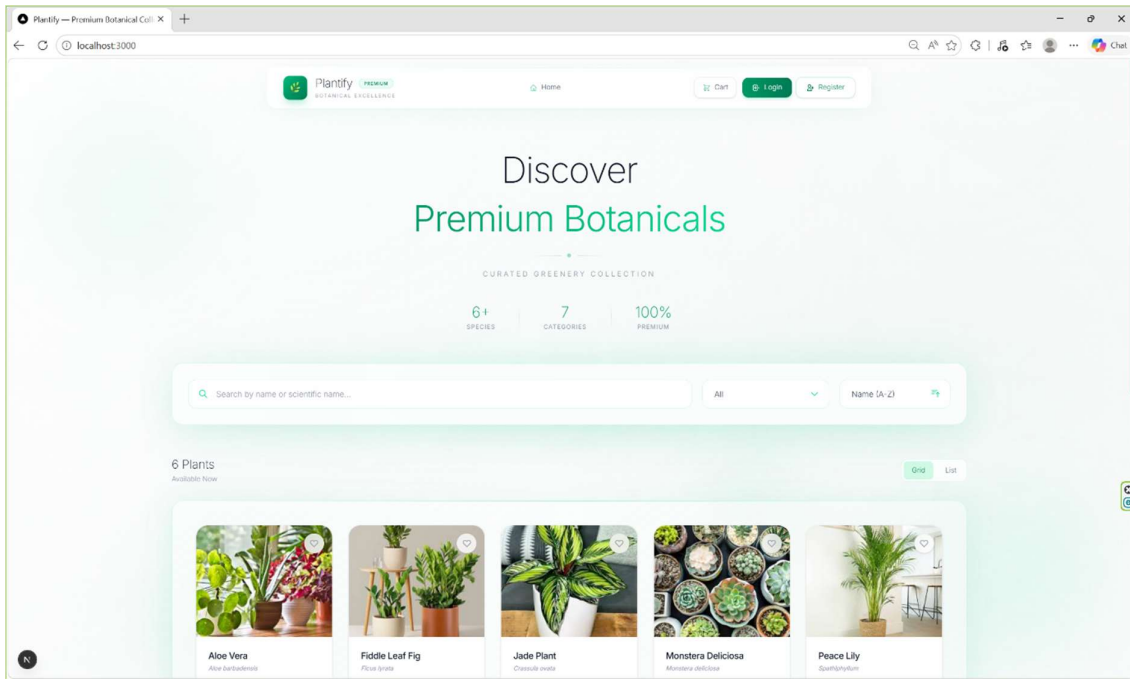


Figure 7.3: Home Page Interface Design

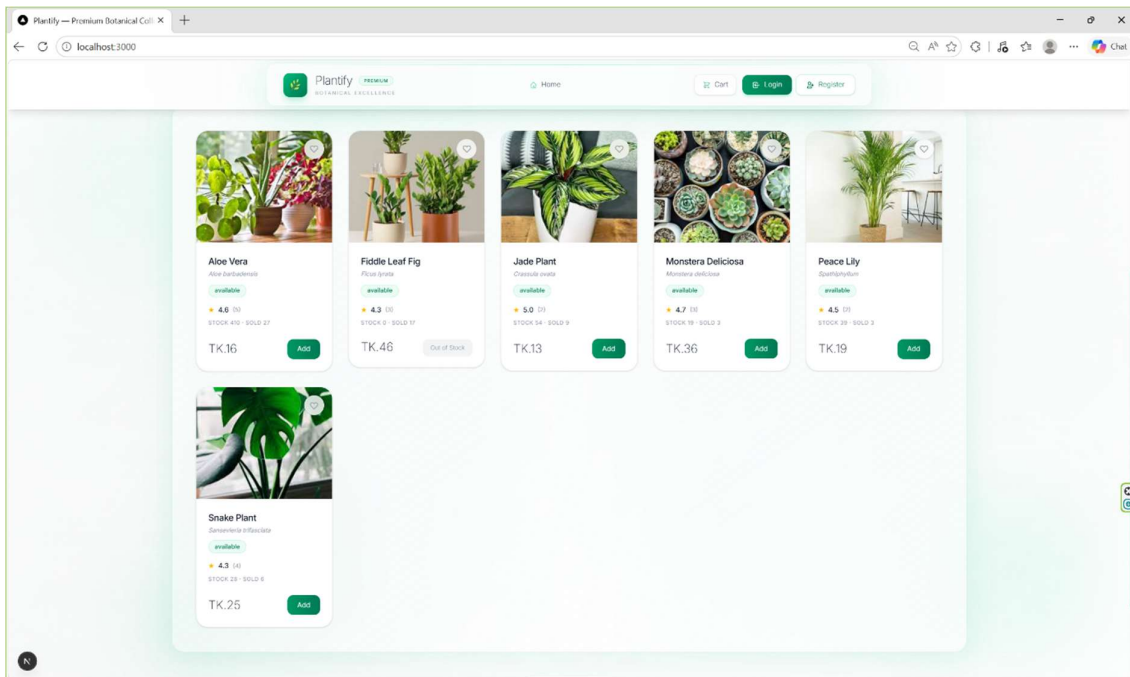


Figure 7.4: Plant Card Interface Design

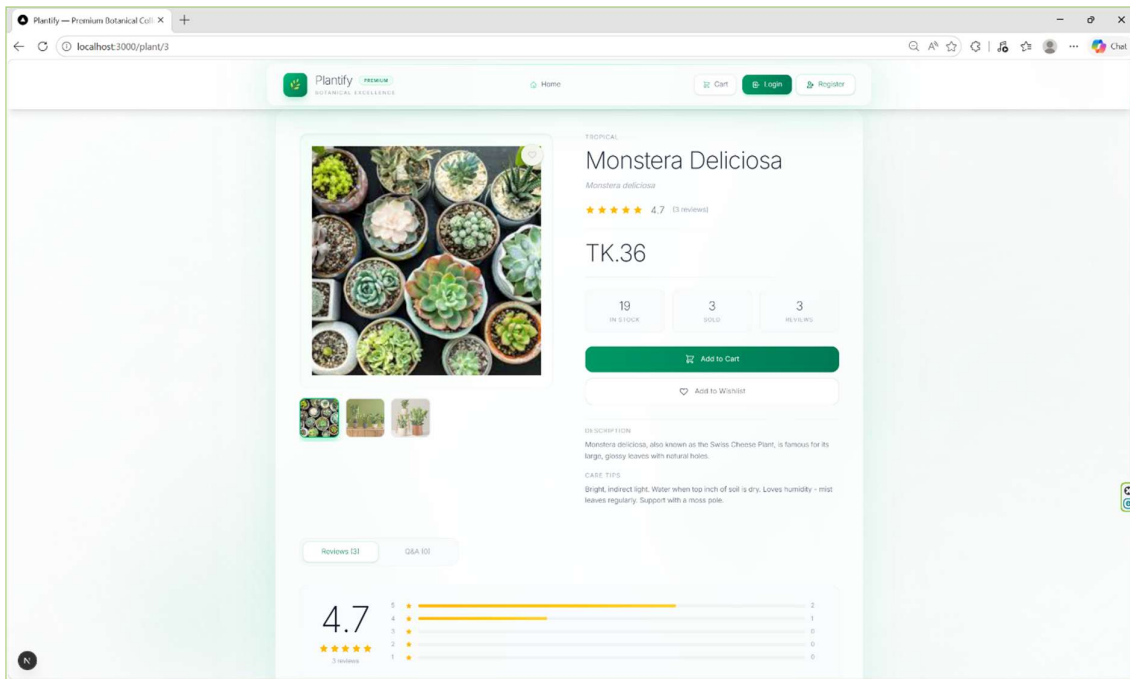


Figure 7.5: Plant Detail Page Interface Design

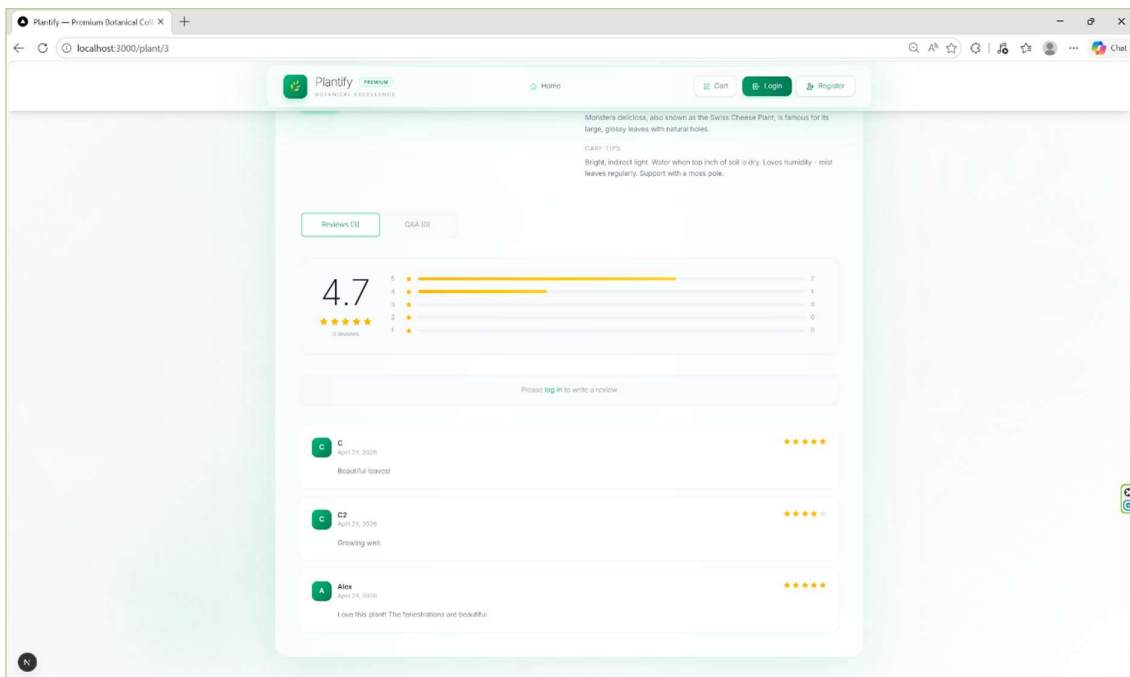


Figure 7.6: Plant Review and Q&A Interface Design

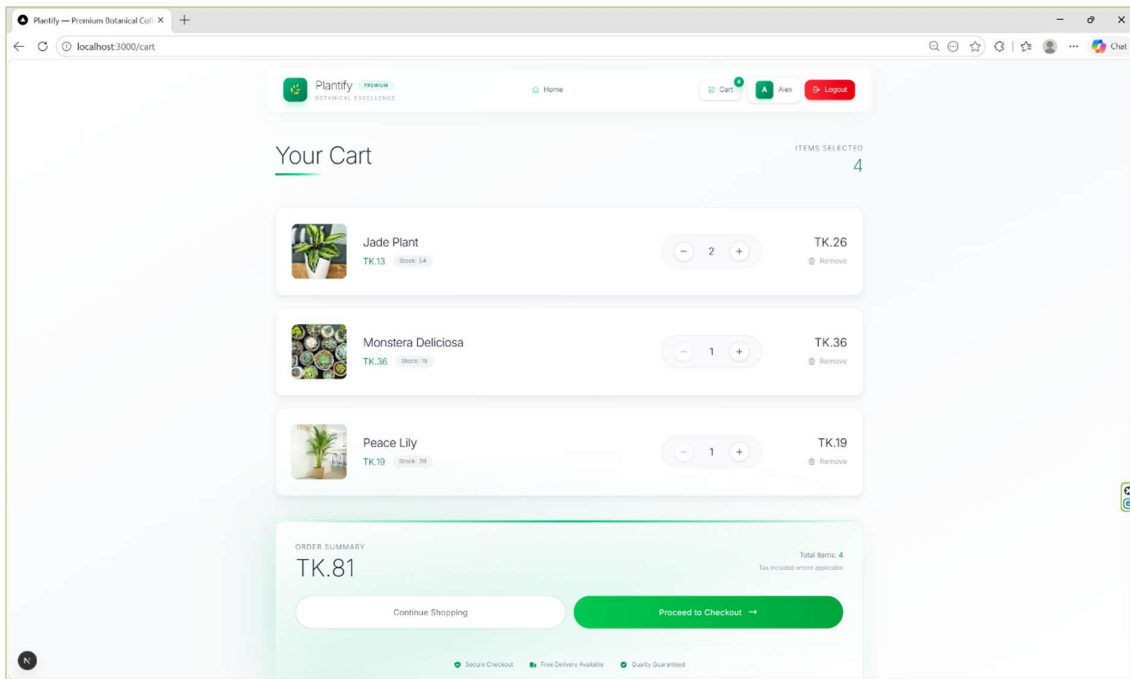


Figure 7.7: Cart Page Interface Design

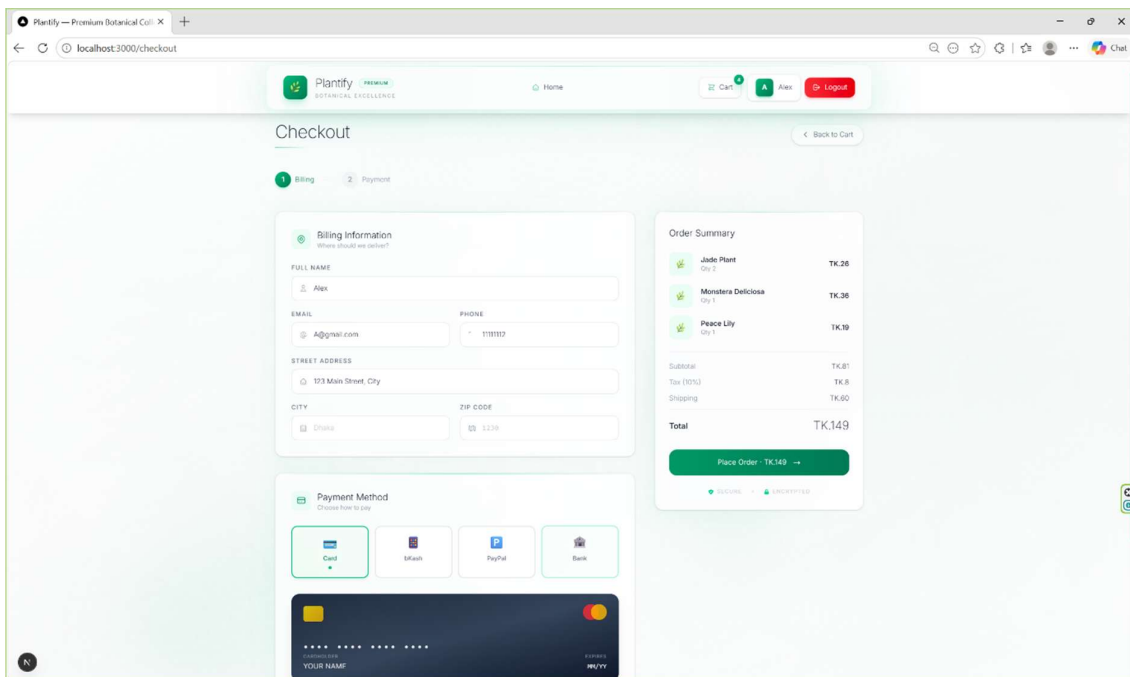


Figure 7.8: Checkout Page Interface Design

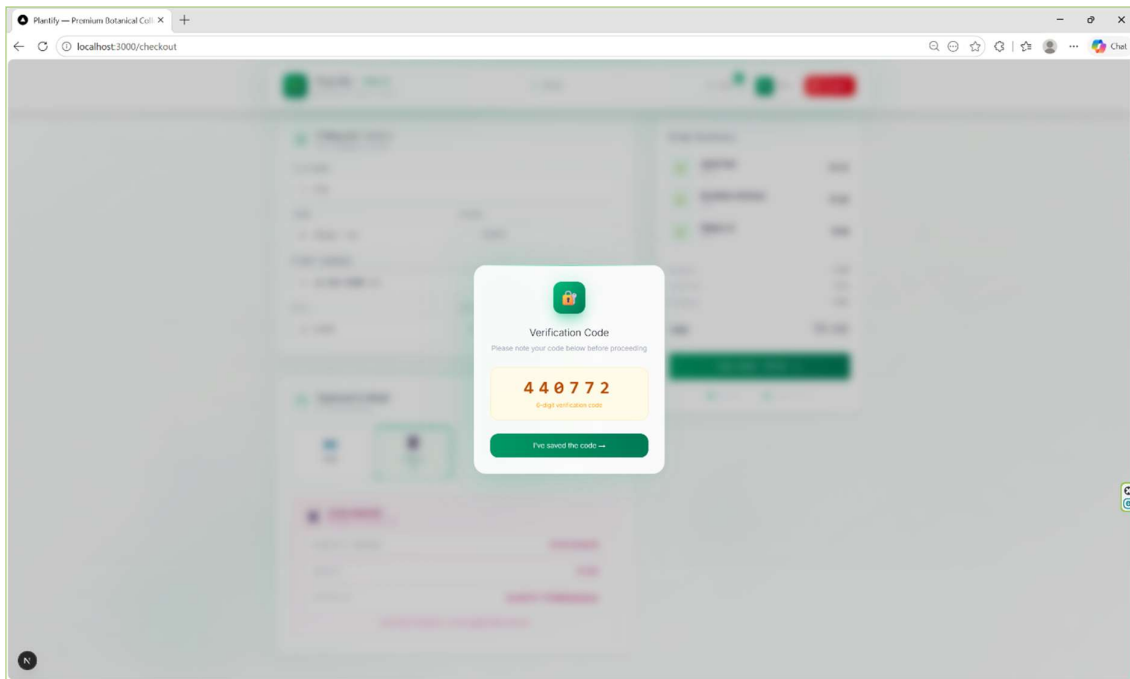


Figure 7.9: Verification Code Request Interface Design

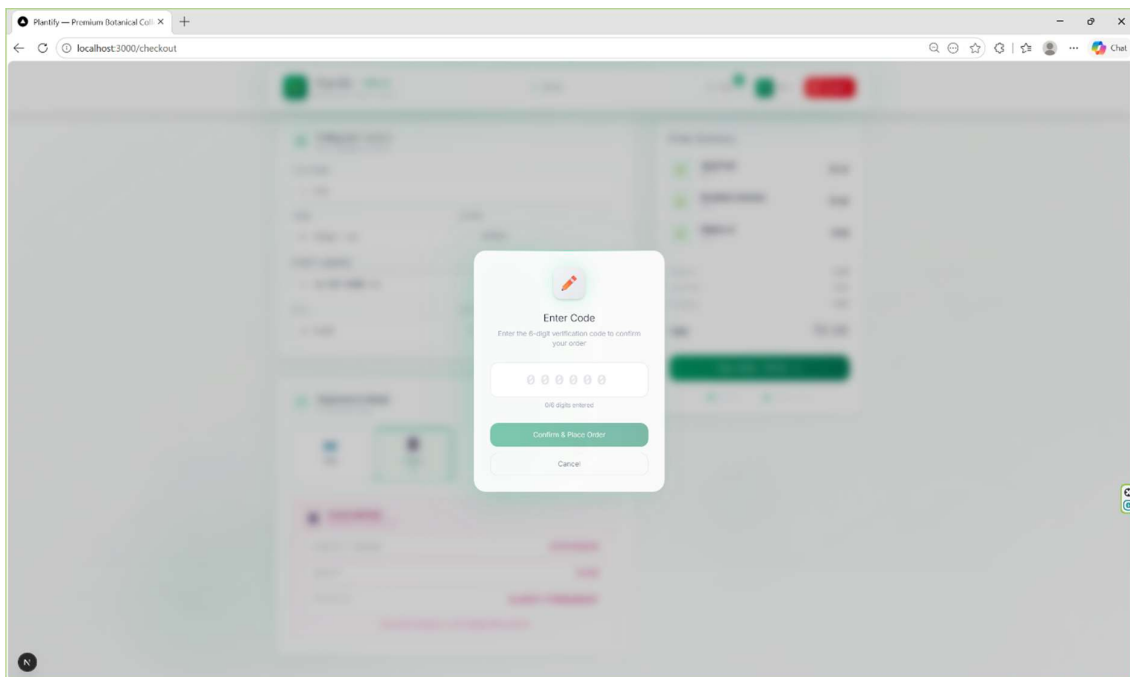


Figure 7.10: Enter Verification Code Interface Design

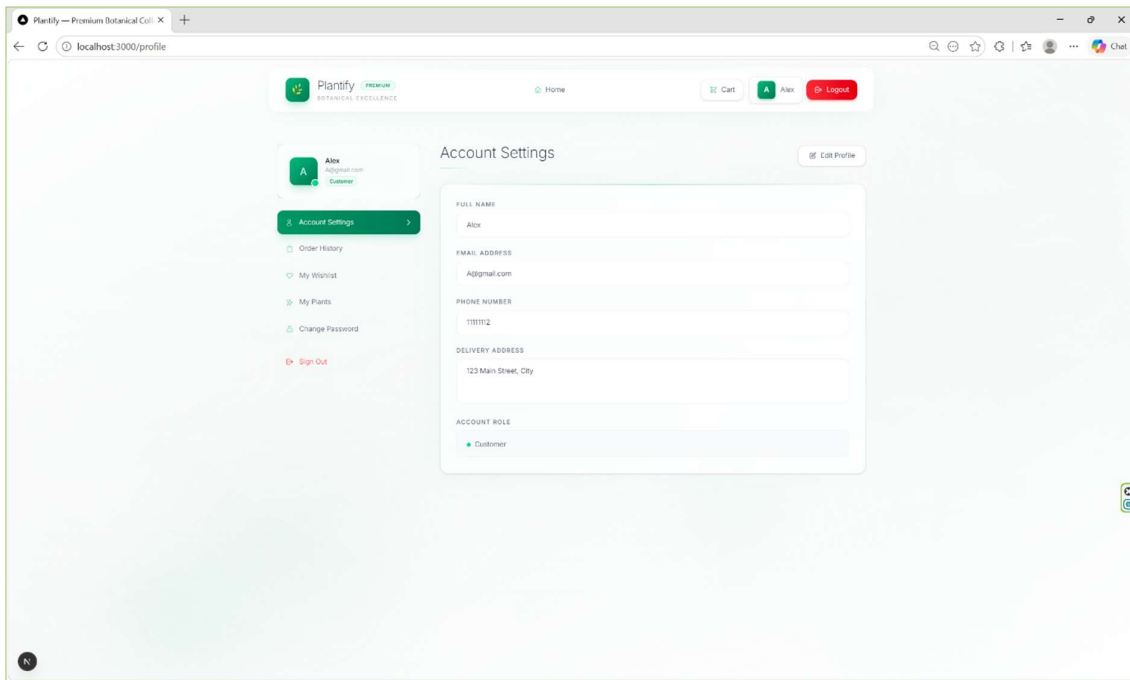


Figure 7.11: Customer Profile Page Interface Design

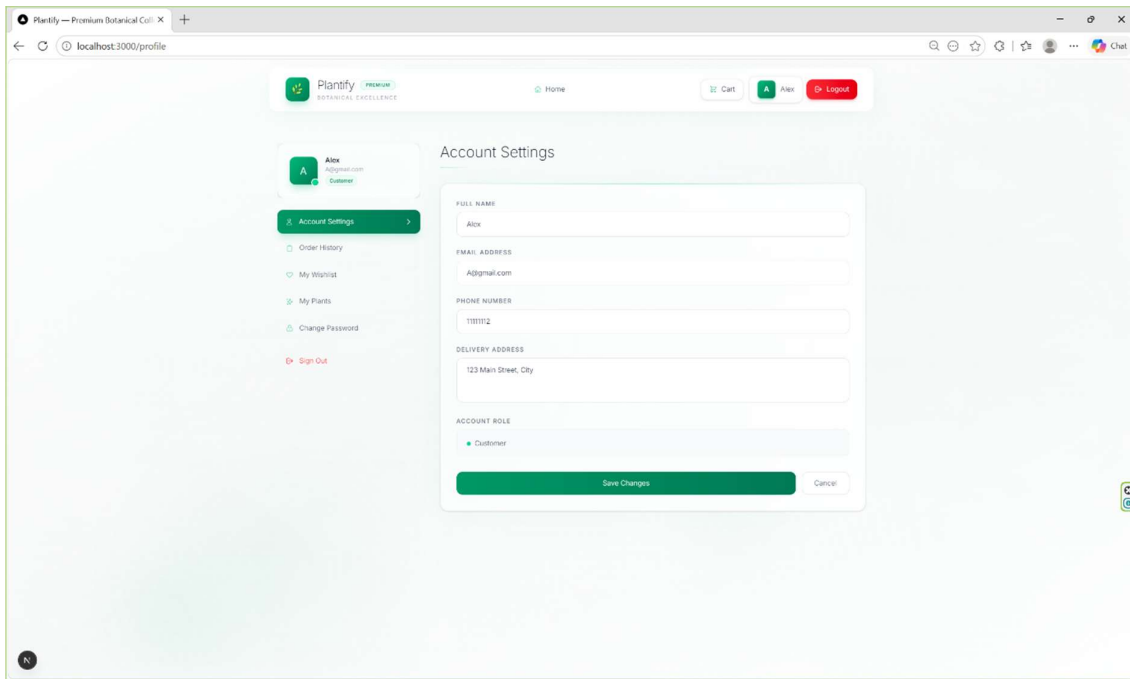


Figure 7.12: Customer Profile Edit Page Interface Design

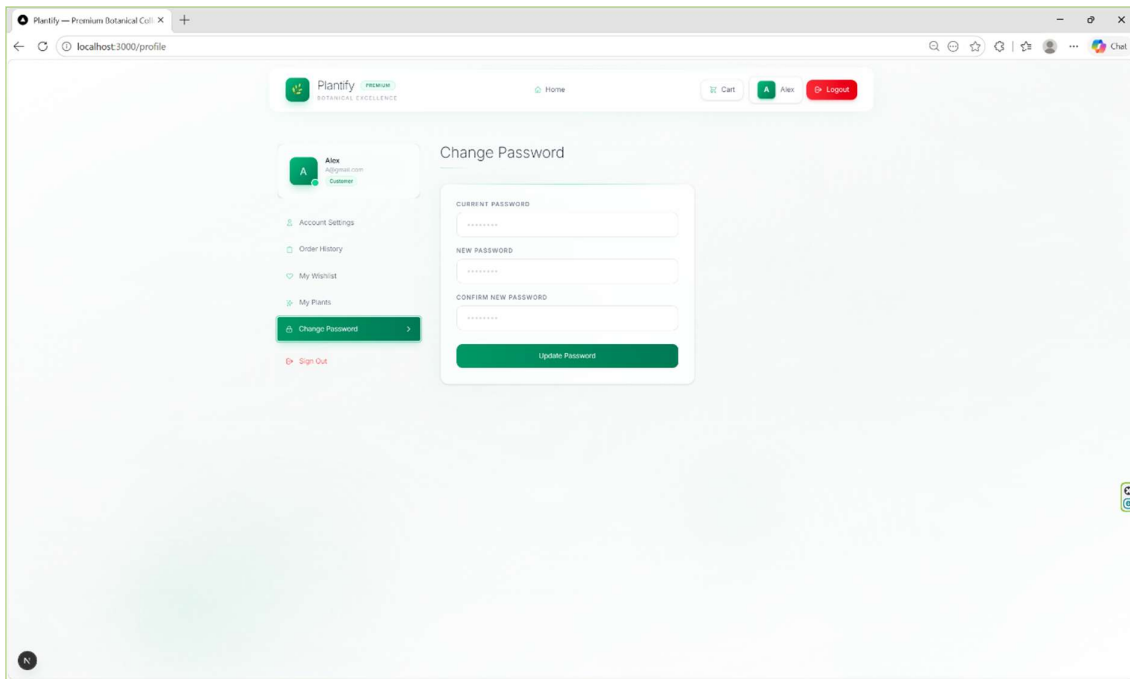


Figure 7.13: Change Password Page Interface Design

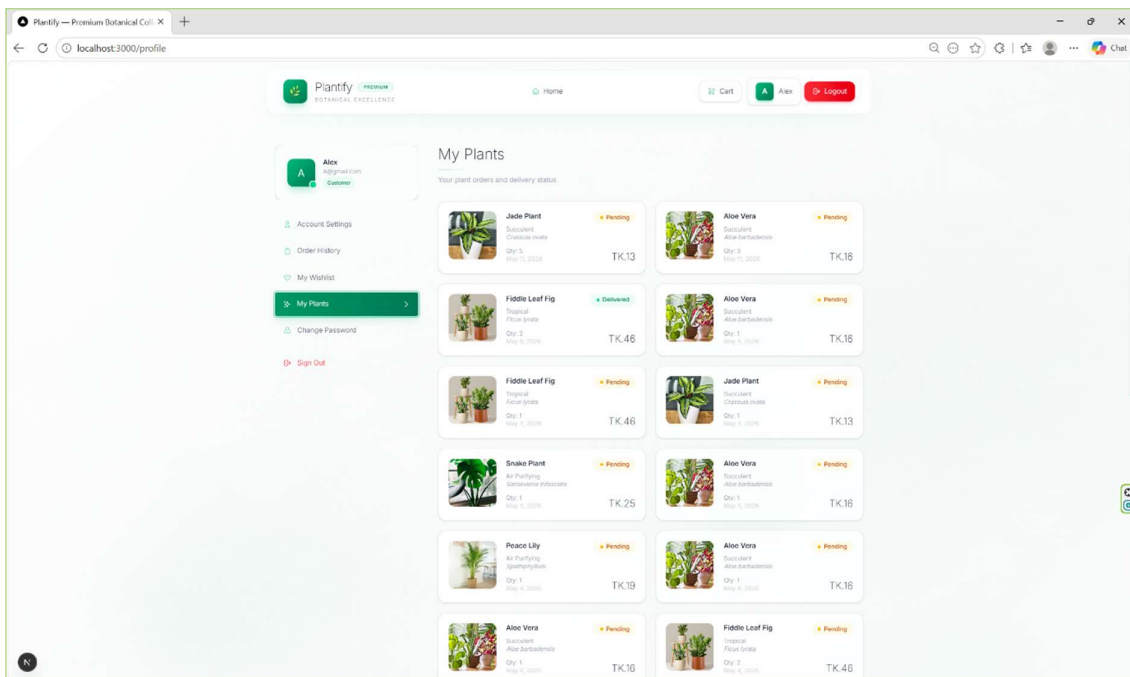


Figure 7.14: My Plants Page Interface Design

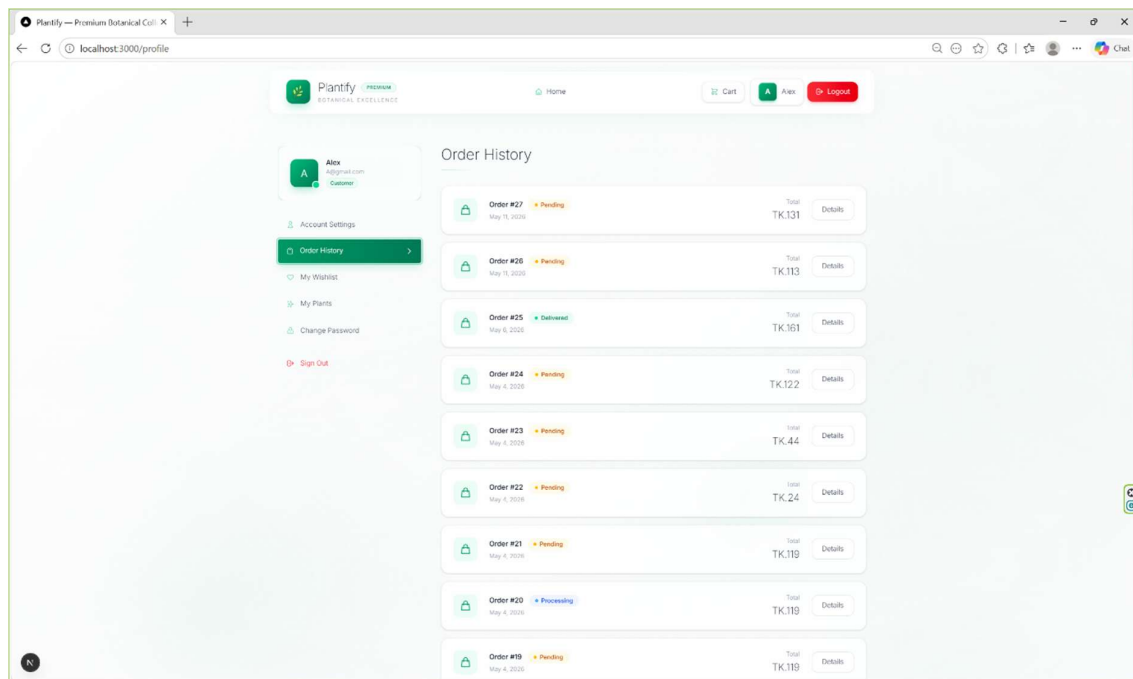


Figure 7.15: Order History Page Interface Design

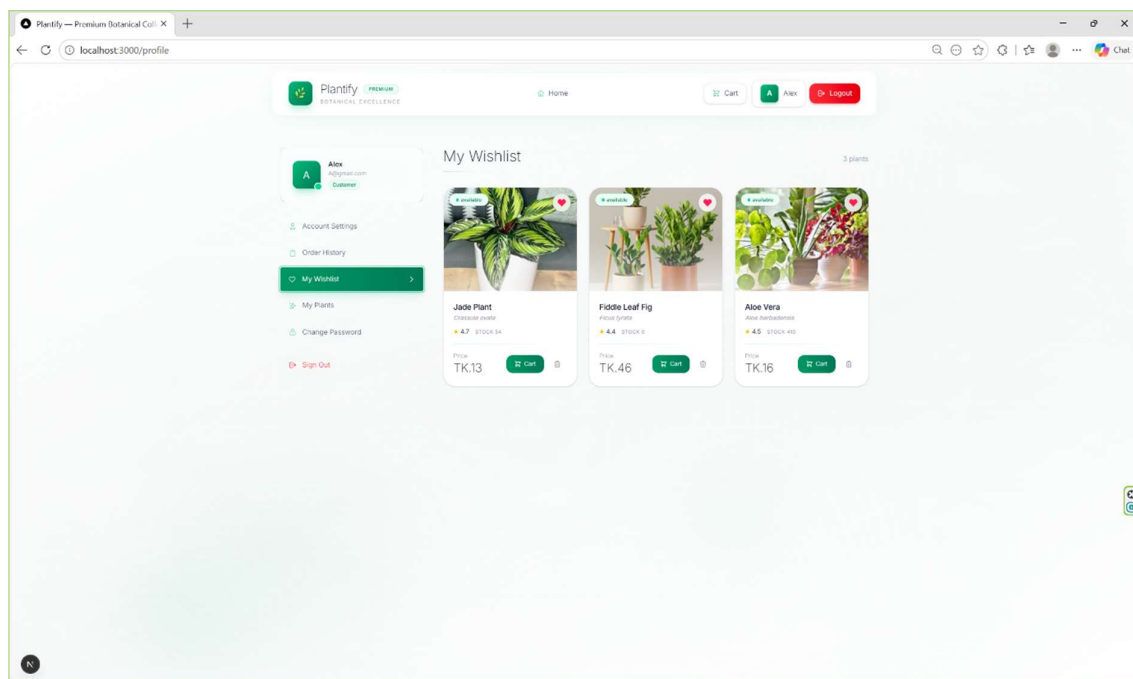


Figure 7.16: Customer Wishlist Page Interface Design

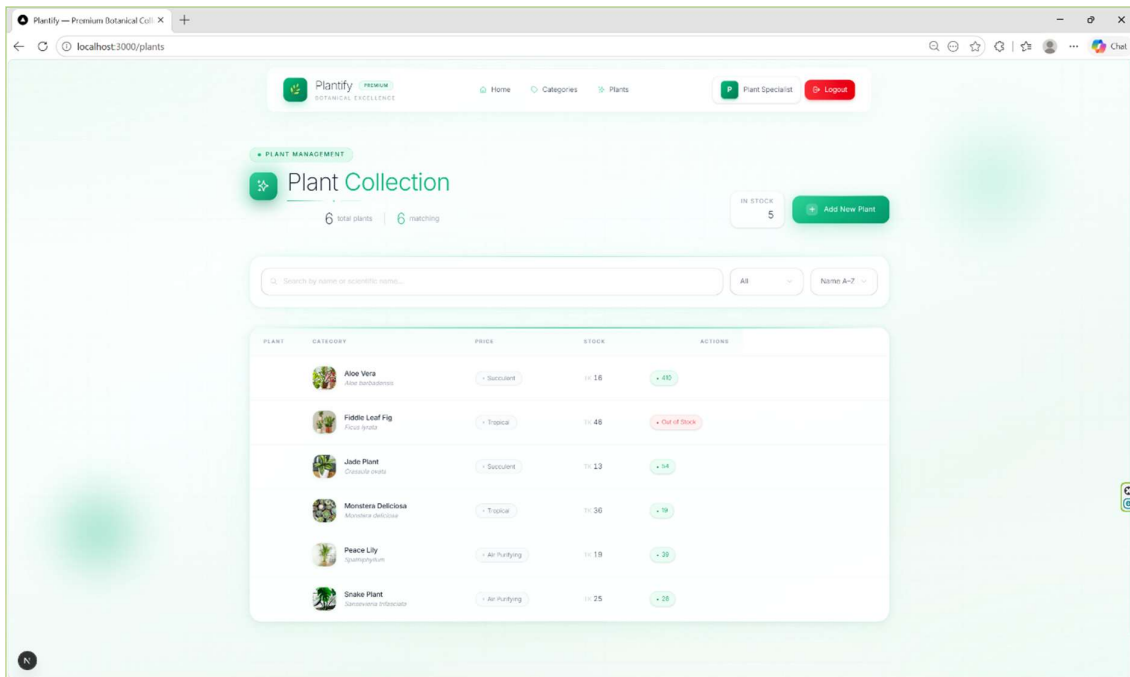


Figure 7.17: Plant Management Interface Design

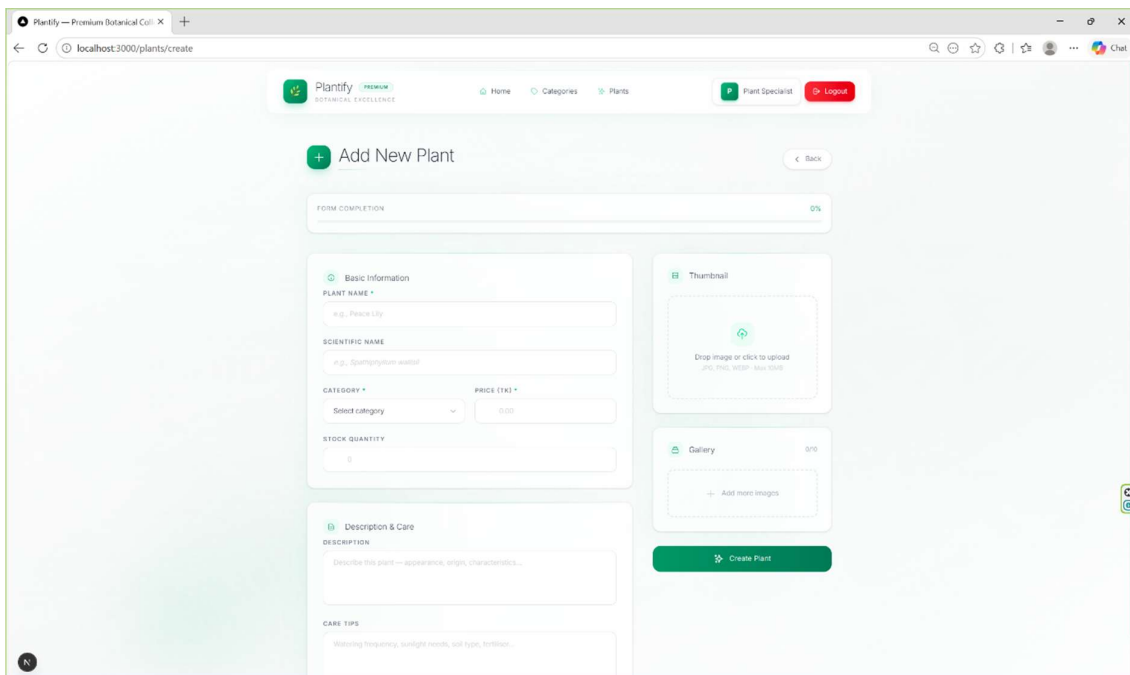


Figure 7.18: Create Plants Interface Design

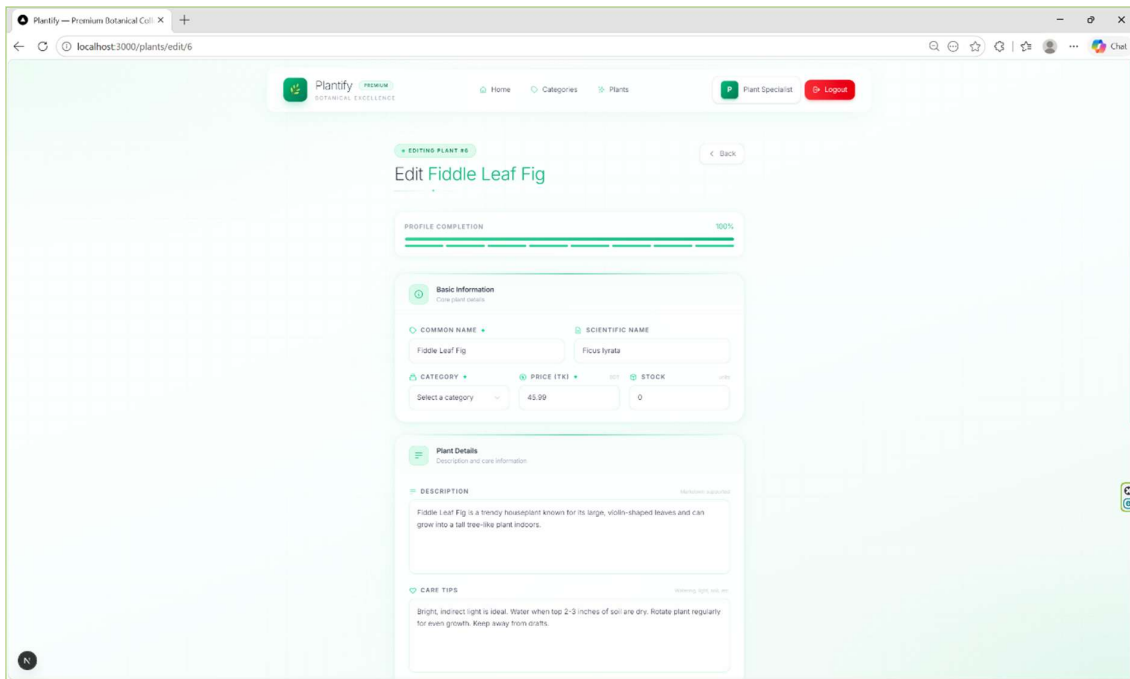


Figure 7.19: Edit Plants Interface Design

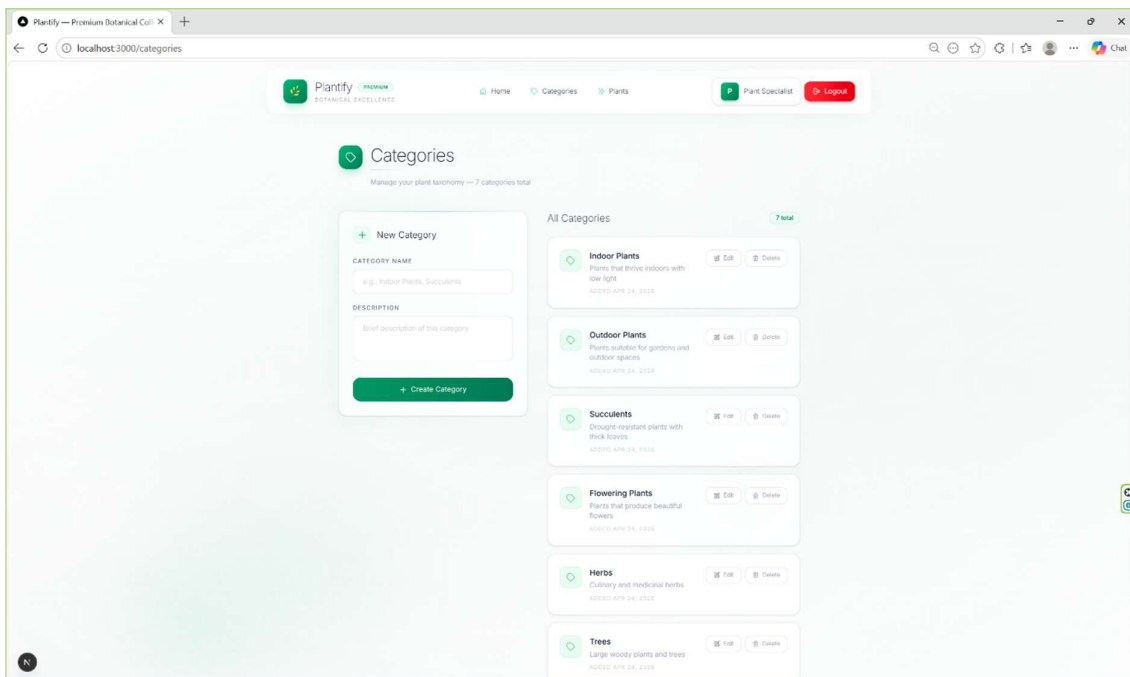


Figure 7.20: Category CRUD Interface Design

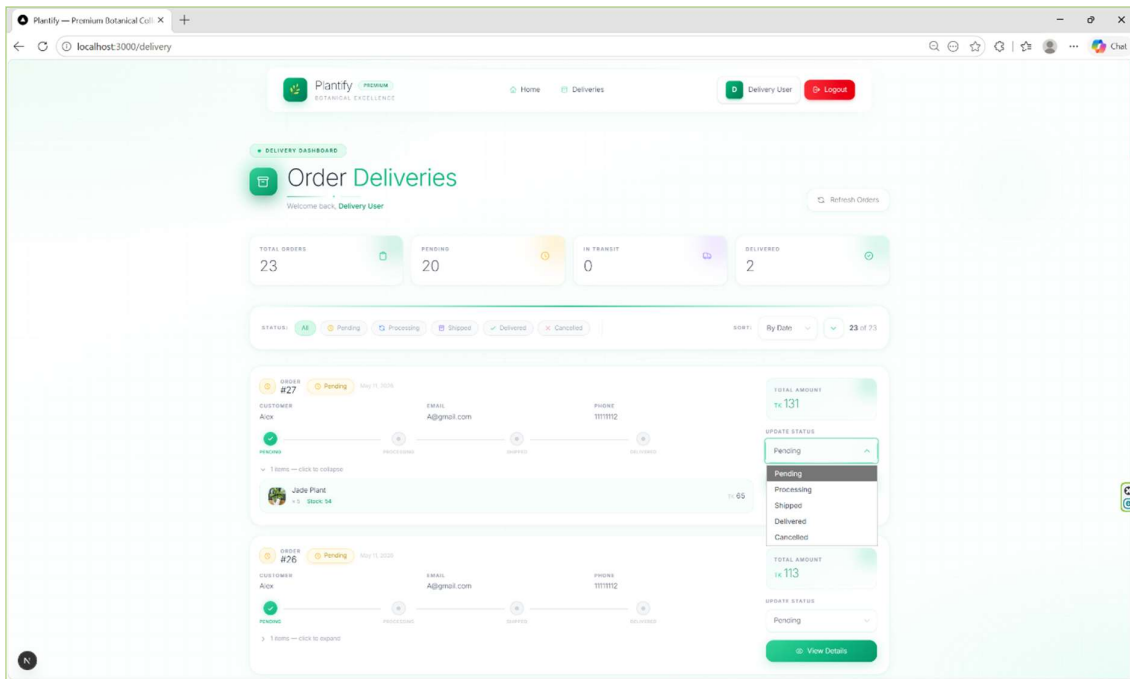


Figure 7.21: Delivery Dashboard Interface Design

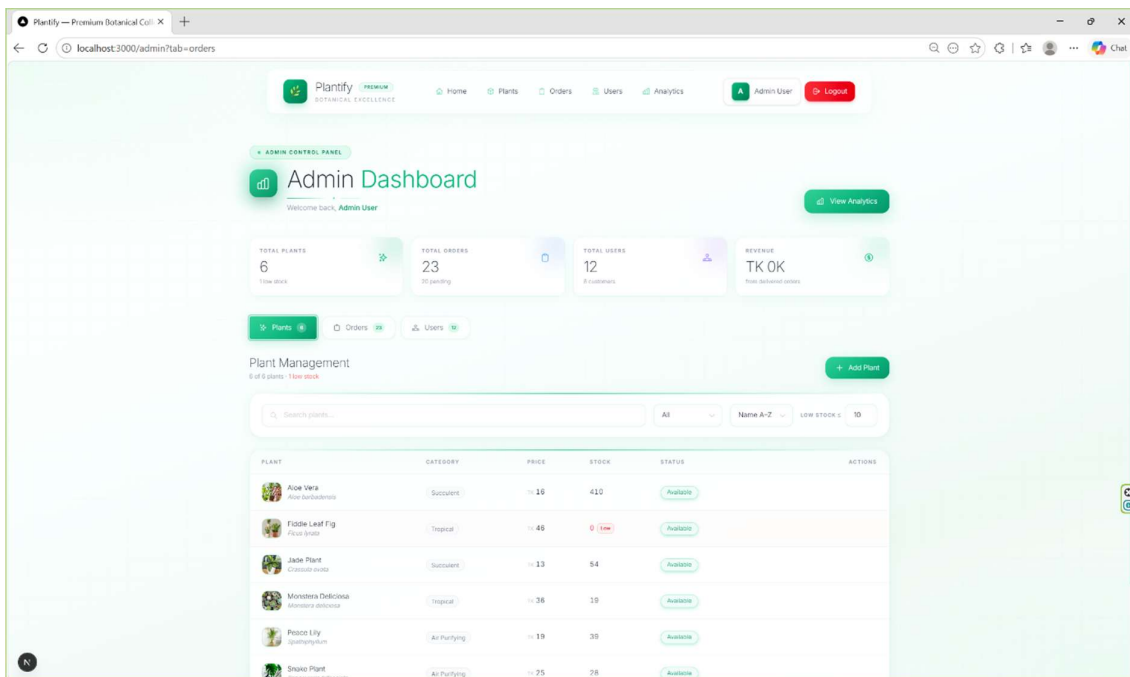


Figure 7.22: Admin Plant Management Interface Design

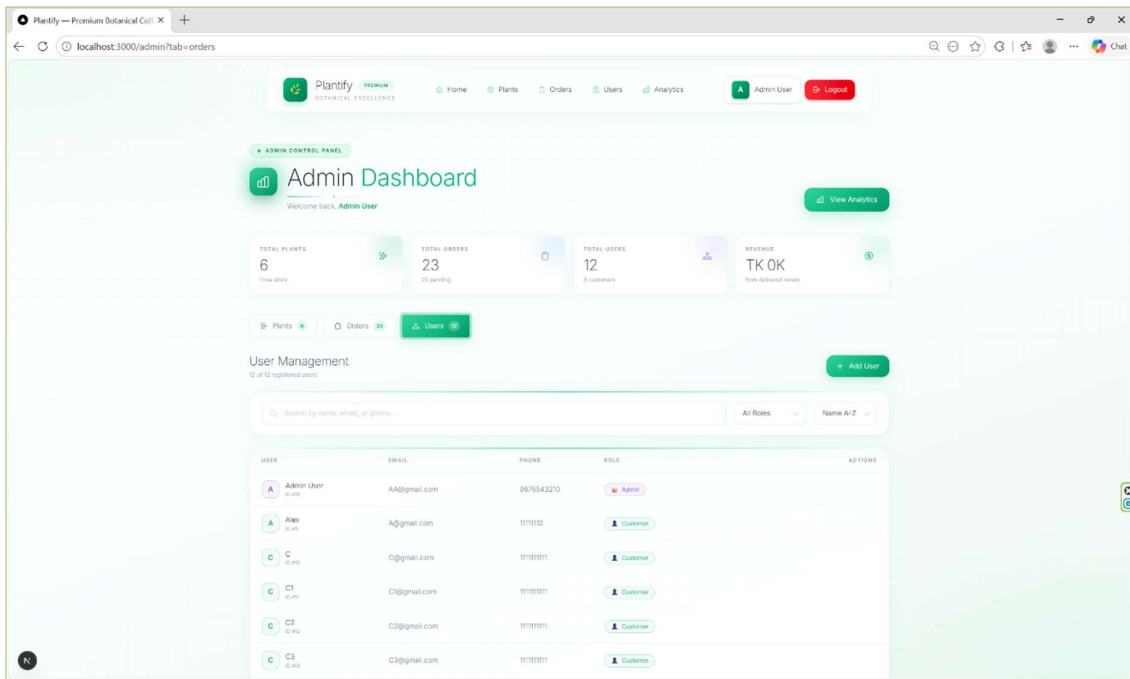


Figure 7.23: Admin User Management Interface Design

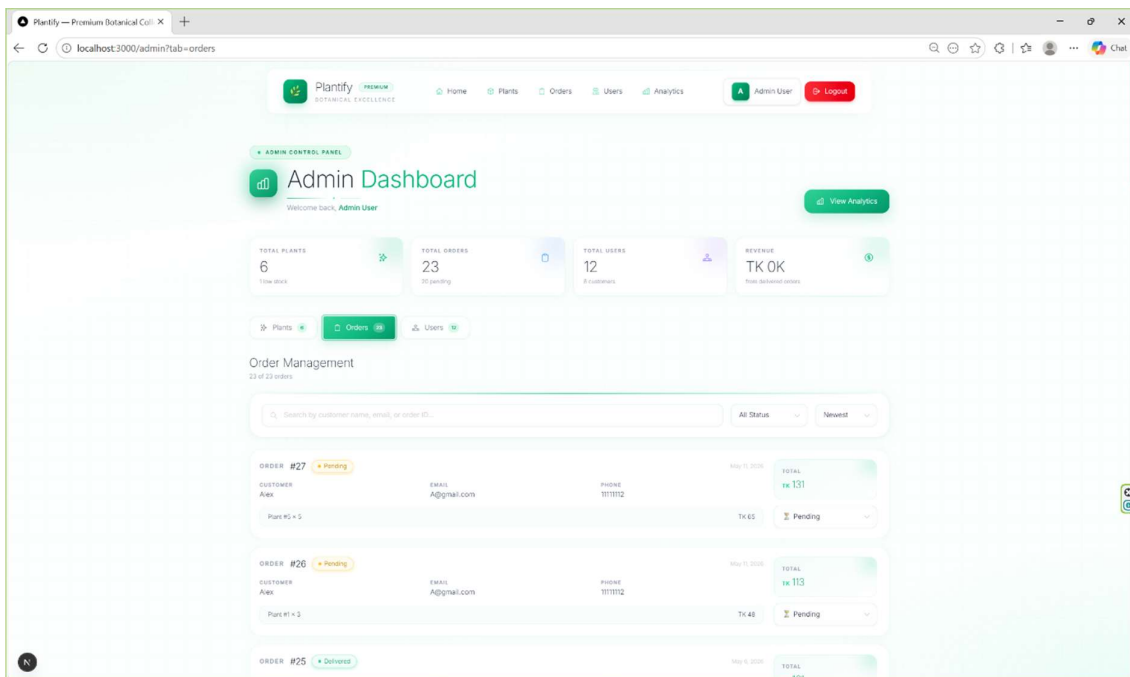


Figure 7.24: Admin Order Management Interface Design

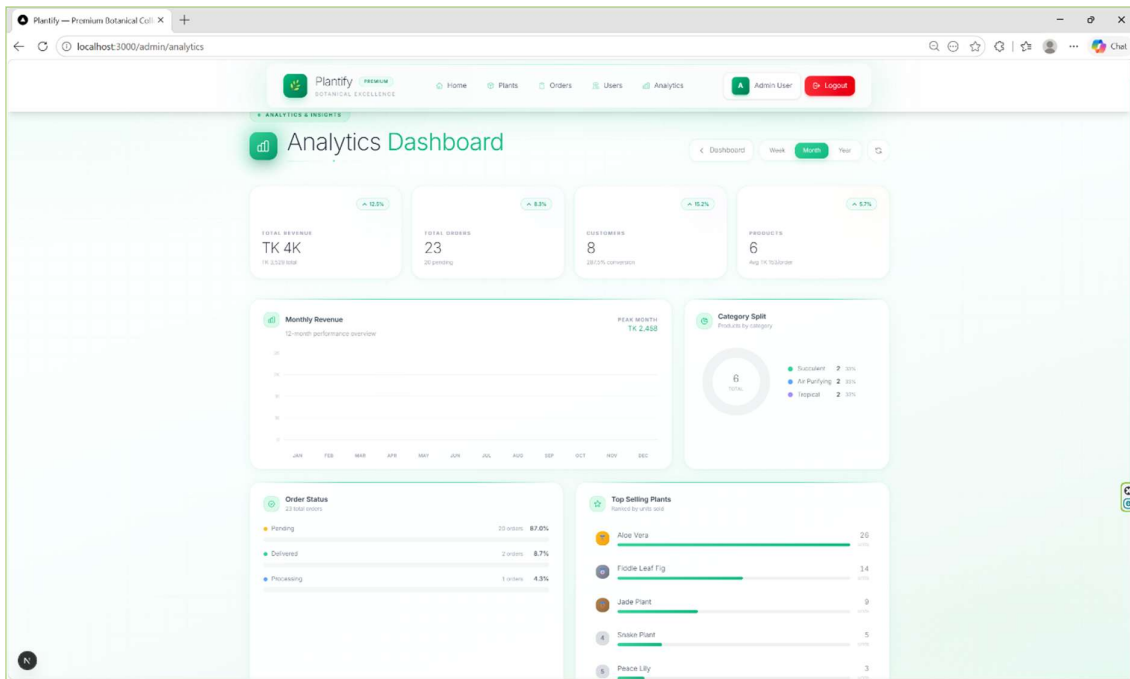


Figure 7.25: Admin Analytics Dashboard Interface Design

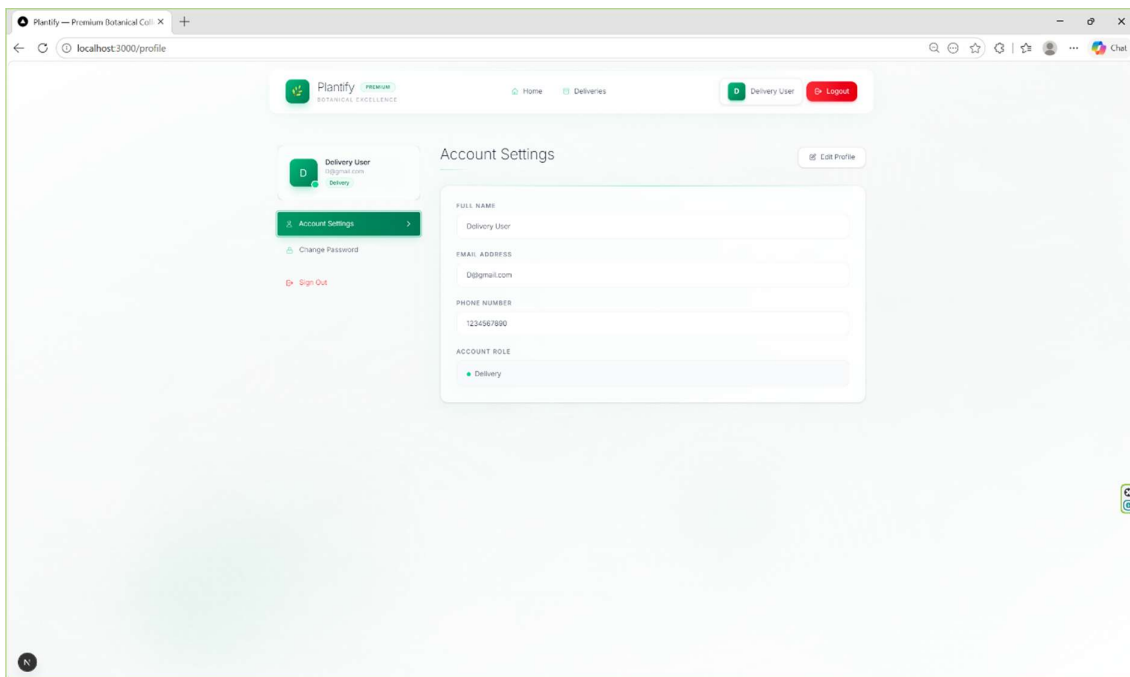


Figure 7.26: Employer Profile Interface Design

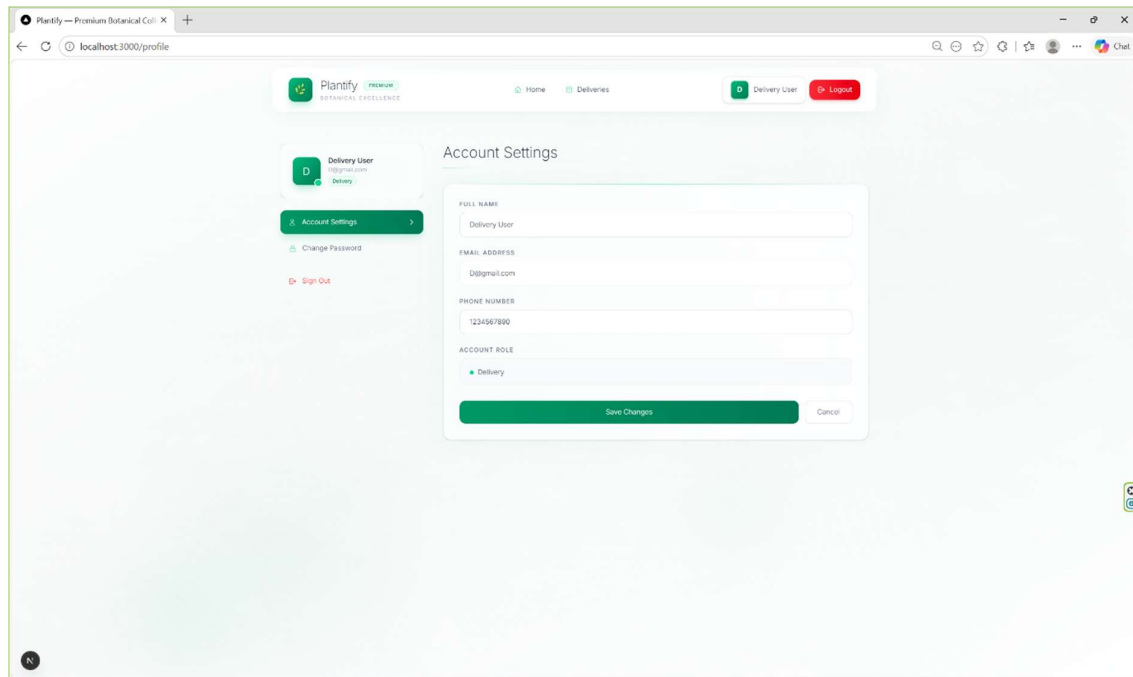


Figure 7.27: Employer Profile Edit Interface Design

7.2 Data Flow Diagram

7.2.1 DFD LVL 0

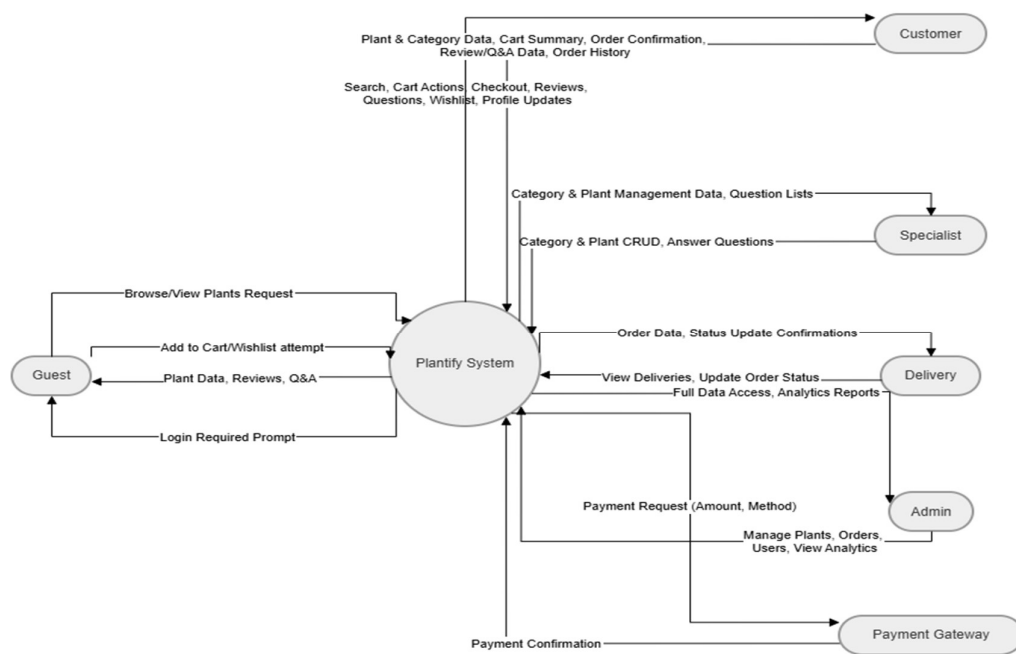


Figure 7.28: Data Flow Diagram – Level 0

7.2.2 DFD LVL 1

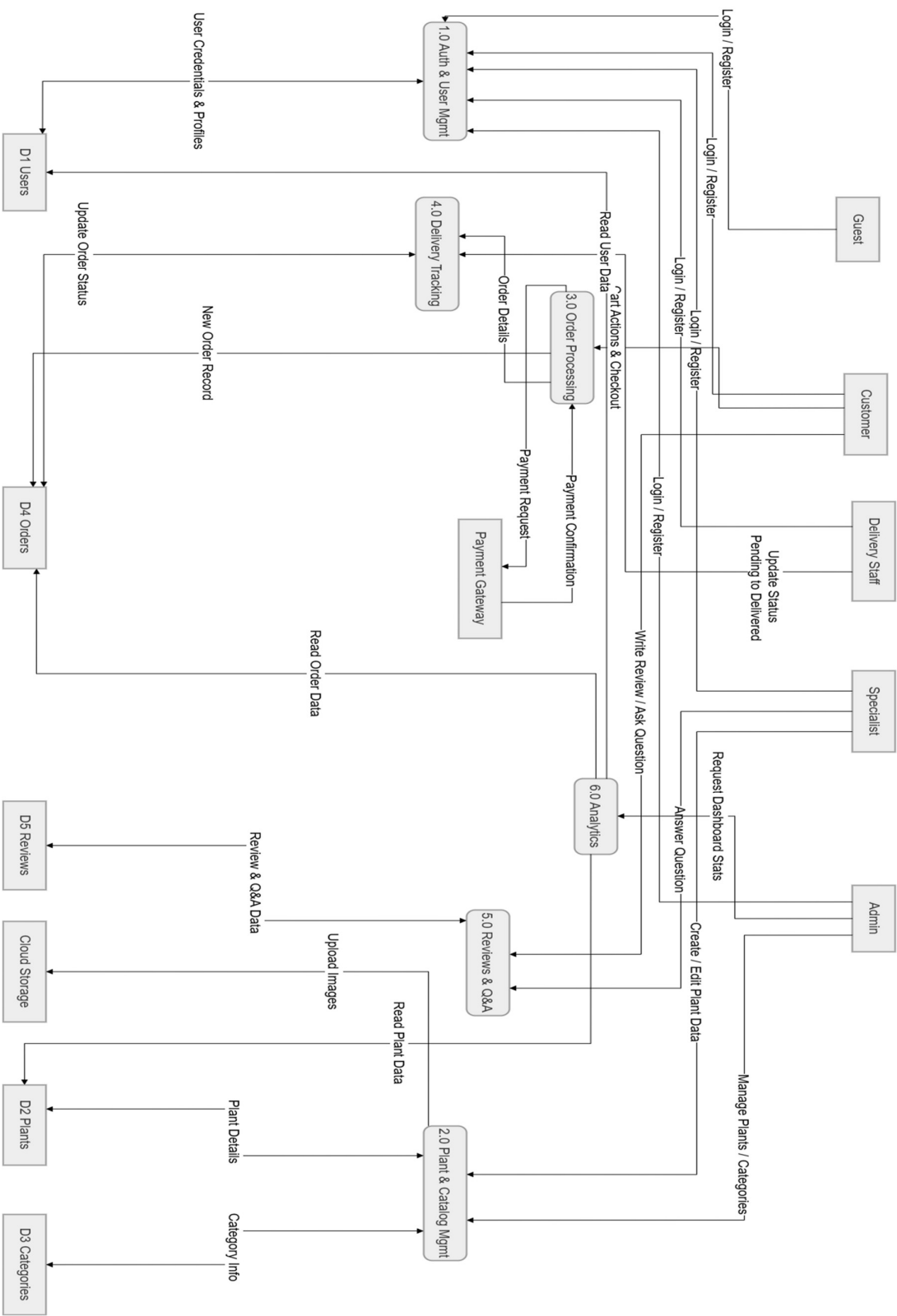


Figure 7.29: Data Flow Diagram – Level 1

7.2.3 DFD LVL 2 (Auth & User Management)

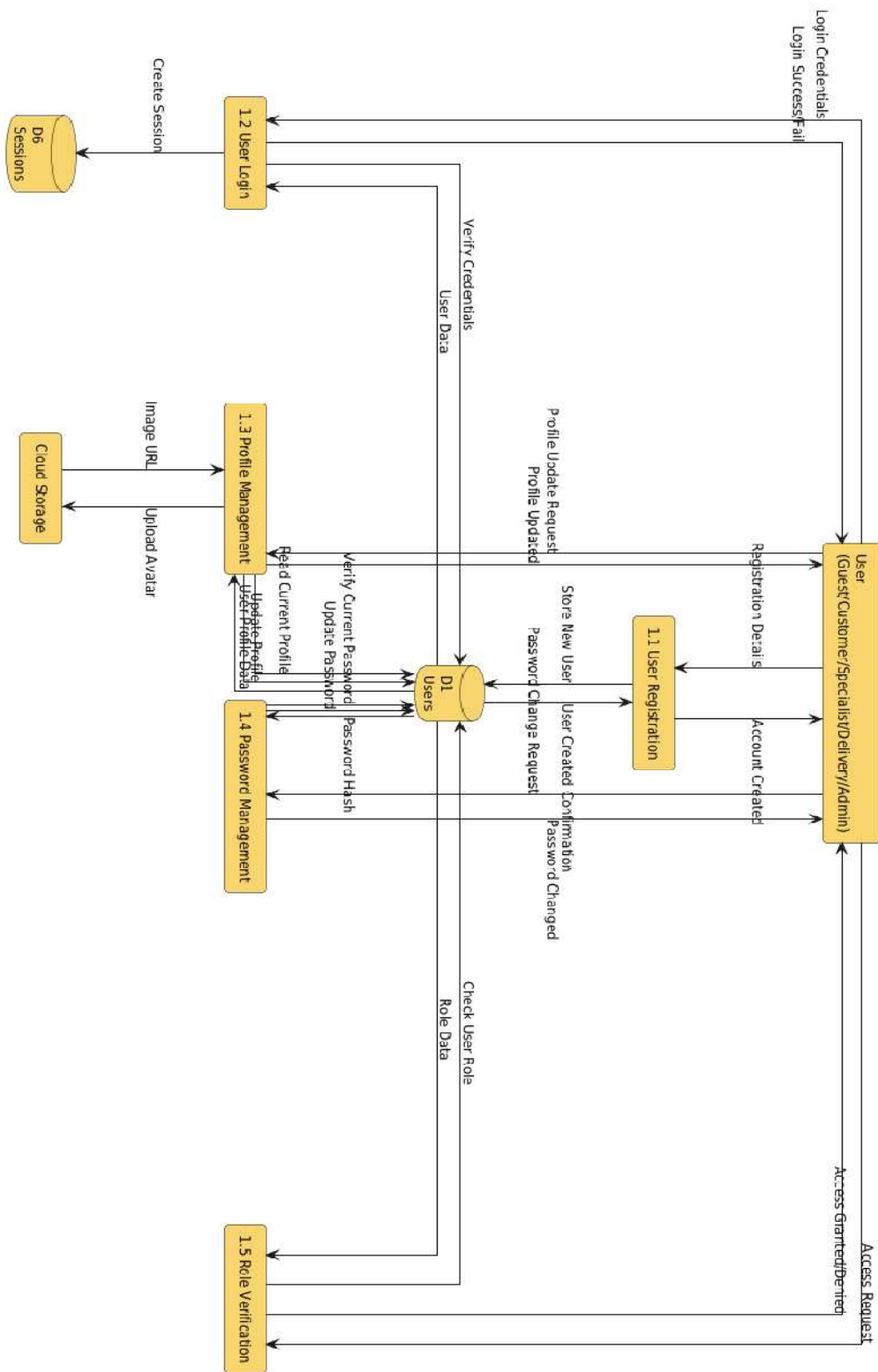


Figure 7.30: Data Flow Diagram – Level 2 (Auth & User Management)

7.2.4 DFD LVL 2 (Plant & Catalog Management)

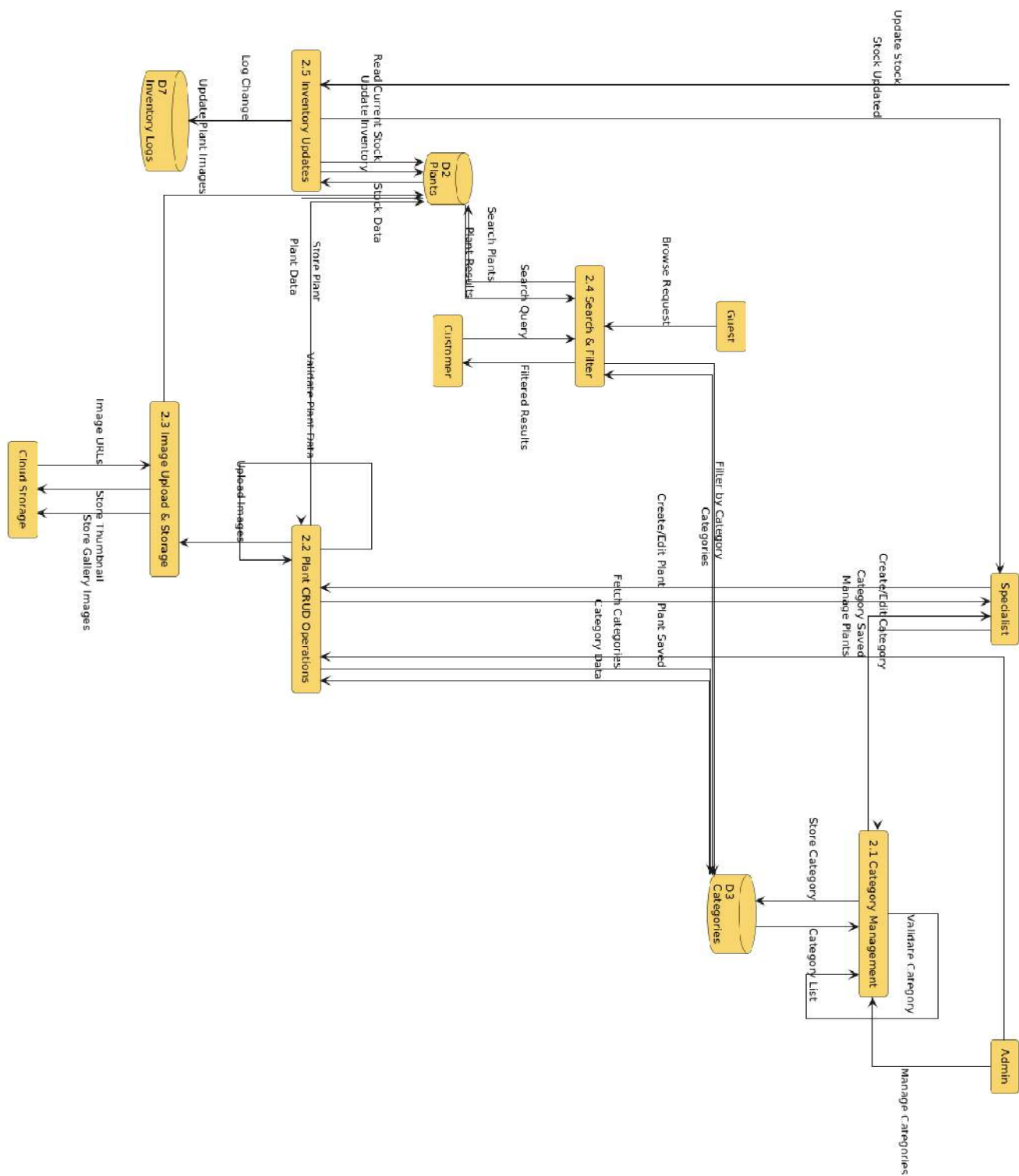


Figure 7.31: Data Flow Diagram – Level 2 (Plant & Catalog Management)

7.2.5 DFD LVL 2 (Order Processing)

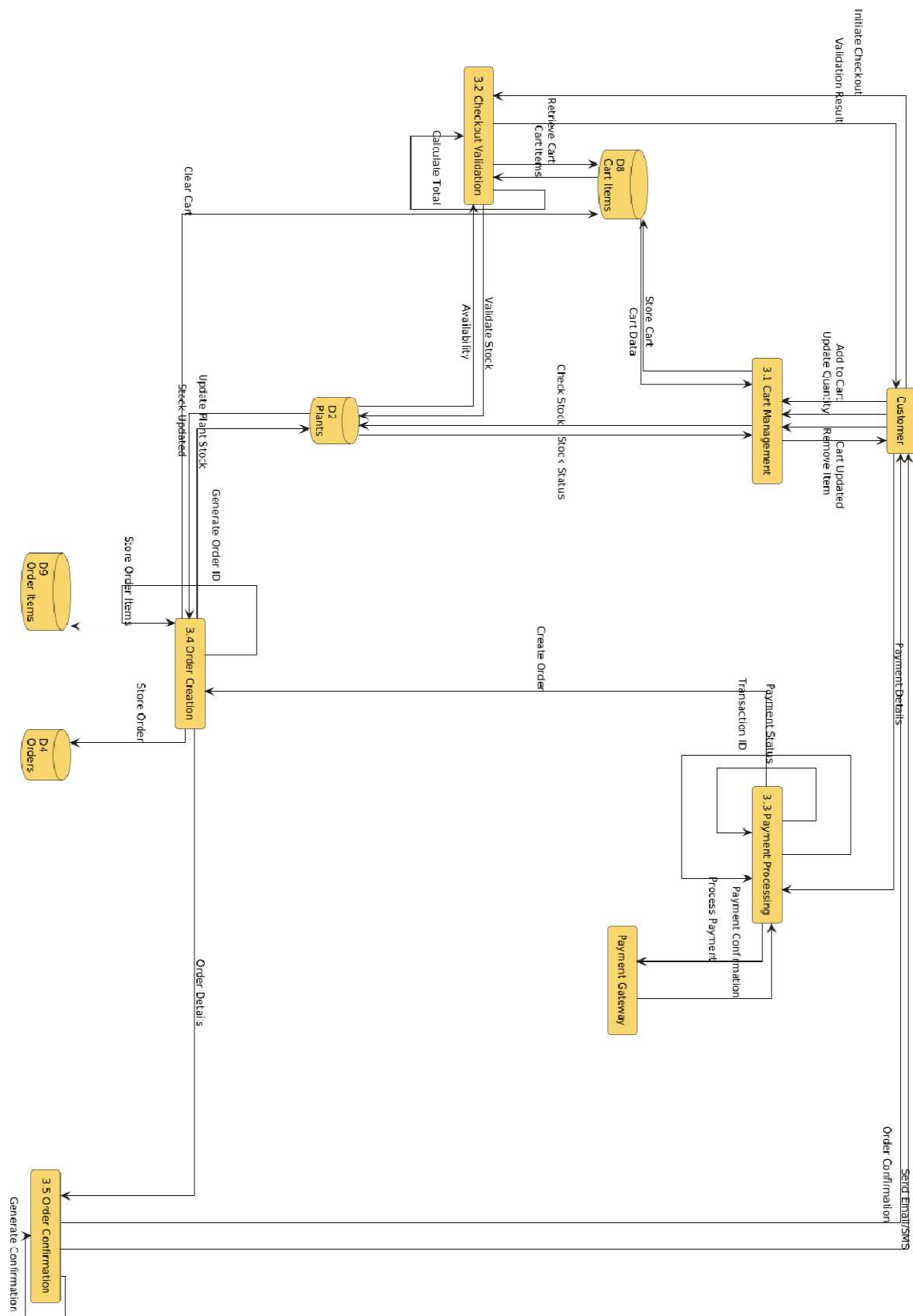


Figure 7.32: Data Flow Diagram – Level 2 (Order Processing)

7.2.6 DFD LVL 2 (Delivery Tracking)

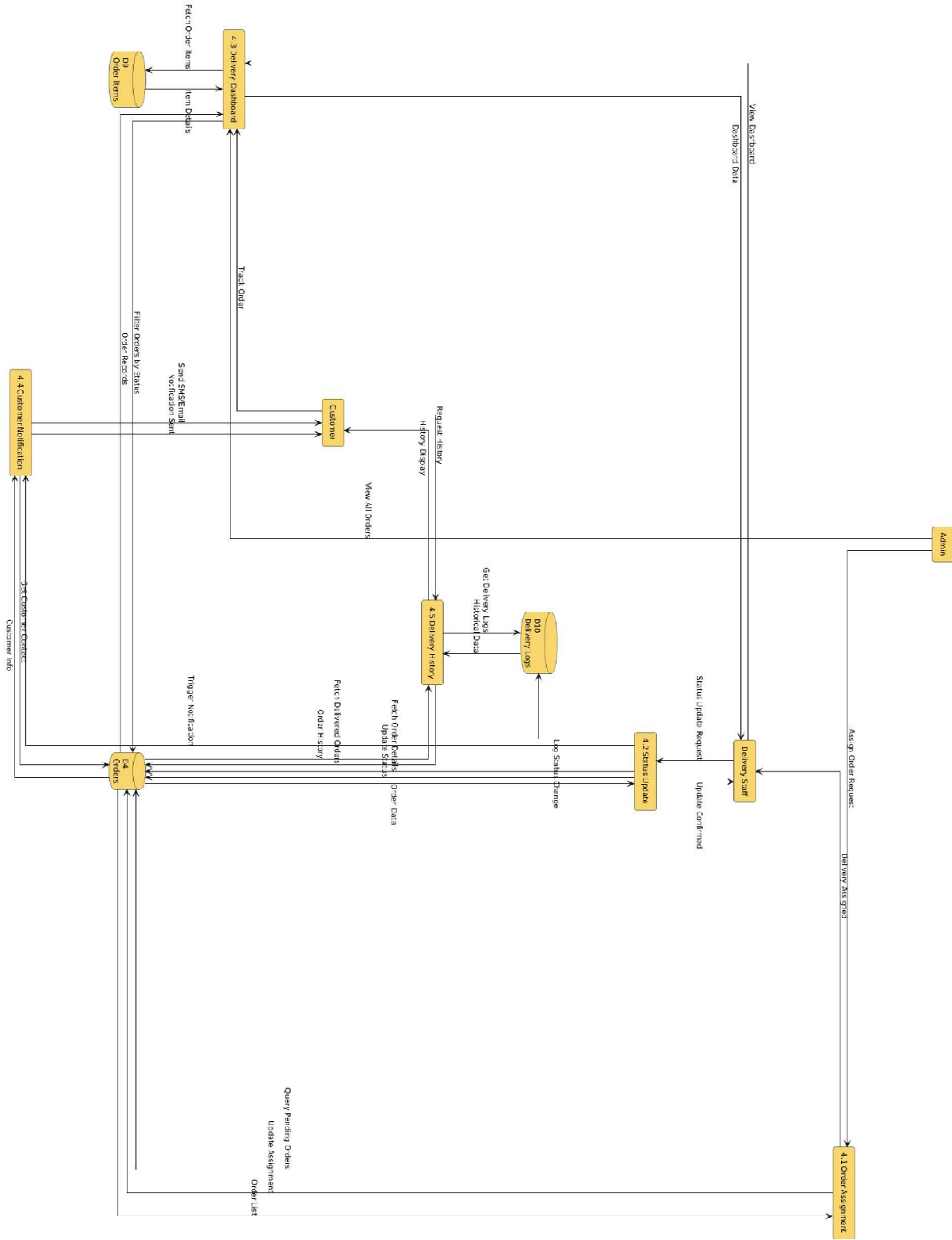


Figure 7.33: Data Flow Diagram – Level 2 (Delivery Tracking)

7.2.7 DFD LVL 2 (Reviews & Q&A)

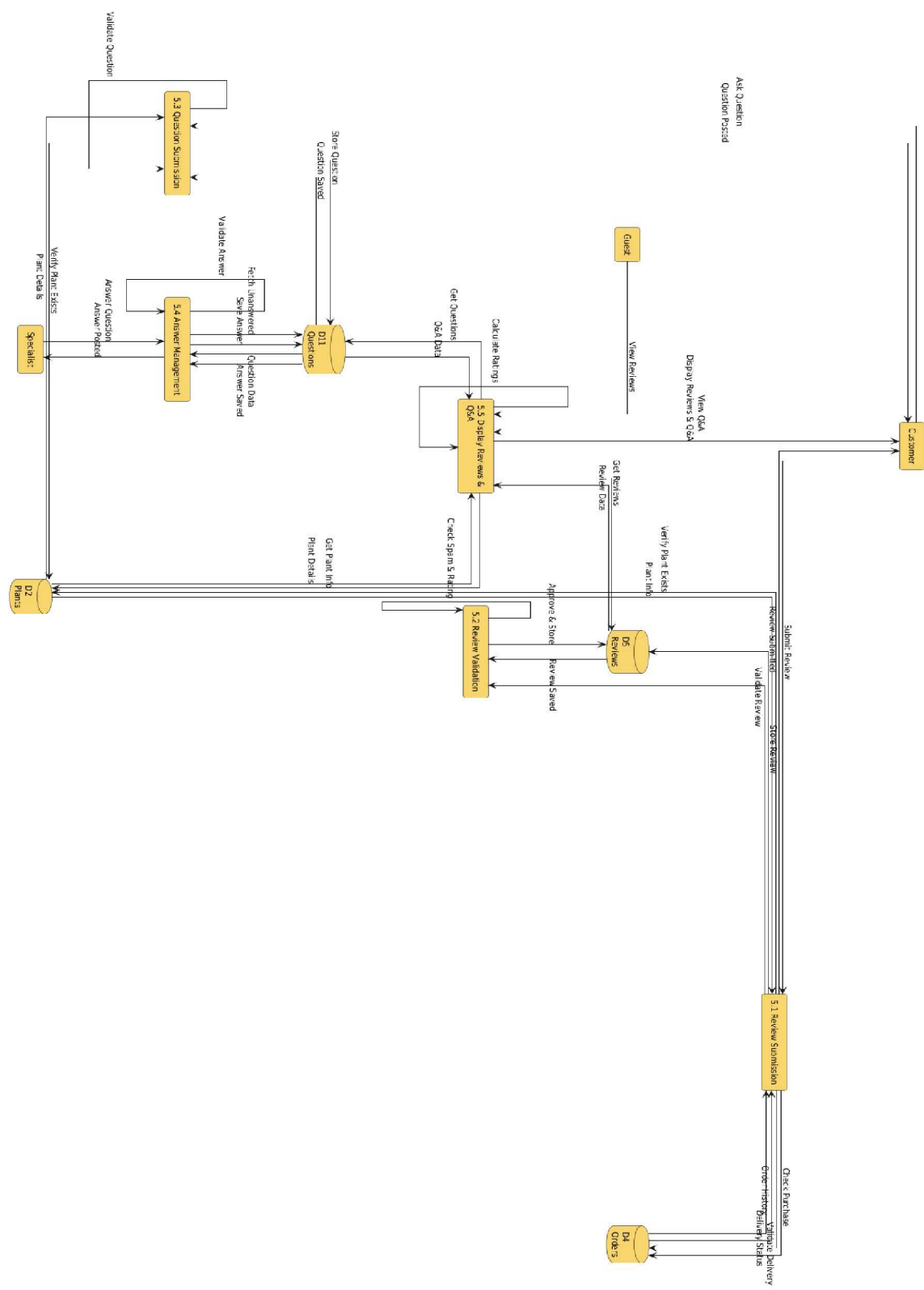


Figure 7.34: Data Flow Diagram – Level 2 (Reviews & Q&A)

7.2.8 DFD LVL 2 (Analytics)

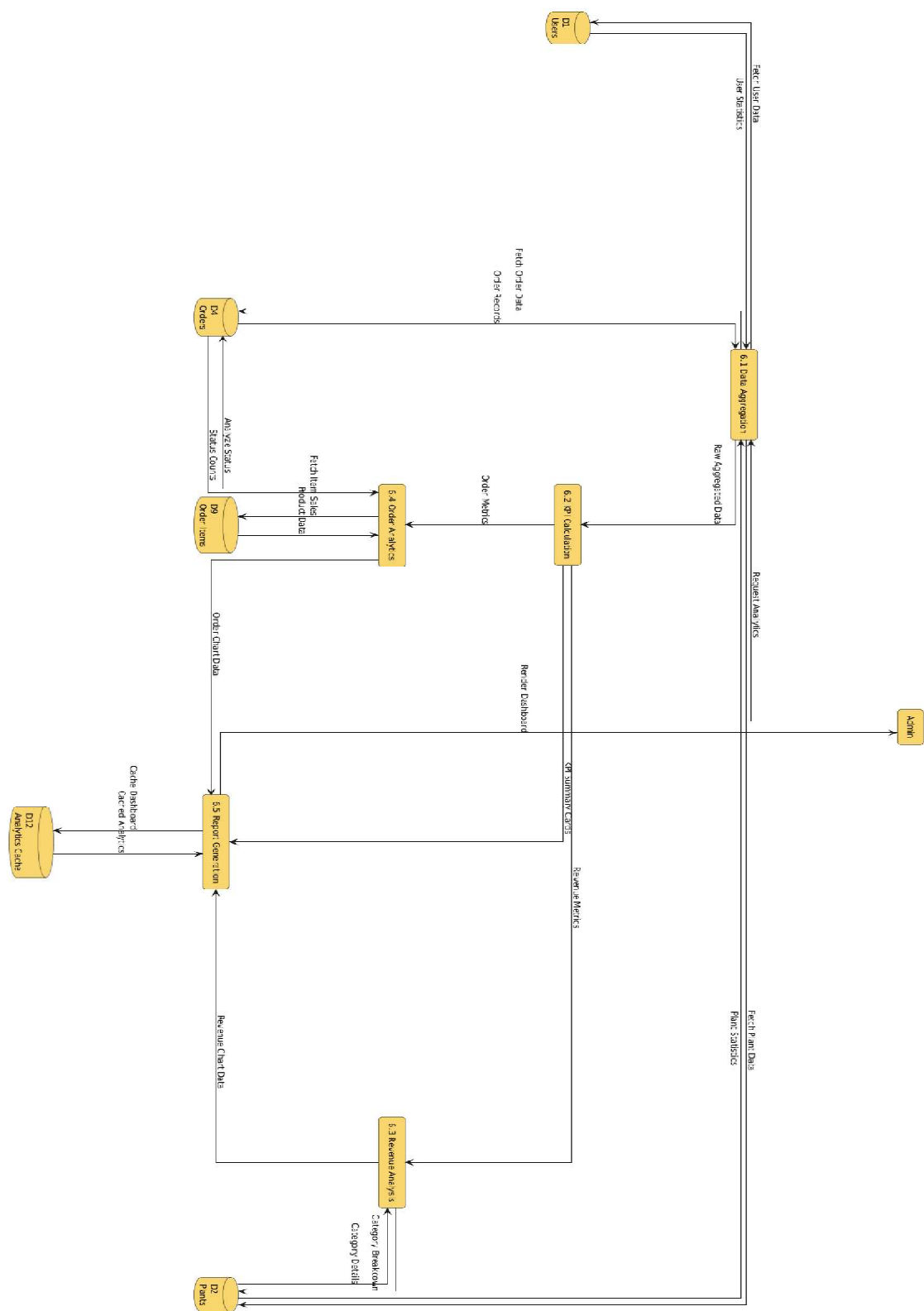


Figure 7.35: Data Flow Diagram – Level 2 (Analytics)

7.3 Database Design

7.3.1 Normalization of Data Fields

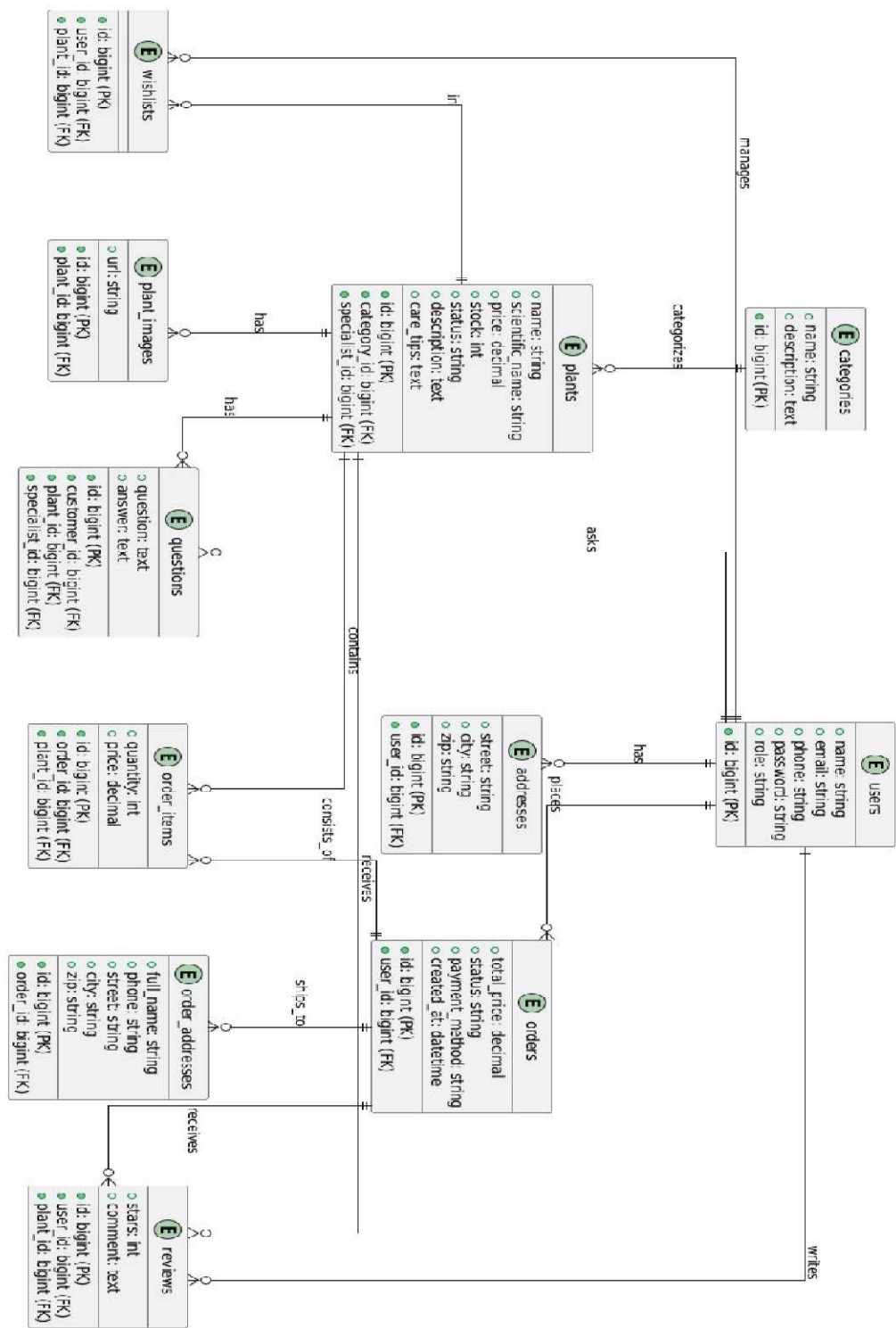


Figure 7.36: Database Normalisation Diagram

7.3.2 Entity Relationship Model

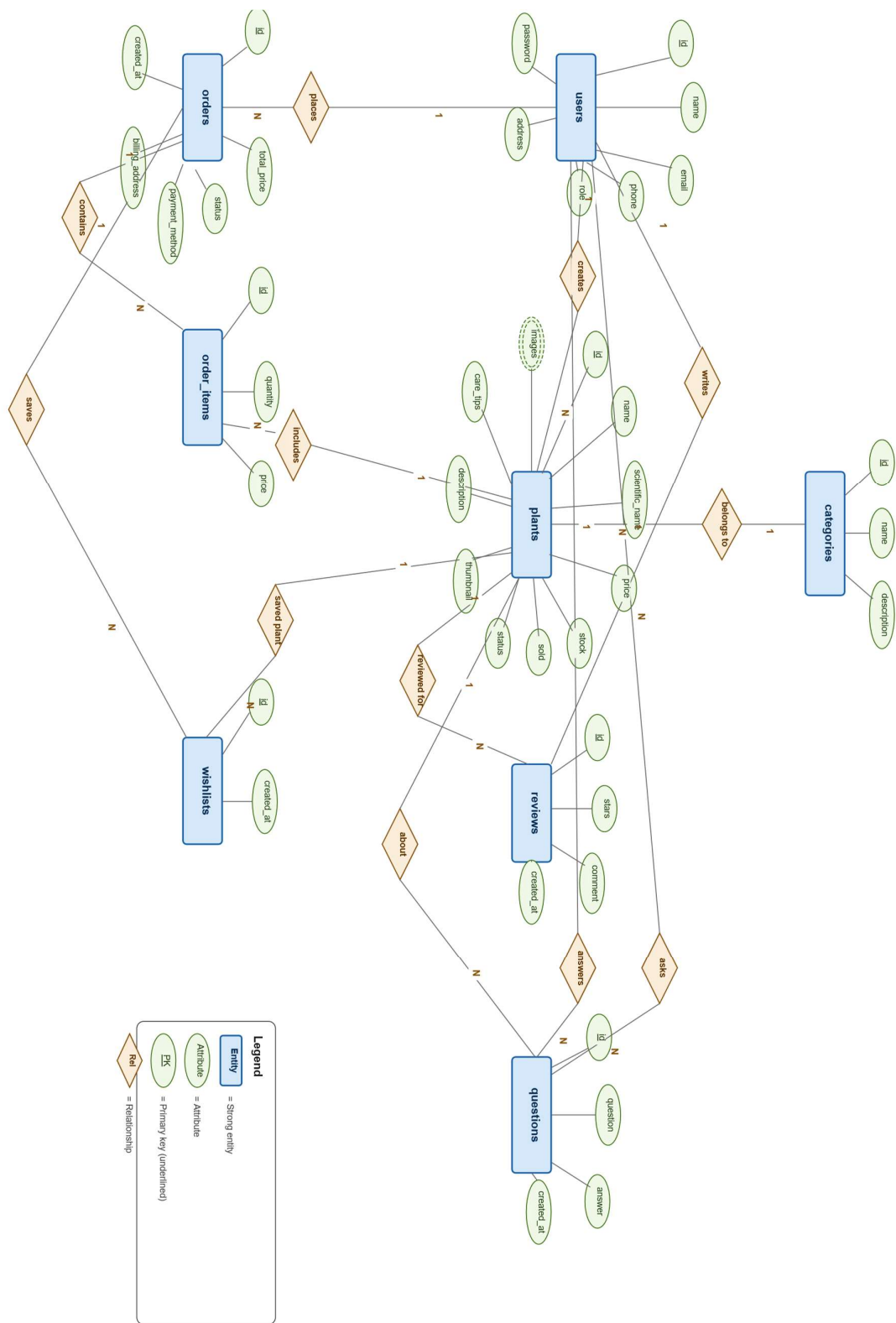


Figure 7.37: Entity Relationship Model (ER Diagram)

7.3.3 Database Table Structure

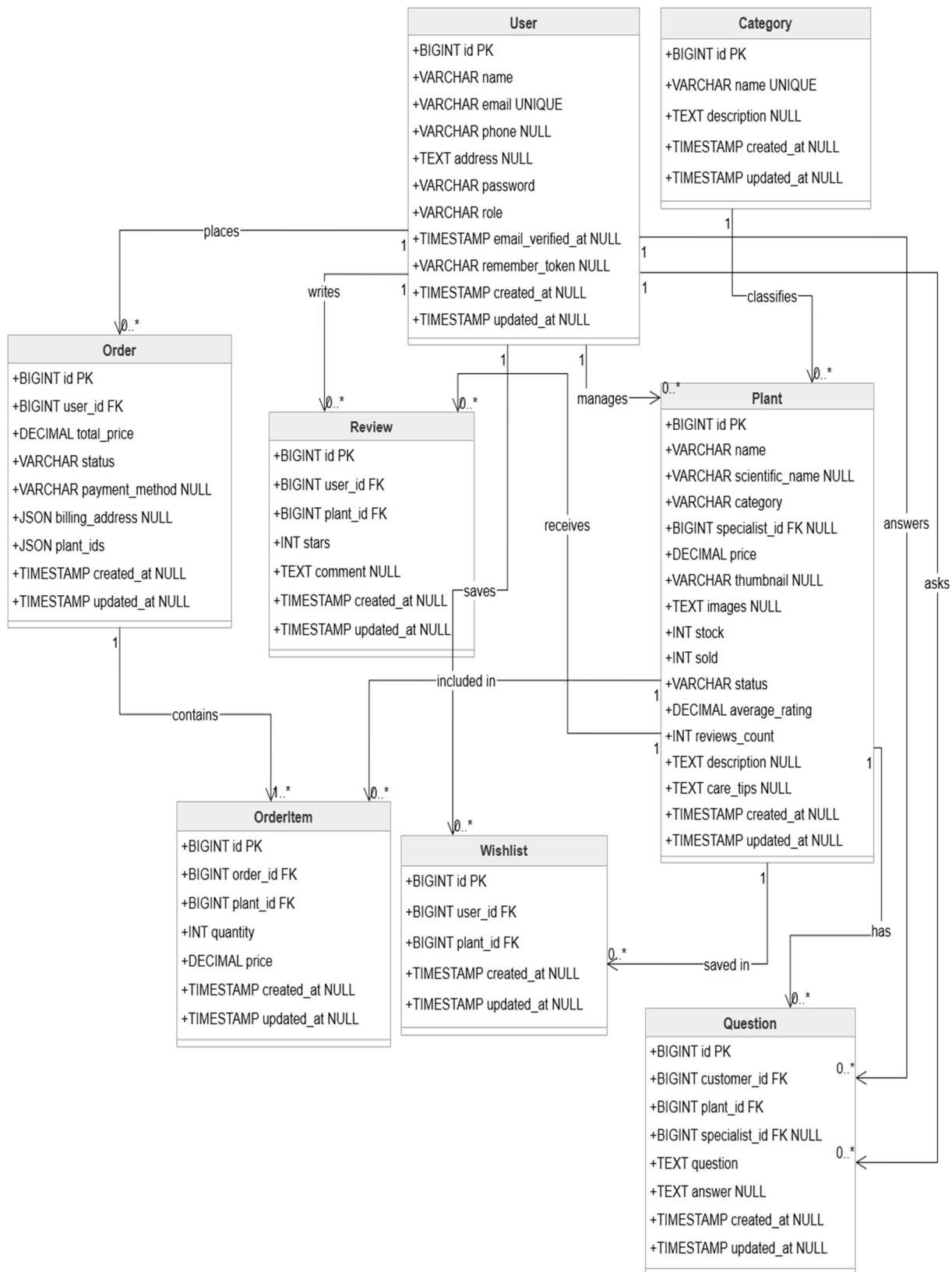


Figure 7.38: Database Table Structure Diagram

Chapter 8.

System Quality and Testing

8.1 System Quality Management

8.1.1 Software Quality Management Process

- **Quality planning:** It is defined through 7 clear objectives. These 7 objectives are functional correctness, API reliability, frontend rendering, data integrity, security, performance and usability. These guides the system development in a structured way and helps maintain overall quality. The objectives ensure that the system behaves correctly under expected conditions and also remains stable when the load increases or when new features are added.
- **Quality assurance:** It is maintained through code reviews, coding standards, static analysis, security, migration and documentation reviews. These practice helps to ensure that the codebase remains clean, consistent and maintainable in the development lifecycle.
- **Quality control:** It is achieved through multiple levels of testing including unit testing integration testing system testing user acceptance testing and regression testing. Each level of testing helps to identify issues at different stages of development and ensures that new changes do not break existing functionality.
- **Quality standards:** It follows TypeScript strict mode, ESLint, WCAG 2.1 AA, and Laravel best practices. These standards ensure that the system meets both technical and usability expectations.

8.1.2 Defect Tracking

The defect tracking follows a structured 7 step lifecycle process which includes identification, logging, severity classification, assignment, resolution, verification and closure using GitHub Issues. The workflow guarantees that every bug is properly documented and tracked. It is also resolved in an organized manner. This improves the system stability. It also reduces the chance of recurring issues.

8.2 Software Test Cases

8.2.1 Test Case Details — PASS/FAIL Status

Authentication & User Management

Table 8.1: Test Cases – Authentication & User Management:

Test ID	Test Description	Status
TC-01	User Registration — Successful registration with valid data	✓ PASS
TC-02	User Registration — Duplicate email returns validation error	✓ PASS
TC-03	User Registration — Password mismatch returns validation error	✓ PASS
TC-04	User Login — Successful login with valid credentials	✓ PASS
TC-05	User Login — Invalid credentials return 401 error	✓ PASS
TC-06	User Login — Non-existent email returns 401 error	✓ PASS
TC-07	List All Users — Admin can retrieve all users	✓ PASS
TC-08	Create User (Admin) — Admin creates a new user successfully	✓ PASS
TC-09	Update User — User profile update succeeds	✓ PASS
TC-10	Delete User — User deletion succeeds	✓ PASS

Plant Management

Table 8.2: Test Cases – Plant Management:

Test ID	Test Description	Status
TC-11	List All Plants — Returns all plant objects	✓ PASS
TC-12	Get Single Plant — Returns plant by ID with all details	✓ PASS
TC-13	Get Single Plant — Non-existent ID returns 404	✓ PASS
TC-14	Create Plant — Successful creation with valid data	✓ PASS
TC-15	Create Plant — Missing fields return validation errors	✓ PASS
TC-16	Update Plant — Price and stock update correctly	✓ PASS
TC-17	Delete Plant — Plant removed successfully	✓ PASS
TC-18	Delete Plant — Non-existent ID returns 404	✓ PASS

Category Management

Table 8.3: Test Cases – Category Management:

Test ID	Test Description	Status
TC-19	List All Categories — Returns all category objects	✓ PASS
TC-20	Create Category — Category created successfully	✓ PASS

Test ID	Test Description	Status
TC-21	Update Category — Category name updated successfully	✓ PASS
TC-22	Delete Category — Category removed successfully	✓ PASS

Order Management

Table 8.4: Test Cases – Order Management:

Test ID	Test Description	Status
TC-23	Create Order — Order created with items, status pending	✓ PASS
TC-24	Create Order — Insufficient stock returns error	✓ PASS
TC-25	List All Orders — Returns all order objects with items	✓ PASS
TC-26	Update Order Status — Status changed from pending to shipped	✓ PASS
TC-27	Update Order Status — Invalid status returns validation error	✓ PASS

Reviews & Questions

Table 8.5: Test Cases – Reviews & Questions:

Test ID	Test Description	Status
TC-28	Get Plant Reviews — Returns reviews with rating and comment	✓ PASS
TC-29	Submit Review — Review created successfully	✓ PASS

Test ID	Test Description	Status
TC-30	Get Plant Questions — Returns questions for a plant	✓ PASS
TC-31	Ask Question — Question submitted successfully	✓ PASS
TC-32	Answer Question — Specialist answer saved successfully	✓ PASS

Wishlist

Table 8.6: Test Cases – Wishlist:

Test ID	Test Description	Status
TC-33	Get Wishlist — Returns wishlist items with plant details	✓ PASS
TC-34	Add to Wishlist — Plant added to wishlist successfully	✓ PASS
TC-35	Remove from Wishlist — Plant removed from wishlist	✓ PASS
TC-36	Check Wishlist Status — Returns wishlisted status as true	✓ PASS

File Upload

Table 8.7: Test Cases – File Upload:

Test ID	Test Description	Status
TC-37	Upload Image — Valid JPEG upload returns path and URL	✓ PASS

Test ID	Test Description	Status
TC-38	Upload Image — Invalid file type returns validation error	✓ PASS
TC-39	Upload Image — File too large returns validation error	✓ PASS
TC-40	Upload Image — No file provided returns validation error	✓ PASS

Frontend Components

Table 8.8: Test Cases – Frontend Components:

Test ID	Test Description	Status
TC-41	Header Component — Navigation bar renders correctly	✓ PASS
TC-42	PlantCard Component — Plant data displays correctly	✓ PASS
TC-43	Home Page — Featured plants grid loads successfully	✓ PASS
TC-44	Plants Listing — Category filter updates plant list	✓ PASS

Frontend User Flow

Table 8.9: Test Cases – Frontend User Flow:

Test ID	Test Description	Status
TC-45	Registration Flow — User registered and redirected	✓ PASS

Test ID	Test Description	Status
TC-46	Registration Flow — Validation errors shown for empty fields	✓ PASS
TC-47	Login Flow — Successful login redirects to home	✓ PASS
TC-48	Login Flow — Invalid credentials show error message	✓ PASS
TC-49	Add to Cart — Plant added, cart badge updates	✓ PASS
TC-50	Cart Page — Correct totals displayed for multiple items	✓ PASS
TC-51	Cart — Quantity increase updates subtotal and total	✓ PASS
TC-52	Checkout — Order created and redirected to confirmation	✓ PASS
TC-53	Empty Cart — Empty state message and CTA displayed	✓ PASS
TC-54	Admin Dashboard — Stats cards display correct data	✓ PASS

Admin Features

Table 8.10: Test Cases – Admin Features:

Test ID	Test Description	Status
TC-55	Admin — User Management table displays correctly	✓ PASS
TC-56	Admin Analytics — Page loads with statistics	✓ PASS

Edge Cases

Table 8.11: Test Cases – Edge Cases:

Test ID	Test Description	Status
TC-57	API Down — Error handled gracefully, no blank page	✓ PASS
TC-58	Network Timeout — Loading indicator and error message shown	✓ PASS
TC-59	Out of Stock — Add to Cart disabled for stock = 0	✓ PASS
TC-60	Page Not Found — 404 page displayed with navigation link	✓ PASS

8.2.2 Test Execution Plan

Table 8.12: Test Execution Plan:

Phase	Test Focus	Timeline	Responsibility
Unit Testing	Controllers, models, components	Sprint 3 (Week 11-12)	Student Developer
Integration Testing	API endpoints, database	Sprint 3 (Week 12)	Student Developer
System Testing	End-to-end workflows	Pre-deployment	Student Developer
UAT	Real user scenarios	Pre-deployment	Supervisor

Chapter 9.

Ethical Consideration

9.1 Ethical Considerations in the Software Development Process

Software that manages personal data, financial transactions, and user roles bears significant ethical responsibilities. It was always considered carefully during design. Although Plantify is an academic prototype, it is developed with strong awareness of ethical dimensions that would be applicable in real-world deployments, and it will guiding future implementations in more responsible way.

9.1.1 Data Privacy and Security

The application collects only essential personal information, such as name, email, phone number, delivery address, and password. All the passwords are hashed using bcrypt, and sensitive credentials are securely stored in the .env file, which is excluded from the version control system. The table below summarizes the main privacy measures and identifies any existing gaps.

Table 9.1: Data Privacy and Security Measures:

Ethical Concern	Safeguard Implemented	Current Gap / Status
Data Minimisation	Registration fields limited to name, email, phone, address, password.	Fully implemented.
Password Storage	bcrypt hashing via Laravel's Hash facade.	Fully implemented.
Access Control	Role-based middleware planned (Admin, Specialist, Delivery).	Not yet implemented critical ethical risk. without RBAC any authenticated user can access other users data
Data Transmission	HTTP in development; designed for HTTPS in production.	HTTPS would be enforced via Let's Encrypt in a live system.

Ethical Concern	Safeguard Implemented	Current Gap / Status
Data Retention	No data retention policy exists yet.	Must be defined before production.
Third-Party Sharing	No analytics, advertising, or data-broker integrations.	Fully implemented.
Breach Notification	No plan currently.	Required for legal compliance (Digital Security Act, etc.).

9.1.2 Intellectual Property

- **Open Source Compliance:** The project utilizes Laravel, Next.js, Tailwind CSS, and MySQL under MIT or GPL licenses, which allow both academic and commercial use with appropriate attribution.
- **Code Ownership:** The entire codebase is original student work, with no proprietary code copied; however, some boilerplate structures were modified.
- **Plant Images:** Prototype images are sourced from free platforms like Unsplash and Pexels. The final production version will require properly licensed images or original photographs.
- **Text Content:** The plant descriptions and care tips are either written manually or rewritten from public domain horticulture references to avoid plagiarism.

9.1.3 User Impact

The platform creates both positive and negative impacts, each of which must be addressed carefully.

Positive Impacts

- Provides 24/7 access for browsing and ordering without time or geographic restrictions.
- Offers small nurseries a low-cost digital storefront.
- Makes expert knowledge accessible to all users.

- Ensures delivery workers have clear task lists without unnecessary tracking systems.

Potential Negative Impacts

- The digital divide may exclude users without smartphones or internet access.
- This system dependency could disrupt sales during outages if no manual backup exists.
- Automation might reduce the need for certain administrative roles.

Mitigation: The system should function as a support tool rather than a complete replacement for physical nurseries. Offline ordering should remain available, and the interface must be simple for non-technical users.

9.1.4 Bias and Fairness

Table 9.2: Bias and Fairness Analysis:

Bias Type	How It Appears	Mitigation
Access Bias	Entirely web-based; excludes users without internet or devices.	Acknowledge limitation; future USSD or mobile app could help.
Economic Bias	No differential pricing; all customers see the same prices.	Not an issue in the current system.
Geographic Bias	Delivery limited to defined radius; customers outside it cannot order.	Clearly communicate delivery zones before cart building.
Rating Bias	Any user can submit review, so fake review possible.	Future: verify purchases before allowing reviews; admin moderation.
Role Rigidity	Users cannot hold hybrid roles (e.g., specialist + delivery).	Extend RBAC to support multiple roles later if needed.

9.1.5 Responsibility to Stakeholders

Table 9.3: Responsibility to Stakeholders:

Stakeholder	Responsibility	Plantify Implementation
Customers	Protect data, transparent pricing, reliable service.	bcrypt passwords, clear cart totals, responsive UI. RBAC still needed.
Specialists	Provide efficient inventory tools, manageable Q&A workload.	Full CRUD plant/category interface and Q&A dashboard.
Delivery Personnel	Respect privacy, clear assignments, no surveillance.	Shows assigned orders only; no GPS tracking.
Administrators	Accurate oversight data, system security.	Basic admin dashboard; RBAC still required for proper security.
Academic Institution	Honest, original, well-documented work.	All code and reports are verifiable and original.
Society	Reduce waste, promote sustainable consumption.	Paperless, lightweight, promotes plant cultivation.

9.1.6 Professional Responsibility

In accordance with the IEEE/ACM Code of Ethics, the developer commits to following principles:

- **Competence:** This involves effectively learning Laravel, Next.js, and MySQL, and testing this system using these 60 test cases.
- **Honesty:** Documenting system limitations clearly, including the absence of Role-Based Access Control (RBAC), a payment gateway, and automated tests.
- **Quality:** Adhering to coding standards such as PSR-4, ESLint, proper error handling, and maintaining version control discipline.

- **Confidentiality:** This takes this full responsibility for these design decisions and this identified risks within this codebase.
- **Professional Growth:** Leveraging the project as a learning opportunity for full-stack development and ethical considerations.
- **Accountability:** This takes this full responsibility for these design decisions and this identified risks within this codebase.

9.2 Sustainability in the Software Development Process

Sustainable software should be economically viable, socially fair, and environmentally responsible. Plantify addresses all three pillars of sustainability.

9.2.1 Economic Sustainability

The project incurs almost no development costs since it is developed by students using free and open-source tools. In a real-world deployment, costs remain very low and cloud hosting options are affordable. The system employs a modular architecture and uses popular frameworks like Laravel and React. It reduces dependency on any single vendor, which facilitates long-term maintenance. This makes the system suitable for small nursery businesses that cannot afford expensive enterprise software.

9.2.2 Social Sustainability

- **Local Business Support:** Assists small nurseries in establishing an online presence at a low cost.
- **Inclusion:** This interface is designed to be this simple, enabling these non-technical users to navigate it easily; this future support for Bangla will enhance this accessibility.
- **Fair Labor:** Delivery personnel are treated fairly without surveillance or exploitative tracking systems.
- **User Autonomy:** No dark patterns are used, allowing users to browse freely without the pressure to make a purchase.

- **Well-being:** This promotes this plant ownership, which can enhance this mental health and improve these living environments.

9.2.3 Environmental Sustainability

- **Paperless System:** Digital records eliminate the need for paper-based inventory and billing.
- **Reduced Travel Emissions:** Online browsing decreases unnecessary physical visits to nurseries.
- **Waste Reduction:** That real-time inventory tracking minimizes that overstock and plant wastage.
- **Efficient Infrastructure:** A lightweight system design reduces server energy consumption.
- **Positive impact:** selling plants contribute to environmental benefits, including carbon absorption and biodiversity improvement. It also help improving green balance in urban areas.

Chapter 10.

Conclusion

10.1 Brief Overview of the Project

Plantify is a nursery plant management application. It is built using Laravel 12 Next.js 16 and MySQL. The application has four types of users. They are Customer, Specialist, Delivery Personnel and Admin. Plantify provides one platform for managing plant inventory selling plants online getting advice from specialists tracking orders and analyzing business performance.

The project planning and estimation process, centred on IFPUG Function Point Analysis, delivered the following key results:

- **Size:** 160 Adjusted Function Points (AFP)
- **Effort:** 3.0 person months (solo developer)
- **Duration:** 13 weeks, organised into 4 Agile sprints
- **Risk Management:** 8 identified risks with a full RMMM plan
- **Cost:** 3,800 – 63,800 TK (mostly free software, optional hardware cost)
- **Requirements:** 24 user requirements, 33 functional requirements, non-functional requirements
- **Testing:** 60 test cases across backend API and frontend components, all passing
- **Documentation:** 10-chapter report covering feasibility analysis, planning, design, quality, lifecycle, and ethics.

10.2 Proposed System Benefits

Plantify makes life easier for everyone at the nursery. It streamlines the day-to-day work and gives customers a far better experience.

For the nursery business Plantify gets rid of the paper system and uses a real-time digital platform instead. The inventory is managed from one place that automatically updates the stock when an order is placed. This means we do not oversell and we do not have to count everything by hand. It makes the system more reliable. Helps us keep track of our inventory better.

The online storefront lets customers buy from us from over the country at any time of day. We are not limited to people who can come to our physical shop. The analytics dashboard shows us how money we are making from orders and which plants are the most popular. This helps us make decisions based on what is really happening.

Using Plantify also reduces the amount of work we have to do as administrators. The system for reviewing and answering questions helps our customers trust us more. Plantify is really helpful for the nursery business because it makes everything easier to manage and it helps us reach customers.

Customers get easy access to the full plant catalog with detailed care tips and reviews. They can shop from any device with obvious pricing, wishlist support, and full order tracking. The review and question system also helps customers learn more before buying.

Specialists get a dedicated dashboard for plant management categories and customer questions. All CRUD operations are centralised, and image upload is simple. Their answers remain permanently visible, which reduces repeated queries and improves knowledge sharing.

Delivery personnel only see assigned orders, which removes confusion. A simple interface allows them to update order status from pending to delivered, which keeps the system updated without manual calls.

Administrators get full control over users, plants, orders, and categories from one dashboard. An analytics system helps decision-making for inventory marketing and resource planning. Role-based management ensures proper control across the system.

10.3 Limitations of the Project

Plantify works fine as a full academic prototype, but it still has a few limitations before real production use.

The primary issue is the absence of authentication middleware and role-based access control, which results in insecure endpoints and potential privilege escalation risks. The payment

gateway isn't added, so everything works only with offline orders. The system lacks a password reset mechanism and does not provide email notifications for order updates.

The cart is saved in localStorage, so it can disappear on other devices. The search and filtering features are basic, and the plant image gallery has limited functionality. There's no automated testing, so bugs can increase in future updates.

Finally, the system is designed for a single nursery and does not support a multi-vendor marketplace structure.

10.4 Practicum and Its Value

Working at Globix Tech was a great practical experience, I got to do actual software engineering, not just theory. I developed more than 20 API endpoints. I created 8 table database schemas. I set up a role-based frontend structure and managed secure file uploads. It connected what I learned in school with real-world practice.

These technical skills improved significantly through working with Laravel MVC architecture, Eloquent ORM, REST API design, Next.js App Router, TypeScript, and Tailwind CSS. I developed 20+ API endpoints, created 8 table database schemas, implemented a role-based frontend structure, and handled secure file uploads. I also learned IFPUG Function Point Analysis and calculated 160 AFP with a 13-week schedule.

Professional skills also got better. I improved my thinking and documentation skills by figuring out system requirements from existing code. Risk management practice helped me identify and plan strategies to reduce risks. Evaluating ethics using IEEE and ACM principles made me more aware of issues like privacy, fairness and sustainability.

Writing a full 10-chapter report improved technical writing ability.

For Globix Tech, the project delivered a working e-commerce system, reusable documentation template, and trained a developer familiar with the company's tech stack. FPA analysis, RMMM plan, and test suite also serve as a reusable reference for future projects.

10.5 Future Plan

Plantify future development is structured into three progressive phases:

Table 10.1: Future Plan – Three-Horizon Roadmap:

Phase	Timeline	Team Size	Key Deliverables	Cumulative Investment
Short-Term	Weeks 13–16	1 developer	Authentication middleware (Sanctum), role-based access control, PHPUnit test suite, password reset, server-side cart persistence, email notifications	+1 month (post-MVP)
Medium-Term	Months 4–9	1 developer	Payment gateway (bKash/Nagad/SSLCommerz), mobile-first responsive optimisation, full-text search, multi-image gallery, advanced analytics dashboard, Docker containerisation	+6 months
Long-Term	Months 9–18	3–5 developers	Multi-vendor marketplace, native mobile apps (React Native/Flutter), AI-powered plant assistant, real-time GPS delivery tracking, loyalty programme, Bangla language support, CI/CD pipeline, performance tuning	+12 months

Reference:

1. [GeeksforGeeks \(2024b\) *Software Project Management Plan \(SPMP\) - Software Engineering.*](#)
2. [GeeksforGeeks \(2024b\) *MVC Framework Introduction.*](#)
3. [GeeksforGeeks \(2025a\) *Software Quality Assurance \(SQA\) Set 2 - Software Engineering.*](#)
4. [GeeksforGeeks \(2025c\) *Steps to Create UML Diagrams.*](#)
5. [GeeksforGeeks. \(2026\) *Agile and Waterfall - Importance of Iterative Building.*](#)
6. [Laravel \(2025\) *Laravel 12.x Documentation.*](#)
7. [MySQL \(2026\) *MySQL 8.0 Reference Manual.*](#)
8. [Next.js \(2026\) *Next.js 16 Documentation.*](#)
9. [React \(2024\) *React 19 Reference.*](#)
10. [Tailwind CSS \(2025\) *Tailwind CSS v4 Documentation.*](#)
11. [TypeScript \(2026\) *TypeScript 5.x Handbook.*](#)